

Sistemas Operativos

Cuando hablamos de usuario, nos estamos refiriendo a usuario de programador.

Cuando hablamos de programa nos referimos al binario en el disco (programa es el que se carga en la memoria). Nos referimos como proceso al programa que es ejecutado.

The difference between a process and a program is subtle, but absolutely crucial. An analogy may help you here. Consider a culinary-minded computer scientist who is baking a birthday cake for his young daughter. He has a birthday cake recipe and a kitchen well stocked with all the input: flour, eggs, sugar, extract of vanilla, and so on. In this analogy, the recipe is the program, that is, an algorithm expressed in some suitable notation, the computer scientist is the processor (CPU),

and the cake ingredients are the input data. The process is the activity consisting of our baker reading the recipe, fetching the ingredients, and baking the cake.

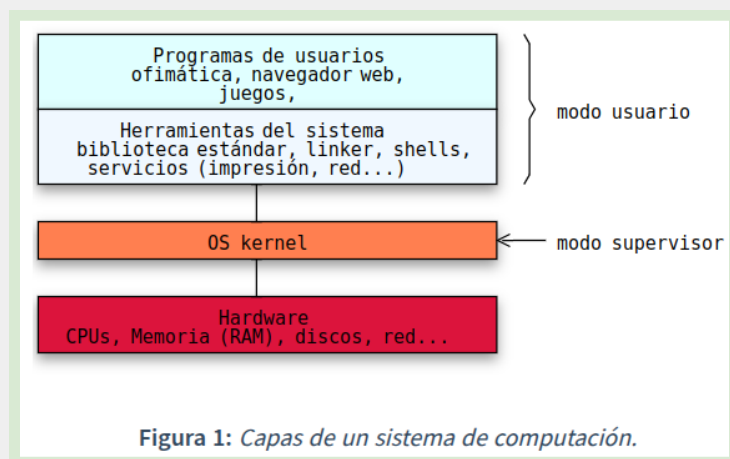
Now imagine that the computer scientist's son comes running in screaming his head off, saying that he has been stung by a bee. The computer scientist records where he was in the recipe (the state of the current process is saved), gets out a first aid book, and begins following the directions in it. Here we see the processor being switched from one process (baking) to a higher-priority process (administering medical care), each having a different program (recipe versus first aid book). When the bee sting has been taken care of, the computer scientist goes back to his cake, continuing at the point where he left off.

Vamos a usar la arquitectura RISC-V.

RISC: tiene pocas instrucciones, están basados en un conjunto muy reducido de instrucciones pero nos dan muchos registros, entonces la programación es más simple. Disponemos de 32 registros creo. Se hace hasta más fácil para los compiladores esta arquitectura.

CISC: por ejemplo los procesadores de Intel, tienen muchas instrucciones y pocos registros.

Un sistema de computación está formado por múltiples componentes de *hardware* y *software*. La única manera de lidiar con la complejidad de estos sistemas es *modularizar*, es decir, crear pequeños componentes interconectados para construir sistemas más complejos. La siguiente figura muestra una estructura típica de componentes en diferentes capas de abstracción un sistema de computación actual de propósitos generales.



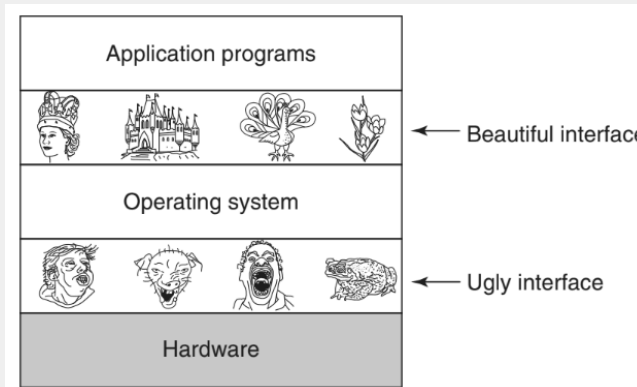
El hardware consiste de chips, boards, discos, un teclado, un monitor y objetos físicos similares.

Arriba del hardware está todo lo que es el software. La mayoría de las computadoras tienen dos modos de operación: modo kernel / supervisor y modo usuario.

El SO se ejecuta en modo kernel, en donde tiene acceso completo a todo el hardware y puede ejecutar cualquier instrucción que se pueda ejecutar. El kernel es el núcleo del sistema operativo. El modo supervisor indica que la CPU lo va a ejecutar con una prioridad mayor que los demás programas. El resto del software se ejecuta en modo usuario, en donde solamente se pueden ejecutar un conjunto de instrucciones.

Un SO es básicamente un software que se ejecuta en modo kernel. Una de las principales tareas del SO es ocultar el hardware y, en su lugar, presentar a los programas (y a sus programadores) abstracciones limpias, elegantes y consistentes con las que trabajar.

Su tarea principal es realizar un seguimiento de qué programas utilizan qué recurso, conceder solicitudes de recursos, contabilizar el uso y mediar en solicitudes conflictivas de diferentes programas y usuarios.



Antes (principios de 1950), no existía el concepto de sistema operativo. Cada programa debía ser auto-contenido, o sea que debía contener todas las instrucciones necesarias aún para las operaciones rutinarias de entrada-salida, por lo que cada programador debe conocer todos los detalles del sistema. Para ejecutar un programa tenías que escribir todo, desde cómo acceder y buscar algo en tal archivo, etc. No había un sistema operativo que te hacía todo y vos con usar las instrucciones open, read, write y close ya podías acceder al archivo, tenías que hacerlo a mano.

Los sistemas denominados por lotes o batch systems permitían cargar programas y bibliotecas (contenedores de rutinas). Los programadores depositaban sus programas y datos de entrada (jobs). Luego un operador los agrupaba (en lotes) y los ordenaba en una cola para que el sistema los cargara y ejecutara en secuencia. Cada lote era precedido por una secuencia de comandos que especificaba las bibliotecas requeridas. El sistema operativo (SO) las cargaba en memoria si no habían sido cargadas previamente.

En estos tipos de sistemas cada programa tenía acceso a todos los recursos (CPU, memoria, dispositivos de entrada-salida, etc) del sistema.

Sistemas operativos multitarea

Las operaciones de entrada-salida dejan demasiado tiempo a la CPU ociosa, la cual sólo debía esperar (en un ciclo, verificando algún registro del controlador del dispositivo) a que esas operaciones finalicen.

Con el objetivo de maximizar el uso de CPU, los próximos sistemas operativos evolucionaron para cargar en memoria varios programas (procesos) y ejecutarlos concurrentemente asignando la CPU a otra tarea cuando la anterior iniciaba una operación de entrada-salida. Estos sistemas se denominan sistemas de multiprogramación o multitareas.

De esta manera se logra que se utilice la CPU y las operaciones de E/S en paralelo.

Generalmente los controladores de dispositivos generan interrupciones al ocurrir un evento como la finalización de una operación de lectura o escritura de un bloque en el disco.

Ante una interrupción recibida por la CPU, ésta hace que el flujo de ejecución salte a una rutina de atención o trap handler. De ese modo, el kernel retoma el control.

En estos sistemas, las tareas cooperan cediendo la CPU en cada llamada al sistema por lo que se conocen como **SO multitarea cooperativa**. Las primeras versiones de MS-Windows tenían esta característica.

Cuando un proceso hace una llamada al sistema, es decir, leer, escribir, imprimir por pantalla algo, etc, provoca una interrupción recibida por la CPU, donde luego se le otorga el CPU a otro programa para que se siga o comience a ejecutarse.

Sin embargo, estos sistemas tenían un problema: si bien las tareas cooperan entre sí, pero a su vez liberan el CPU cuando llaman al sistema e interrumpen. Pero qué pasa si tenés un programa que hace un `while true` y nunca hace una llamada al sistema? Este proceso monopolizó la CPU y el SO nunca obtuvo más el control. O por ejemplo un proceso que tienen que hacer un cálculo que le lleva mucho tiempo hacerlo, 2 días por ejemplo, en donde no hace ninguna llamada al sistema. Este proceso entonces monopoliza el CPU por dos días enteros y no deja a otro programa ejecutarse.

La solución que se le da a esto es los **SO multitarea preventivos**. En estos, si un proceso no hizo nunca una llamada al sistema y ha estado usando el CPU por demasiado tiempo ya, el SO se la quita. Cada proceso tiene un intervalo del tiempo de uso del CPU y luego de que su tiempo se termine, tiene que liberarla para que otro proceso la use, es como un timer que tienen. El SO captura la interrupción del reloj y le saca el CPU a ese proceso y se lo asigna a otro.

Contexto de ejecución

Cuando la CPU está ejecutando código de un proceso, está operando en modo usuario. Al ocurrir una interrupción, la CPU salta a ejecutar el interrupt handler en modo kernel.

Dentro del código del kernel hay que identificar si el código se está ejecutando en el contexto de un proceso, es decir que una interrupción ocurrió mientras la CPU estaba en modo usuario o no (el que fue interrumpido fue código del kernel). En éste último caso, en el kernel, no existe la noción de proceso corriente.

Cambios de contexto (Context Switch)

Cuando el kernel de un SO decide quitar la CPU a un proceso y asignarla a otro se realiza un *context switch* y este consiste en los siguientes pasos:

1. Guardar (salvar) el estado (registros) de la CPU en el descriptor del proceso.
2. Recuperar en los registros de la CPU el estado guardado del próximo proceso.

(Después de guardar el estado, decide a qué proceso le va a dar la CPU, entonces luego se recuperan el estado en el cual se le sacó el CPU, sus registros).

Esto producirá un salto de un punto de ejecución de un hilo de ejecución al otro ya que cambian el program counter (pc) y el stack pointer (sp).

En el capítulo de gestión de procesos se describe este mecanismo en mayor detalle.

El **Program Counter (PC)**, o contador de programa, es un registro interno de la CPU que guarda la dirección de la próxima instrucción a ejecutar. Cada vez que se procesa una instrucción, el PC se actualiza (generalmente se incrementa) para apuntar a la siguiente instrucción en la secuencia de ejecución. Los saltos o llamadas a subrutinas alteran este orden.

Por otro lado, el **Stack Pointer (SP)**, o puntero de pila, es un registro especial que apunta a la posición actual de la cima de la pila en la memoria. La pila es una estructura de datos que opera bajo el principio LIFO (Last In, First Out) y se utiliza para almacenar información temporal, como direcciones de retorno en subrutinas, variables locales y otros datos esenciales durante la ejecución del programa. Mediante operaciones de "push" (para agregar datos) y "pop" (para extraer datos), el SP ayuda a gestionar este almacenamiento de manera dinámica, facilitando la correcta ejecución de las llamadas a función y el retorno de información.

Protección

Estos sistemas requieren de algún mecanismos de protección:

- **Confinamiento:** cada proceso debe acceder sólo a su propio espacio de memoria. Como tenemos varios procesos usando memoria, deberíamos tener cada procesos accediendo a su propia memoria y que no esté permitido que uno acceda a la memoria del otro.
- **Control:** los procesos no deberían poder ejecutar ciertas instrucciones privilegiadas, como deshabilitar interrupciones o usar registros especiales que permitan quitarle el control al kernel. Es necesario restringir las instrucciones que pueden ejecutar los procesos.

Instrucciones privilegiadas: sólo debería ejecutarlas el código del kernel, no el código de un usuario. Por ejemplo, deshabilitar la interrupción. Haciendo eso le quitamos el control al kernel (este solo vuelve a tomar el control cuando hay una interrupción), podríamos quedarnos con la CPU solo para nuestro programa haciendo esto. Un proceso común no debería poder ejecutar esta instrucción. Otro ejemplo son instrucciones de acceso a registros especiales o espacios de memoria especiales. El kernel se ejecuta en un modo privilegiado donde puede ejecutar todo. Los procesos de usuarios en cambio no, no tienen este privilegio.

El kernel es un binario independiente y auto-contenido, porque no tiene dependencias de ningún tipo. Comúnmente tiene acceso a todos los recursos del sistema.

Cuando hagamos el kernel para imprimir por pantalla vamos a necesitar el código para imprimir por pantalla y ponerlo en el kernel.

La protección de memoria requiere de un mecanismo especial para que una tarea requiera un servicio del SO ya que no puede simplemente hacer una llamada a una función del kernel (ya que son programas diferentes). Los programas normales (como un juego o un editor de texto) no tienen permiso directo para tocar el hardware o meterse en áreas sensibles de la memoria, porque si lo hicieran, podrían causar errores graves o incluso dañar el sistema entero. Esto se llama protección de memoria: el SO separa lo que puede hacer un programa de usuario de lo que hace el kernel.

Pero a veces, un programa necesita algo que solo el kernel puede hacer, como escribir en un archivo o usar la red. El problema es que el programa y el kernel son como dos mundos distintos: no están "conectados" directamente, así que el programa no puede simplemente "llamar" al kernel como si fuera una función normal de su propio código.

Es necesario que el hardware disponga de un mecanismo para acceder a los servicios del SO como por ejemplo, una instrucción `trap` (trampa). De esta manera, los programas se podrán comunicar y acceder a los servicios del SO.

Las aplicaciones no están enlazadas al código del kernel, (el kernel ofrece servicios basicamente), entonces necesitamos de un mecanismo para que un programa salte y acceda a los servicios del SO, para eso usamos la instrucción `trap`. Esta instrucción provoca un salto a una rutina de atención que forma parte del kernel, entonces el kernel sabe lo que le está pidiendo.

Una **llamada al sistema (syscall)** es una función que permite a los procesos solicitar un servicio o recurso al kernel del SO. La interfaz de un SO comúnmente se describe como el conjunto de syscalls que ofrece a las aplicaciones. La interfaz de un SO es el conjunto de los syscall.

En una syscall el flujo de ejecución del proceso se interrumpe saltando del espacio de memoria de un proceso al espacio del kernel. Luego, el kernel procesa el syscall y retorna al espacio de usuario del proceso el cual continúa su ejecución.

Durante el procesamiento de una syscall el kernel puede decidir quitarle la CPU al proceso corriente y continuar la ejecución de otro. Generalmente, en un salto al kernel, la CPU cambia su nivel de privilegio o modo de operación. La mayoría de las CPU modernas incluyen al menos dos modos de ejecución: modo kernel (o supervisor) y modo usuario.

Cualquier computadora con una sola CPU solo puede ejecutar una instrucción a la vez. Si un proceso ejecuta un programa de usuario en modo usuario y necesita un servicio del sistema, como leer datos de un archivo, debe ejecutar una instrucción **trap** para transferir el control al sistema operativo. El sistema operativo, al inspeccionar los parámetros, determina qué necesita el proceso que realiza la llamada. Después, ejecuta la llamada al sistema y devuelve el control a la instrucción que le sigue. En cierto sentido, realizar una llamada al sistema es como realizar un tipo especial de llamada a un procedimiento: solo las llamadas al sistema entran en el kernel, mientras que las llamadas a procedimiento no. Para entender mejor cómo funciona esto, vamos a ilustrar un ejemplo.

Tenemos la system call read. Esta tiene tres parámetros: el archivo, el buffer y el número de bytes a leer. Como todas las system calls, estas pueden ser invocadas desde un programa C al invocar un library procedure con el mismo nombre que la syscall: read. Una llamada desde el program C se puede ver así:

```
count = read(fd, buffer, nbytes);
```

Esta read en C va a ser la que haga la llamada a la syscall read. Luego, se ejecuta una instrucción trap para cambiar el modo, es decir, se pasa de modo usuario a modo kernel y se comienza la ejecución en el kernel. Una vez completado su trabajo, el control puede devolverse al library procedure de espacio de usuario en la instrucción posterior a la instrucción TRAP. Este procedimiento regresa al programa de usuario de la forma habitual en que regresan las llamadas a procedimientos.

La CPU ejecuta el código del núcleo del SO en modo kernel o supervisor que es más privilegiado que el modo usuario en el cual ejecutan los procesos de usuario. En modo kernel la CPU permite la ejecución de instrucciones privilegiadas y acceder a registros de dispositivos, habilitar y deshabilitar interrupciones, entre otras. Estas instrucciones privilegiadas no son accesibles en modo usuario.

En la arquitectura que vamos a usar RISC-V vamos a tener un modo de ejecución más (machine mode), aparte del kernel y usuario mode.

Cuando se ejecuta un proceso que pide al usuario información, por ejemplo, insertar datos por medio del teclado, a este proceso se lo manda a dormir hasta que el usuario responda, y el CPU se lo da a otro proceso mientras este duerme.

Sistemas de Tiempo Compartido

A medida que se incorporaron dispositivos interactivos como las consolas (teclado + pantalla), se comenzaron a desarrollar aplicaciones interactivas, las cuales permitían a los usuarios ingresar datos y visualizar resultados en una sesión de ejecución de un programa.

En un sistema con **multi-programación** una tarea puede monopolizar la CPU o hacer que el sistema sea muy poco interactivo ante usuarios (éstos podrían tener demoras significativas si la tarea en ejecución no hace una llamada al sistema).

Para evitar este problema el kernel puede quitarle (preempt) la CPU a una tarea que ya la ha usado por un intervalo de tiempo determinado y asignársela a otra tarea y garantizar un progreso más uniforme. Esto se logra mediante el manejo de las interrupciones periódicas de un reloj, las cuales son atrapadas por el kernel y actuando en consecuencia.

Los sistemas **multi-usuarios** soportan diferentes espacios de trabajo para diferentes usuarios (home directories) con sistemas de control de acceso a los recursos. Muchos de estos sistemas permiten la conexión de múltiples terminales o consolas a la computadora la cual se usa de manera compartida.

Estos sistemas permiten usar el sistema mediante diferentes usuarios, con diferentes espacios de usuario, etc. Para lograr esto, se tiene que incluir un sistema de archivos para que los usuarios no puedan acceder a los archivos del otro y viceversa. No son lo mismo que los sistemas multitarea. Existen SO multitareas que no son multiusuarios, como Android por ejemplo, o las tablets ya que son dispositivos personales. Tu celular por ejemplo, solo tiene un usuario.

Actualmente las computadoras personales permiten múltiples consolas virtuales. Una consola o terminal virtual es un programa que emula una terminal física (hardware). También existen aplicaciones que emulan terminales remotas por red como telnet, ssh y aplicaciones de remote desktop.

Objetivos de un Sistema Operativo

Un SO tiene varios objetivos. El principal es ofrecer una *máquina abstracta* que oculta los detalles del hardware. Ofrece una máquina abstracta que oculta cómo las cosas se almacenan en el disco por ejemplo.

Uno de los principales objetivos es *virtualizar* recursos de hardware. Esta virtualización permite su mejor utilización ya que incluye el *multiplexado* de los mismos. Básicamente un SO crea abstracciones de los recursos.

Por ejemplo, un proceso representa una unidad de ejecución en una CPU (virtual) y su memoria. Un SO multitarea multiplexa la CPU entre los procesos. Esto es un multiplexado en el tiempo en el caso de la CPU, los procesos comparten entre sí el uso de la CPU.

En algunos casos, como en la gestión de la memoria, ésta se divide lógica o físicamente para asignarle espacios a cada proceso y al kernel. Un sistema de memoria virtual permite ejecutar un conjunto de procesos cuya demanda de memoria es mayor que la memoria física correspondiente mediante la técnica de *swapping* (que se describe en el capítulo [memory.md]). Esto es un multiplexado en el espacio, en el caso de la memoria, procesos diferentes tienen diferentes espacios de memoria.

Otro ejemplo de abstracción del hardware es el sistema de archivos el cual ofrece a los procesos una interfaz de operaciones sobre archivos y directorios (carpetas), ocultando los detalles técnicos de los dispositivos de almacenamiento (discos, cintas, etc).

Los sistemas operativos tienen dos funciones según el libro: proveer abstracciones a los usuarios de los programas y manejar los recursos de la computadora.

El diseño de UNIX abstrae la interacción con los dispositivos de E/S (Entrada / Salida) presentando a éstos como archivos especiales que se pueden leer y/o escribir mediante la conocida API de llamadas al sistema sobre archivos como `open(...)`, `read(...)`, `write(...)`, `close(...)` y otras. Cuando hacemos un `read` o un `write` ejecutamos una instrucción para que el SO haga la operación que le pedimos.

Esta virtualización o multiplexado hace que el SO también sea un administrador de recursos ya que debe incluir algoritmos de planificación de la asignación de los recursos para optimizar su uso.

El **multiplexado**, en otras palabras es esto:

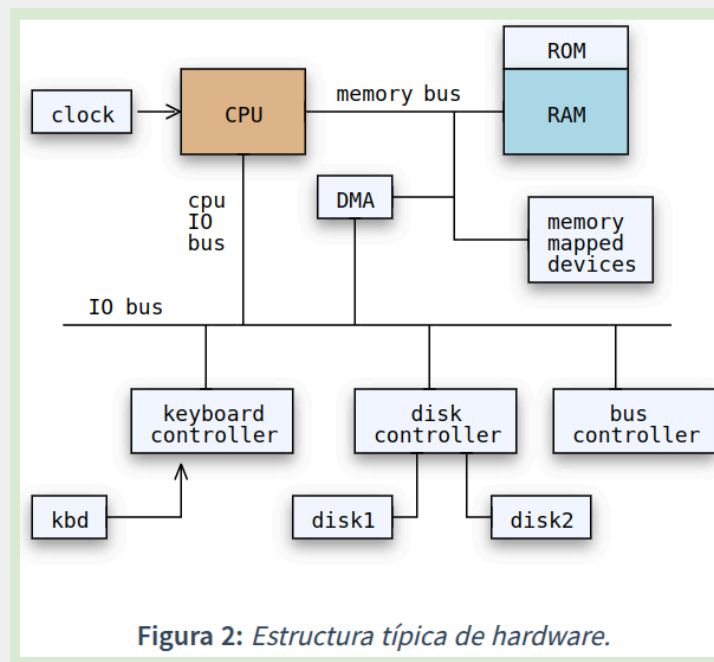
La gestión de recursos incluye la multiplexación de recursos de dos maneras diferentes: en el tiempo y en el espacio. Cuando un recurso se multiplexa en el tiempo, diferentes programas o usuarios se turnan para usarlo. Primero, uno de ellos usa el recurso, luego otro, y así sucesivamente. Por ejemplo, con una sola CPU y varios programas que desean ejecutarse en ella, el sistema operativo primero asigna la CPU a un programa; luego, tras un tiempo de ejecución suficiente, otro programa la usa, luego otro, y finalmente, el primero, de nuevo. Determinar cómo se multiplexa en el tiempo el recurso (quién lo usa a continuación y durante cuánto tiempo) es tarea del sistema operativo.

El otro tipo de multiplexación es la multiplexación de espacio. En lugar de que los clientes se turnen, cada uno obtiene una parte del recurso. Por ejemplo, la memoria principal normalmente se divide entre varios programas en ejecución, por lo que cada uno puede residir simultáneamente (por ejemplo, para usar la CPU por turnos). Suponiendo que haya suficiente memoria para varios programas, es más eficiente almacenar varios programas en memoria a la vez que asignarle toda la memoria a uno solo, especialmente si solo necesita una pequeña fracción del total.

Otro recurso con multiplexación de espacio es el disco. En muchos sistemas, un solo disco puede almacenar archivos de varios usuarios simultáneamente. Asignar espacio en disco y controlar quién usa qué bloques de disco es una tarea típica del sistema operativo.

Hardware

El siguiente diagrama muestra un esquema simplificado de una arquitectura general del hardware.



Como se puede apreciar en la figura de arriba, el hardware de un sistema de computación consta de diferentes componentes.

- **CPU:** uno o más procesadores.
- **ROM:** memoria no volátil que contiene código de arranque (boot) y datos de la plataforma de hardware.
- **RAM:** memoria principal (random-access-memory). Esta memoria es volátil. Es la memoria en donde se cargan los programas y los datos de este para ejecutarlo.

- **I/O bus and ports:** bus de I/O al cual se conectan los dispositivos de entrada/salida como en la plataforma x86 de Intel. Las instrucciones MOV van por el bus de memory bus, si quieres acceder a algo por el IO bus debemos usar instrucciones que se llaman IN o OUT.
- **Device controller:** expone un conjunto de puertos o registros de control y estado para un cierto tipo de dispositivos. Un controller puede manejar varios dispositivos del mismo tipo. Existen varios estándares como IDE/PATA para dispositivos de almacenamiento, USB para varios tipos de dispositivos.
- **Memory mapped devices (MMIO):** dispositivos de entrada y salida conectados al memory bus. Ejemplos: display/video controllers y otros (el teclado, discos, tarjetas gráficas, etc). Son mapeados a ciertas direcciones de memoria. Estos dispositivos tienen registros.
- **Direct-Memory-Access (DMA):** dispositivo que interconecta controladores de dispositivos al bus de memoria para permitir la transferencia de datos desde/hacia los dispositivos sin necesidad que intervenga la CPU.

Un SO debe conocer los detalles de estos componentes. Los módulos de software que interactúan con los controladores de dispositivos se denominan *device drivers*.

El **clock** es el que hace las interrupciones. **Memory bus** es utilizado para comunicarse con la memoria, las instrucciones MOV van por este bus.

Controller se refiere a un controlador de dispositivo de hardware. Los drivers son los componentes del software que interactúan con el controlador del hardware, son una pieza de software que corren en la CPU que es el que se encarga de comunicarse con el controller. Si tenemos un keyboard controller, el SO debería tener un bus driver que sepa tratar con ese controlador.

En RISC todos los dispositivos usan el memory bus, no el IO bus.

Diseño

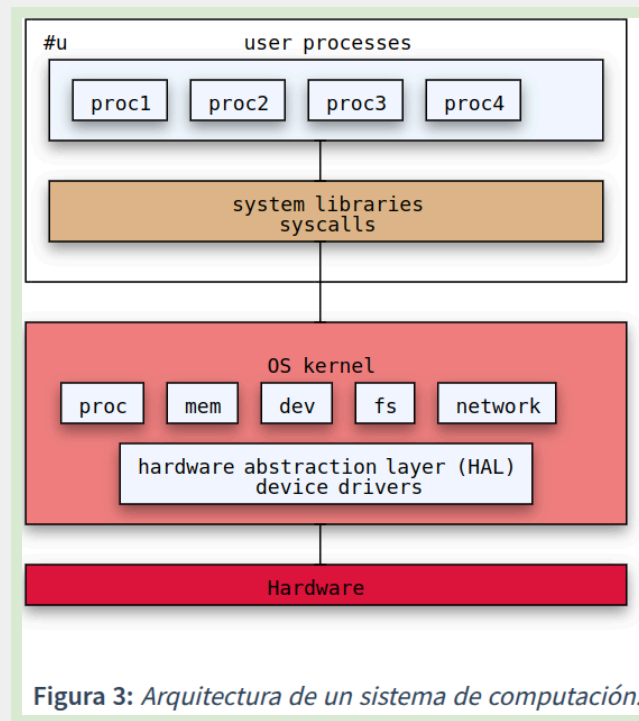
Los SO están diseñados en forma modular, en subsistemas:

1. Núcleo (kernel):

1. Gestión de procesos y usuarios. Controla los procesos (programas en ejecución) y decide cómo se reparten los recursos (como la CPU) entre ellos. También maneja permisos para que los usuarios no interfieran entre sí.
2. Gestión de la memoria. Administra la RAM, asignando espacio a los procesos y asegurándose de que no se "pisen" unos a otros.
3. Subsistema de entrada-salida: conjunto de device drivers. Se encarga de gestionar la comunicación con los dispositivos de hardware, como teclados, mouse, discos, impresoras, etc. Los "drivers" son programas específicos que le dicen al SO cómo hablar con cada dispositivo. Por ejemplo, un driver de disco sabe cómo leer o escribir datos en un disco duro.
4. Sistemas de archivos. Organiza y gestiona cómo se almacenan los datos en dispositivos como discos duros o SSD.
5. Subsistema de red (comunicaciones). Se ocupa de todo lo relacionado con las conexiones de red, como Wi-Fi.
6. Mecanismos de Seguridad: generalmente es un sistema transversal a los demás subsistemas.
7. IPC: (Inter-Process Communication) la parte de la comunicación de los procesos. Es el sistema que permite que los procesos "hablen" entre sí.

2. **Aplicaciones y bibliotecas del sistema** (ej: libc en sistemas tipo UNIX, que ofrece funciones comunes (por ejemplo, para abrir archivos o gestionar memoria) que los programadores usan para no tener que escribir todo desde cero).
- **Utilidades:** Shells (bash en linux), comandos básicos (cd, ls), editores de texto (vim), ...
 - **Servicios:** procesos de usuario (daemons) que brindan servicios de uso común como el servicio de impresión u otros.

Algunos de los procesos de usuario que forman parte de las utilidades y servicios del sistema ejecutan con mayores privilegios que los programas de usuario comunes.



En la figura 3, los programas de usuario utilizan (se enlazan con) las bibliotecas del sistema y otras de propósitos específicos. La biblioteca del sistema contiene la interfaz del programador de aplicaciones (API) la cual incluye las funciones que implementan las llamadas al sistema y otras utilidades de uso rutinario.

Una llamada al sistema (syscall) activa el mecanismo de interrupciones para saltar al código del kernel que la procesa.

El subsistema de E/S, representado como dev en la figura 2, incluye los device drivers los cuales son módulos de software que controlan una cierta clase de dispositivos, como discos IDE, teclado, mouse, pantalla, etc.

Uno de los desafíos en el diseño de un SO es encapsular el software dependiente de la plataforma de hardware del resto del código independiente de la plataforma. Esto se logra definiendo abstracciones de los componentes de hardware para facilitar su portabilidad a otras arquitecturas. Esta capa de abstracción en el kernel se conoce como **hardware abstraction layer (HAL)** y se considera la capa más baja dentro del kernel.

Los diseños de los SO se puede clasificar en:

- **Monolíticos:** los subsistemas del kernel se enlazan en un solo ejecutable. Cada módulo puede invocar a funciones de otros módulos. En un SO monolítico, todos los subsistemas del núcleo (como gestión de procesos, memoria, entrada-salida, etc.) se combinan en un solo programa grande (un único ejecutable). Los módulos dentro del núcleo pueden llamarse entre sí directamente, como si fueran funciones de un mismo programa.
 - **Ventajas:** simple y eficiente.

- **Desventajas:** un error en un módulo puede afectar a todo el núcleo. Prácticamente no existe el ocultamiento de información.

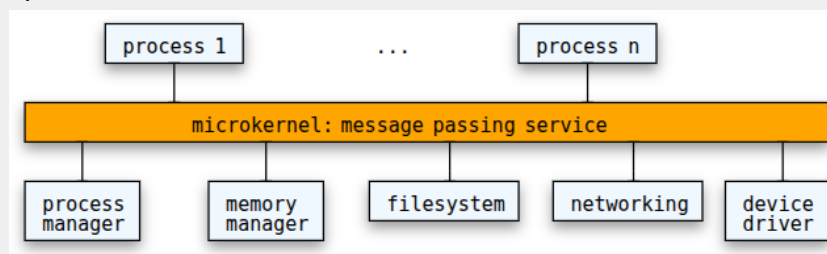
Ejemplos: UNIX, [GNU-Linux][<https://www.gnu.org/gnu/linux-and-gnu.en.html>], FreeBSD y Fuchsia.

Se podría decir que el kernel es un único programa, todos sus componentes están en un mismo programa. Pero tiene un problema, qué pasa si el código de mi SO tiene un montón de bus drivers y mi máquina tiene pocos, entonces va a haber cosas sin usarse. Linux es monolítico y resolvió esto haciendo los drivers como plugins. Se puede solucionar de otra manera, que los distintos componentes (proc, mem, dev, fs, network) sean programas independientes, lo que se llaman microkernels.

Según el libro, en los SO monolíticos tienen todo el sistema operativo se ejecuta como un único programa en modo kernel. El SO está escrito como una colección de procedimientos, enlazados en un único y gran programa binario ejecutable. Al utilizar esta técnica, cada procedimiento del sistema puede llamar a cualquier otro, si este último proporciona algún cálculo útil que el primero necesita. Poder llamar a cualquier procedimiento es muy eficiente, pero tener miles de procedimientos que pueden llamarse entre sí sin restricciones también puede resultar en un sistema complejo y difícil de entender. Además, un fallo en cualquiera de estos procedimientos podría paralizar todo el sistema operativo.

Para construir el programa objeto real del sistema operativo con este enfoque, primero se compilan todos los procedimientos individuales (o los archivos que los contienen) y luego se enlazan en un único archivo ejecutable mediante el linker del sistema. En cuanto a la ocultación de información, prácticamente no existe: cada procedimiento es visible para todos los demás procedimientos (a diferencia de una estructura que contiene módulos o paquetes, donde gran parte de la información está oculta dentro de los módulos, y solo los puntos de entrada designados oficialmente pueden ser llamados desde fuera del módulo).

- **Microkernels:** cada subsistema del kernel se compila como un ejecutable independiente. Para invocar un servicio de otro subsistema se usa un mecanismo de comunicación basado en mensajes. En un microkernel, el núcleo es mucho más pequeño y cada subsistema (gestión de memoria, drivers, sistemas de archivos, etc.) se compila como un programa separado (ejecutable independiente). En lugar de llamarse directamente entre sí, los subsistemas se comunican enviándose mensajes a través de un mecanismo especial.



- **Ventajas:** un error (ejemplo, de memoria) en un subsistema no afecta a los demás ya que cada componente es independiente y está confinado en su propia área de memoria. Fácilmente se puede convertir en un SO *distribuido* (subsistemas ejecutándose en diferentes nodos de una red).
- **Desventajas:** el mecanismo de pasaje de mensajes crea una sobrecarga computacional importante afectando su rendimiento. Son menos eficientes porque tienen que comunicarse entre sí mediante mensajes. Su rendimiento es menor.

Ejemplos: Minix, L4, MS-Windows

Según el libro la idea básica del diseño del microkernel es lograr una alta fiabilidad dividiendo el sistema operativo en pequeños módulos bien definidos, de los cuales solo uno (el microkernel) se

ejecuta en modo kernel, mientras que el resto se ejecuta como procesos de usuario comunes, relativamente ineficaces. En particular, al ejecutar cada device driver y sistema de archivos como un proceso de usuario independiente, un error en uno de ellos puede bloquear ese componente, pero no todo el sistema. Por lo tanto, un error en el driver de audio provocará que el sonido se distorsione o se detenga, pero no bloqueará la computadora. Por el contrario, en un sistema monolítico con todos los drivers en el kernel, un driver de audio defectuoso puede fácilmente referenciar una dirección de memoria no válida y paralizar el sistema de forma instantánea.

- **Máquinas virtuales:** el sistema tiene un hypervisor que se encarga de multiplexar el hardware a los SO huéspedes que se ejecutan encima (en modo usuario). El hypervisor ofrece a cada SO huésped un ambiente aislado de ejecución. En este diseño, hay un hypervisor (o monitor de máquinas virtuales) que actúa como una capa entre el hardware y los sistemas operativos que corren encima, llamados SO huéspedes. El hypervisor divide (multiplexa) los recursos del hardware (CPU, memoria, etc.) y crea entornos aislados para que cada SO huésped piense que tiene el control total del hardware. Cada SO huésped opera en modo usuario (sin acceso directo al hardware), mientras que el hypervisor tiene el control real en modo núcleo.

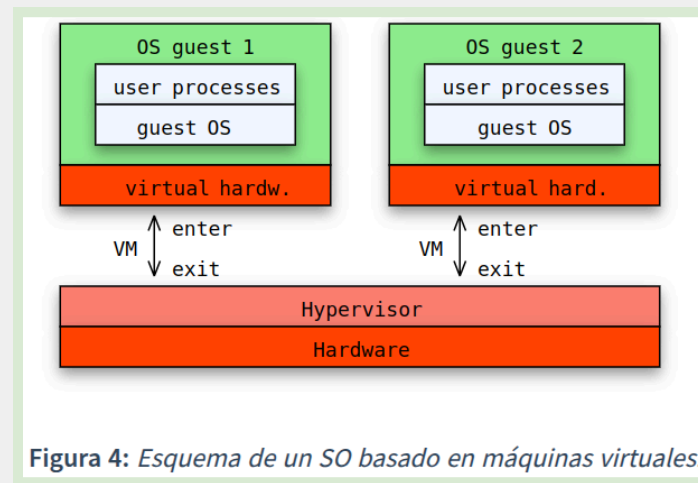


Figura 4: Esquema de un SO basado en máquinas virtuales.

La máquina virtual es el hypervisor. Los dos cuadrados verdes son las máquinas virtuales y podríamos tener a nuestro hypervisor que es Linux y tener en los dos cuadrados un Windows y un Mac.

El hypervisor se encarga de multiplexar el hardware. Cada cosa verde es un SO completo que va a estar corriendo sobre un hardware virtualizado que le provee el hypervisor. Podemos tener uno sea un Windows y el otro OS un Linux por ejemplo.

Cuando el hypervisor está integrado en un SO, a éste se le denomina el host kernel (anfitrión) como por ejemplo Linux KVM que provee mecanismos básicos para las transiciones entre un SO huésped (guest) y el host y redirigir a un virtualizador en modo usuario como qemu o VirtualBox.

Generalmente para poder implementar un hypervisor, el hardware debe proveer mecanismos como nuevos modos de operación de la CPU como por ejemplo Intel VMX root mode (modo del anfitrión) y VMX non-root-mode (modo del huésped) y otras estructuras de datos como registros sombras (shadowed).

Algunas arquitecturas modernas como RISC-V incluyen modos de protección o privilegios para facilitar la implementación de la virtualización. Ejemplos: Xen e IBM System 370.

No se debe confundir la *virtualización* con *contenedores*. Un contenedor es un grupo de procesos y recursos (memoria, filesystem, dispositivos y otros) que son independientes del resto del sistema. Esto permite la creación de sistemas auto-contenidos minimalistas, es decir que incluyen sólo las dependencias requeridas.

Se ejecutan en forma confinada o aislada del resto del sistema. El SO debe proveer mecanismos para su ejecución. Ejemplos: GNU-Linux cgroups y FreeBSD jails. Aplicaciones como Docker permiten crear, configurar y ejecutar contenedores. Aparte es más eficiente que una máquina virtual. No tiene ninguna sobrecarga que tienen las máquinas virtuales, que son el enter y el exit.

La virtualización es diferente a los contenedores. Un container no es una máquina virtual, es agrupar todos los contenedores que están corriendo en la misma máquina y se independizan del sistema. Es ejecutar un conjunto de procesos en un ambiente confinado. Hace que se estén corriendo en un ambiente aislado de manera confinada. En estos containers incluimos las dependencias que necesitan y nada más. Se usa mucho para la seguridad, si hay un hackeo solo va a poder acceder a los recursos del container y no el resto del sistema.

Consideraciones de diseño

Muchos SO actuales tienen una arquitectura híbrida, generalmente con un kernel pequeño monolítico y otros componentes fuera de él comunicándose por mensajes. Ejemplos de estos diseños son MS-Windows, Mac-OS y FreeBSD.

Algunos SO modernos, aún cuando sean monolíticos como GNU-Linux, son altamente modulares, permitiendo el desarrollo de ciertos subsistemas como device drivers o filesystems como módulos de carga y descarga dinámica (ver Linux Kernel Modules), permitiendo al administrador del sistema cargar o descargar componentes en caliente, es decir sin necesidad de reiniciar (reboot) el sistema.

Un principio de diseño sugerido para los SO es definir e implementar mecanismos en las capas más bajas y que las políticas se definan en los niveles más altos. Esto garantiza una buena adaptabilidad de un kernel a diferentes escenarios.

Interfaz de un Sistema Operativo

Un SO presenta a los desarrolladores de aplicaciones interfaces, tanto a nivel de código fuente como en formatos binarios.

- **Application Programming Interface (API):** interfaz a nivel de código fuente que incluye las llamadas al sistema y las funciones (y tipos de datos) de la biblioteca estándar. Por ejemplo, llamadas al sistema (system calls): funciones que le dicen al SO qué hacer, como abrir un archivo (open), leer datos (read), etc.

O por ejemplo, las funciones y tipos de datos de la biblioteca estándar: en sistemas tipo UNIX, la biblioteca libc ofrece funciones como printf para imprimir en pantalla o malloc para reservar memoria.

El estándar portable OS interface (POSIX) define una API de portabilidad a nivel de sistema y usuario como también con un conjunto de utilidades (como shells y otros comandos) con el fin de compatibilizar variantes del SO UNIX. En particular, éste estándar define la API como declaraciones de funciones y tipos de datos en C.

- **Application Binary Interface (ABI):** interfaz de bajo nivel que especifica los formatos binarios y convenciones para permitir la interacción de programas de usuario con las bibliotecas y con el kernel del sistema operativo. La ABI es una interfaz de bajo nivel que define cómo los programas ya compilados (en formato binario, es decir, ceros y unos) interactúan con el SO y sus bibliotecas. Mientras la API es para humanos que escriben código, la ABI es para las máquinas que ejecutan ese código.

Los sistemas operativos generalmente especifican una ABI para cada plataforma de hardware (x86, x86-64, ARM, risc-v, etc). La ABI incluye:

1. Convenciones para las invocaciones a funciones.
2. Convenciones para los syscalls.
3. Formato de archivos binarios (objetos y ejecutables)

A modo de ejemplo, GNU-Linux usa binarios en formato Executable and Linking Format (ELF) para todas las plataformas soportadas. La convención para las llamadas a funciones en x86 (32 bits) es que los parámetros se apilan por el invocante en orden reverso (de derecha a izquierda) y en x86-64 se pasan hasta 6 argumentos en los registros `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` (el resto en la pila en orden reverso). Los valores retornados quedan en el registro `eax/rax`, respectivamente. En ambas arquitecturas se usan las instrucciones `call/system` para una invocación.

En arquitecturas RISC, como RISC-V, los parámetros se pasan en los registros `a0`, ..., `a7` y el valor retornado queda en `a0`.

Estas convenciones deben ser respetadas por cualquier compilador de la plataforma.

Usuarios, sesiones, shell y variables de ambiente

Generalmente el uso de un sistema multi-usuario se basa en sesiones. Al comienzo un usuario debe presentar sus credenciales como su identificador y una contraseña. Luego el sistema lanza un shell, el cual puede basarse en una interfaz de texto (consola) o gráfica. Luego el usuario puede interactuar con el sistema lanzando otros programas para finalmente cerrar la sesión.

El shell define un conjunto de variables de ambiente que el usuario y los programas pueden usar y modificar.

Algunas variables de ambiente comunes en sistemas tipo UNIX son:

- HOME: directorio o espacio de trabajo del usuario.
- PATH: lista de directorios de búsqueda de programas.
- PWD: directorio corriente (print working directory).
- SHELL: programa (path) shell.
- LANGUAGE: lenguaje de la interfaz de usuario

Se puede usar el comando `echo` para acceder al valor de una variable de ambiente. El valor de una variable `V` se accede con la expresión `$V` en UNIX (`%V` en MS-Windows).

Que es la shell?

El sistema operativo es el código que ejecuta las llamadas al sistema. Editores, compiladores, ensambladores, linkers, programas de utilidad e intérpretes de comandos no forman parte del sistema operativo, aunque son importantes y útiles.

La shell es un intérprete de comandos de UNIX. Si bien no forma parte del sistema operativo, es la interfaz principal entre un usuario sentado en su terminal y el sistema operativo, a menos que el usuario esté usando una interfaz gráfica de usuario. Existen muchas shells, como lo son `sh`, `csh`, `ksh`, y `bash`.

Cuando un usuario inicia sesión, se inicia una shell. Esta shell tiene la terminal como entrada y salida estándar. Comienza escribiendo el prompt, que es símbolo del sistema (`$`), que indica al usuario que la shell está esperando para aceptar un comando.

Si por ejemplo el usuario escribe el siguiente comando:

```
date
```

La shell crea un proceso hijo y ejecuta el programa `date` como el hijo. Mientras que este proceso hijo se ejecuta, la shell espera a que termine.

Es posible listar las variables de ambiente con el comando `printenv` en UNIX o `set` en MS-Windows. Una variable de ambiente se puede crear o modificar su valor mediante el comando `var=value` o `export` (variable global), dependiendo del shell. En MS-Windows se puede crear/modificar una variable con el comando `set`.

El siguiente ejemplo muestra parte de la API que nos ofrece un SO POSIX.

```
#include <sys/types.h> /* system data types */
#include <unistd.h>     /* UNIX standard API */
#include <stdio.h>      /* I/O standard API */

int main(int argc, char *argv[], char *envv[])
{
    int i;
    printf("Process id: %d\n", getpid()); /* getpid() is a syscall */
    printf("Command line arguments:\n");
    for (i = 0; i < argc; i++)
        printf("%s\n", argv[i]);
    printf("Session environment variables:\n");
    for (i = 0; envv[i]; i++)
        printf("%s\n", envv[i]);
    exit(0); /* exit syscall: process exit code */
}
```

En este ejemplo se puede observar que:

1. Un programa de usuario comienza por una función denominada `main` que recibe como parámetros el número de los argumentos de la línea de comandos, un arreglo de punteros (strings) a ellos y un arreglo (null-terminated) de strings con las variables de ambiente de la forma `variable=valor`.
2. Los programas interactúan con los dispositivos de entrada/salida (en este caso la consola o terminal) indirectamente por medio de funciones de biblioteca, las cuales a su vez ejecutan syscalls. En este caso `printf()`, internamente ejecuta el syscall `write(1, buffer, count)`. El descriptor de archivo cero (0) está conectado a la entrada estándar (generalmente el teclado). Los descriptors 1 y 2 refieren a la salida y salida de errores estándar, asociados a la pantalla.
3. Las llamadas al sistema (como `getpid()` o `exit(n)`) están implementadas como funciones de la biblioteca estándar.
4. Al compilar este programa (con `gcc -o myprog myprog.c`) el linker lo enlaza con la biblioteca estándar.

Inicio del sistema (booting)

Al encenderse el sistema éste está configurado para comenzar la ejecución de código desde una dirección de memoria predeterminada que corresponde a una memoria no volátil (ROM, EPROM, etc) que contiene el programa inicial de carga de un SO.

PC BIOS

En el caso de la IBM-PC, basada en x86, el código (conjunto de funciones o firmware) en la ROM se conoce como Basic Input-Output System (BIOS). Las PCs modernas contienen un firmware evolucionado conocido como UEFI.

BIOS simplemente carga el contenido del primer bloque de 512 bytes (master boot record o MBR) del disco primario en la dirección de RAM física 0x7c00 y salta (jmp) a esa dirección.

El programa cargado deberá cargar en memoria el kernel o un second stage boot loader como GRUB. GRUB soporta el uso de múltiples OS (Linux, MS-Windows, etc) en diferentes discos o particiones.

El MBR se graba durante la instalación o actualización del sistema.

UEFI

UEFI simplifica el proceso de inicio mediante una partición en el disco (con rótulo EFI) que contiene archivos de configuración y ejecutables. Un ejecutable puede ser un boot loader (como GRUB) o directamente un kernel con soporte UEFI (contiene una cabecera específica).

El firmware carga el archivo de configuración y presenta un menú de opciones de arranque y carga el ejecutable correspondiente.

UEFI también soporta booteo seguro que se basa en que un kernel esté firmado por el fabricante. En éste caso, las claves privadas de cada fabricante deben almacenarse en la memoria EPROM. Esto complica las actualizaciones frecuentes, práctica común en usuarios Linux.

Otros tipos de firmware

En algunas arquitecturas como aquellas basadas en system on chip (SoC), como ARM o RISC-V, comúnmente están configuradas para cargar el kernel desde algún punto específico de algún dispositivo de almacenamiento, comúnmente una tarjeta de memoria de estado sólido (SD cards).

En éstos sistemas el kernel se carga a partir de una dirección física de memoria predeterminada y especificada para cada board.

Inicio del SO

El kernel debe estar compilado y enlazado a partir de la dirección de carga. En su inicio realiza los siguientes pasos:

1. Inicializa subsistemas.
2. Lanza el primer proceso de usuario (init en sistemas tipo UNIX). init se va a ejecutar en modo usuario, o sea no va a poder ejecutar ciertas instrucciones privilegiadas, como ser, deshabilitar las instrucciones lo cual le saca el control al kernel porque este en base a estas recupera el control.
3. Este proceso ejecuta un programa de control de la terminal (tty).
4. tty lanza login.
5. login lanza el intérprete de comandos o shell.

Interfaces

APIs de Sistemas Operativos

Un sistema operativo ofrece a las aplicaciones servicios que permiten acceder a recursos del sistema y facilidades como mecanismos de comunicación y sincronización entre procesos y otros.

El sistema operativo UNIX, creado por Ken Thompson y Dennis Ritchie provee interfaces simples y coherentes. La mayoría de los SO modernos se basan en estas APIs.

Linux no es 100% compatible con POSIX, es compatible en un 99%.

La API de UNIX se describe como un conjunto de *funciones* C que implementan syscalls, son un conjunto de llamadas al sistema. A continuación se muestra una lista parcial de llamadas al sistema correspondientes a operaciones sobre procesos y archivos.

Procesos

1. **fork():** crea un nuevo proceso (hijo) el cual es una copia del proceso invocante. Este hijo es una copia exacta del proceso invocante, se crea una copia idéntica del proceso padre. El hijo hereda todo el contexto del padre, todo su estado (program counter, stack pointer, valores de los registros de la CPU, etc) y memoria, los archivos que había abierto el padre con un open, el hijo los va a tener abiertos también, etc. Es la única función que crea un proceso. En nuestro SO vamos a tener un árbol de procesos, la raíz es el proceso `init`, que es el primer proceso que ejecuta el SO y este luego se encarga de ejecutar los demás. Si el resultado del fork es 0, estamos en el proceso hijo. El hijo se ejecuta a partir del fork, en el estado en el que su padre estaba, como si ya se hubiera ejecutado anteriormente como el padre hizo.
2. **exit(exit_code):** finaliza el proceso con código de salida `exit_code`. Termina el proceso que ejecutó esa llamada, normalmente recibe un código de salida para saber si finalizó bien o no. 0 es una finalización exitosa, cualquier otra cosa es un código de error.
3. **pid wait(&exit_code):** espera (se bloquea) hasta que finalice un proceso hijo. Retorna el pid del hijo y en `exit_code` se recibe el código de salida. Función que un proceso padre ejecuta para esperar a que terminen alguno de sus procesos hijos, esto provoca que el padre se bloquee hasta que alguno de sus procesos hijos finalice. Retorna el pid del proceso hijo que terminó así el padre sabe cuál de ellos terminó, y el `exit_code` es el código de finalización del hijo. Es mala práctica que el padre termine antes que sus hijos. Cada proceso debería tener un padre, excepto `init`. Si el padre termina antes rompemos el invariante porque provoca que sus hijos no tengan padre. Si un hijo terminó antes que el padre pero su padre todavía sigue corriendo y este nunca hizo un `wait`, a este hijo se lo denomina *zombie*.
4. **getpid():** retorna el identificador del proceso corriente.
5. **getppid():** retorna el identificador del proceso padre.
6. **kill(pid):** envía una señal (terminación) al proceso con id `pid`.
7. **sleep(n):** espera (se bloquea) por `n` segundos.
8. **exec(filename, args):** carga el programa `filename` y lo ejecuta. Reemplaza la imagen de código y datos de la memoria del proceso corriente. Recibe un path de un archivo que es un programa y un arreglo de strings que son los comandos que puede recibir. `exec` no crea un nuevo proceso sino que reemplaza la imagen de la memoria del proceso corriente, o sea que este proceso corriente desaparece. Todo el programa que invocó a `exec` desaparece. Podemos usar el `fork` para crear un nuevo proceso y usamos el `exec` para ese hijo. `args` son los argumentos de comando, en C se tiene que pasar como un arreglo de strings.
9. **sbrk(n):** asigna `n` bytes de memoria adicionales al proceso corriente. Asigna más memoria de lo que normalmente se le asigna cuando se carga en la memoria al proceso. Esto sirve para implementar un heap por ejemplo. El heap usa el `sbrk` para pedir más memoria.

Archivos (todas estas funciones retornan un entero, si fallan retornan un número negativo, estas son las verdaderas llamadas al sistema, las `fopen`, `fread`, etc en realidad invocan a estas)

1. **open(filename, mode):** abre un archivo en el modo (read, write, ...) dado. Las funciones fopen que usamos son funciones de alto nivel, lo que retorna es un puntero a una estructura, en cambio, esta función open retorna un entero.
2. **read(fd, buffer, count):** lee hasta count bytes (empezando desde cero) del archivo en buffer. El buffer es en donde se van a copiar los valores que se leen del archivo y count es la cantidad máxima de bytes del archivo que puede leer. Esta función retorna la cantidad de bytes leídos.
3. **write(fd, buffer, count):** escribe count bytes de buffer al archivo.
4. **close(fd):** cierra el archivo (libera recurso). No siempre que hacemos un read o write se accede al disco, sino que se guarda en un buffer y cuando se hace un close, todos esos datos se guardan en el disco, recién se hace en el close esto.
5. **dup(fd):** duplica (copia) fd en el primer descriptor no usado. El dup busca la tabla de archivos (ofiles) y cuando encuentra una entrada vacía lo copia ahí.
6. **pipe():** crea un *pipe* para la comunicación entre procesos. Este solo funciona para procesos que tengan una relación de parentesco entre sí, o sea, uno sea hijo y el otro padre. Devuelve dos descriptors de archivos que los devuelve en un arreglo. Estos pipes permiten que procesos se comuniquen entre sí.
7. **chdir(dirname):** cambia de directorio.
8. **mkdir(dirname):** crea un directorio.
9. **mknod(name, major, minor):** crea un archivo especial (device).
10. **fstat(fd):** retorna el estado y metadatos (size, ...) del archivo.
11. **link(f1, f2):** crea un alias (f2) para el archivo f1. Crea enlaces simbólicos.
12. **unlink(filename):** borra el archivo o el enlace simbólico.

En algunas versiones modernas de SO tipo UNIX algunas de estas llamadas al sistema se han extendido ampliamente, como por ejemplo en GNU-Linux. Todas las llamadas al sistema se presentan al programador como funciones de biblioteca. Su implementación dispara un syscall, es decir una entrada a un punto del kernel. A partir de allí se ejecuta código del kernel hasta su retorno.

Todas las llamadas al sistema retornan un entero. Generalmente en caso de falla retornan un valor negativo (comúnmente -1).

Interfaz del Usuario

Un SO presenta algún tipo de interfaz al usuario. Generalmente el programa que se encarga de interactuar con el usuario es un *shell*, el cual puede ser basado en línea de comandos (como bash, zsh, csh de UNIX o cmd en MS-Windows).

Un shell de línea de comandos permite al usuario ingresar comandos para ejecutar programas y soportan operadores sobre el control, sincronización y comunicación entre los procesos creados. El shell se dispara al inicio del sistema, generalmente luego que el usuario inicia su sesión (login) asociada a la consola o terminal.

Típicamente un shell presenta al usuario un prompt que puede incluir su identificador de usuario, finalizado comúnmente por el carácter #. Cabe aclarar que el prompt de un shell es configurable por el usuario (Ej: .bash_profile).

El sistema presenta al usuario un sistema de archivos. Hay archivos de diferentes tipos:

1. Programas: archivos ejecutables (o comandos).
2. Datos: contienen datos en algún formato (binario o texto).

Un archivo de textos contiene una secuencia de códigos de caracteres (ej: ASCII, UTF-8, ...). Generalmente se estructuran en líneas, es decir, secuencias de caracteres finalizados por `\n` (newline) o `\n\r` (newline/carriage-return).

El conjunto de archivos del sistema se organiza en una estructura jerárquica (de árbol) con directorios (o carpetas), los cuales a su vez contienen otro subárbol de directorios y archivos.

Para hacer referencia a un archivo se puede hacer de dos formas:

1. **Full path**: camino desde la raíz del sistema de archivos hasta el archivo. ejemplo: `cat /home/user/docs/myfile.txt`
2. **Relativa**: en base al directorio corriente, es decir, el directorio de trabajo actual al que se accede mediante el comando `cd dir-path`). Ejemplo: `cd docs ; cat myfile.txt`

En los sistemas tipo UNIX la convención es que un proceso con código de salida=0 significa terminación normal, mientras que otros valores representan algún código de error, dependiendo de la aplicación.

Un shell típico en sistemas tipo UNIX reconoce los siguientes operandos y operadores:

- **Operandos**: pueden ser un comando o un nombre de archivo
- **Operadores**:
 - `cmd1 ; cmd2`: ejecuta `cmd1` para luego ejecutar `cmd2` (cuando `cmd1` finalice).
 - `cmd1 && cmd2`: ejecuta `cmd2` sólo si `cmd1` finaliza exitosamente.
 - `cmd1 || cmd2`: ejecuta `cmd2` sólo si `cmd1` finaliza con error.
 - `cmd &`: lanza el `cmd` de fondo (en background), el shell no espera a su terminación, devolviendo el prompt inmediatamente.
 - `cmd n> file`: se redirige el descriptor de archivo de salida `n` al archivo `file`. Si `n` se omite, se asume 1 (salida estándar).
 - `cmd < file`: redirige la entrada estándar (comúnmente el teclado) a que sea desde el archivo `file`. Por ejemplo: `wc < file.txt`.
 - `cmd1 | cmd2`: ejecuta `cmd1` y `cmd2` concurrentemente, redirige la salida estándar de `cmd1` a un pipe y la entrada estándar de `cmd2` a la salida del pipe.

Procesos

En esta sección se analizan las llamadas al sistema para la creación, destrucción, sincronización y comunicaciones entre procesos (Inter Process Communication - IPC).

En una API tipo UNIX la única llamada al sistema para crear un nuevo proceso es `fork()`, la cual crea una copia del proceso corriente, conjuntamente con su estado.

El estado de un proceso (mantenido por el kernel) tiene generalmente los siguientes atributos:

1. Su estado: RUNNING, READY, SLEEPING, ...
2. Su identificador o `pid`: un número `> 0`
3. Su imagen de memoria: código, datos globales y stack.
4. El conjunto (tabla) de archivos/pipes abiertos.
5. Una referencia a su padre (parent)
6. Una stack para ejecución en modo kernel
7. Espacio para salvar su contexto (estado de la CPU) cuando cambie de estado.
8. El código de salida o terminación.

La llamada al sistema `fork()`, ejecutada inicialmente por un proceso padre hace que el kernel cree una copia (el hijo) del proceso corriente, el cual hereda su estado: el conjunto de valores de los registros de la CPU, en particular la tabla de archivos abiertos, etc.

Esto hace que cuando en el futuro el planificador de procesos decida otorgarle la CPU al proceso hijo éste iniciará su ejecución en el punto del retorno del `fork()` ya que ese era el contenido del program counter.

En el proceso padre `fork()` retorna el `pid` (>0) del nuevo proceso (hijo). En el hijo retorna 0 (cero).

El nuevo proceso (hijo), al heredar el estado del padre, inicia su ejecución en el retorno del `fork()`, aunque aquí retorna 0.

El siguiente ejemplo muestra el uso de `fork()`.

```
#include <unistd.h> /* UNIX syscalls */
#include <stdio.h> /* portable i/o functions */

int main(void) {
    int pid = fork();
    if (pid == 0) {
        printf("I'm the child. Pid: %d\n", getpid());
    } else {
        printf("I'm the parent. Pid: %d\n", getpid());
        printf("My child is: %d\n", pid);
    }
}
```

```
delfina@delfina-G5-5590:~/UNRC/S0$ gcc -o exec exercise6.c
delfina@delfina-G5-5590:~/UNRC/S0$ ./exec
I'm the parent. Pid: 78530
My child is: 78531
I'm the child. Pid: 78531
```

Eso imprime este ejemplo.

Si después del `fork` no hacemos un `if`, lo que se haga lo hacen ambos (hijo y padre), entonces separamos las cosas para que el hijo haga una cosa y el padre otra.

¿Por qué puede fallar un `fork`? Puede fallar si llegamos a un límite de procesos creados vivos y no se pueda crear más. Sin embargo, ese límite es grandísimo así que es muy raro que pase, a menos que tengamos un `fork` que crea procesos en un loop y en algún momento falle obviamente. Pero es muy difícil que falle el `fork`.

Un proceso padre debería esperar hasta que alguno de sus hijos termine antes de finalizar. La llamada al sistema `wait(&status)` retorna en el parámetros de salida `status` el código de terminación del proceso hijo.

A continuación se muestra un ejemplo del uso de `wait` y `exit`.

```

#include <unistd.h> /* UNIX API (syscalls) */
#include <stdio.h> /* portable i/o functions */

int main(void) {
    if (fork() == 0) {
        printf("I'm the child. Pid: %d\n", getpid());
        exit(0);
    } else {
        int child, status;
        printf("I'm the parent. Pid: %d\n", getpid());
        child = wait(&status);
        printf("In parent: child %d finished\n", child);
    }
}

```

```

delfina@delfina-G5-5590:~/UNRC/S0$ ./exec
I'm the parent. Pid: 78690
I'm the child. Pid: 78691
In parent: child 78691 finished

```

Eso imprime este ejemplo.

Ese `exit(0)` se podría reemplazar con un `return 0` que es lo mismo.

La llamada al sistema `exec(program, args)` reemplaza la imagen de memoria (código y datos) del proceso corriente con el código y datos de program. Cabe aclarar que `exec()` no crea un nuevo proceso. El proceso comienza su ejecución desde el entry point (function address) que está especificado en el archivo ejecutable program.

Es común que un programa lance otro programa para realizar alguna tarea para luego continuar otras operaciones. En este modelo se debe hacer como en el siguiente ejemplo, el cual ejecuta el comando `ls -l`:

```

#include <unistd.h> /* UNIX syscalls */
#include <stdlib.h> /* wait() */
#include <stdio.h> /* portable i/o functions */

int main(void) {
    char *args[] = {"ls", "-l", 0};

    if (fork() == 0) {
        /* in child */
        execve("/bin/ls", args, 0);
        /* on success, below is unreachable code */
        printf("Ooops, exec() failed!\n");
        exit(-1);
    } else {
        wait(0);
        printf("In parent: child finished\n");
    }
}

```

execve es una variante de exec. Creamos un proceso hijo y hacemos que este ejecute un ls. El 0 en el args significa que terminó, porque no le estamos diciendo cuantos argumentos tiene. Si execve tiene éxito, lo que está abajo no se va a ejecutar nunca.

Si hacemos man exec accedemos al manual en la consola.

Shell: Control de procesos

Un comando de la forma `cmd &` lanza el programa `cmd` como un proceso de fondo (background). El shell le asigna un identificador de job como se muestra abajo.

```

$ ping google.com > output &
[1] 4287
$ _

```

El shell muestra el job id (ejecutando en background) 1 asociado al proceso con pid 4287. El comando `jobs` listará los procesos en background y su estado.

```

[1]+  Running                  ping -i 5 google.com &

```

Un proceso en foreground (el shell está esperando a que termine) puede suspenderse o pararse usando `Ctrl-Z`. El shell devuelve el prompt y puede usarse el comando `fg` para continuarlo de fondo. Es posible finalizar (matar) un proceso de fondo mediante el comando `kill %job_id`.


```

$ ping google.com
PING google.com (74.125.226.71) 56(84) bytes of data.
64 bytes from lga15s44-in-f7.1e100.net (74.125.226.71): icmp_seq=1 ttl=55 time=12.3 ms
64 bytes from lga15s44-in-f7.1e100.net (74.125.226.71): icmp_seq=2 ttl=55 time=11.1 ms
64 bytes from lga15s44-in-f7.1e100.net (74.125.226.71): icmp_seq=3 ttl=55 time=9.98 ms
Ctrl-Z
[1]+  Stopped                  ping google.com
$ bg
[1] 4579
...
kill %1
[1]+  Finished                  ping google.com

```

También es posible enviar al foreground un proceso corriendo en background con el comando fg.

Viéndolo en la consola:

El [1] es el job id (para la shell) del proceso que se lanzó con el ping.

```

marcelo@marcelo-linux-desktop:tmp$ ping google.com > output &
[1] 8152
marcelo@marcelo-linux-desktop:tmp$ jobs
[1]+  Ejecutando                  ping google.com > output &

```

Si ahora lo mandamos al foreground:

```

marcelo@marcelo-linux-desktop:tmp$ fg 1
ping google.com > output

```

Si apretas Ctrl+z se detiene:

```

marcelo@marcelo-linux-desktop:tmp$ jobs
[1]+  Detenido                    ping google.com > output

```

Lo puedes matar de la siguiente manera:

```

marcelo@marcelo-linux-desktop:tmp$ kill %1
[1]+  Terminado                  ping google.com > output

```

Redirección de entrada/salida

En UNIX, cada proceso (como un programa que se ejecuta) tiene archivos abiertos que usa para leer datos (entrada) o escribir datos (salida). Estos archivos están asociados a descriptores de archivo (file descriptors), que son números enteros que identifican cada archivo abierto dentro del proceso.

Cada proceso inicia comúnmente con tres archivos abiertos (heredados) desde init, el primer proceso del sistema, lanzado por el kernel luego del boot.

El file descriptor 0 está asociado a la entrada de la consola, comúnmente un teclado, por lo que el proceso leerá lo que el usuario escribe.

Los descriptores 1 y 2 están asociados a la salida de la consola, típicamente la pantalla o display.

Cuando un proceso inicia (por ejemplo, el primer proceso del sistema, init, que es lanzado por el kernel después del arranque), el sistema operativo le asigna tres descriptores de archivo estándar:

- `stdin` (0): el file descriptor 0 está asociado a la entrada de la consola, comúnmente un teclado, por lo que el proceso leerá lo que el usuario escribe.
- `stdout` (1): están asociados a la salida de la consola, típicamente la pantalla o display. Así que lo que el proceso escribe se muestra ahí.
- `stderr` (2): también está conectado a la consola, pero se usa para mensajes de error.

Es posible ver los archivos abiertos de un proceso como una tabla de la forma:

ofiles	
0	stdin
1	stdout
2	stderr
3	

Los descriptores 0, 1 y 2 siempre están definidos al inicio. Si el proceso abre más archivos (como un archivo de texto), se le asignarán nuevos descriptores (3, 4, etc.).

Cada vez que se ejecuta un programa, se abren esos tres archivos de la tabla ofiles.

`init` es el primer proceso que se ejecuta y la única forma de crear un proceso es con el `fork()`. `init` es un programa que lo primero que hace es abrir estos tres archivos y se los asigna a la consola o al dispositivo que corresponda, como el teclado por ejemplo.

Como `init` ya los tiene abierto a esos tres archivos, cuando hace un `fork()`, se copia esos archivos, o sea se heredan. Esta tabla está en el kernel en una tabla, no en la memoria del proceso.

La llamada al sistema `open(filename, mode)` retorna un nuevo descriptor de archivo, el cual puede verse como un índice de la tabla ofiles (opened files).

El modelo de dos pasos `fork()/exec()` permite al proceso padre modificar el ambiente de ejecución del nuevo programa a lanzar. En particular es posible redireccionar las entradas y salidas.

El siguiente ejemplo muestra cómo es posible lanzar un proceso hijo que en lugar de escribir en la entrada estándar lo hará en el archivo `/tmp/out.txt`.

```
#include <unistd.h> /* UNIX syscalls */
#include <fcntl.h> /* file operations and flags */

int main(void) {
    if (fork() == 0) {
        /* in child: open/create file with permissions (user: rw, group:r) */
        int fd = open("out.txt", O_WRONLY | O_CREAT, 0640);
        printf("File fd: %d\n", fd); /* fd should have the value 3 */
        close(1); /* close stdout */
        dup(fd); /* duplicate fd: ofiles[1] == fd */
        write(1, "Hello world\n", 12);
    }
}
```

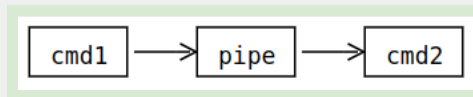
El `fd` probablemente sea el 3 (fd de file descriptor) porque es el primero vacío de la tabla ofiles, y el `dup(fd)` va a copiar ese `fd` en el file descriptor 1.

`close(1)` cierra el descriptor 1, que es `stdout` (salida estándar). Esto libera el descriptor 1 en la tabla de archivos abiertos del proceso, dejándolo disponible para ser reutilizado. `dup(fd)` duplica el descriptor de archivo `fd` (que apunta a `out.txt`) y lo asigna al menor descriptor disponible en la tabla de archivos abiertos. Como acabamos de cerrar el descriptor 1 (`stdout`), `dup(fd)` asignará el descriptor 1 al archivo `out.txt`. Ahora, cualquier cosa que el proceso escriba en `stdout` (descriptor 1) irá al archivo `out.txt` en lugar de la pantalla.

El file descriptor es el índice de la tabla `ofiles`, que nos devuelve el `open` por ejemplo.

Pipes

Un **pipe** es un mecanismo de comunicación (unidireccional) entre procesos. Un comando de shell de la forma `cmd1 | cmd2` hace que se lancen concurrentemente `cmd1` y `cmd2` previa redirección de la salida estándar al extremo de escritura del pipe y la entrada estándar de `cmd2` al extremo de lectura del pipe. (La salida de `cmd1` se convierte en la entrada de `cmd2`). Gráficamente:



`cmd1` escribe los datos en el pipe y `cmd2` los lee,

Por ejemplo: `grep -i 'google.com' file2.txt | wc -l`

Otro ejemplo, tenemos los siguientes dos comandos:

`cat file.txt | wc`

Estos dos comandos se van a ejecutar concurrentemente, el primer comando va a escribir por ese pipe datos y el `wc` los va a leer cuando se los mande. Para crear ese pipe el shell por dentro usa la función `pipe()`.

La llamada al sistema `pipe(p)` retorna en el argumento de salida `p` dos file descriptors (enteros). El primer elemento del arreglo corresponde al extremo de lectura del pipe (que usara `cmd2`) y el segundo al de escritura (usado por `cmd1`). `pipe(p)` crea un pipe y devuelve dos descriptors de archivo en el arreglo `p`:

- `p[0]`: descriptor para el extremo de lectura.
- `p[1]`: descriptor para el extremo de escritura.

El siguiente ejemplo, muestra su uso.

```

#include <unistd.h> /* UNIX syscalls */
#include <stdio.h>

#define N 20

int main(void) {
    int p[2];
    pipe(p);    /* create pipe */

    if (fork() == 0) {
        /* in child: read from pipe */
        char buffer[N];
        close(p[1]);    /* close write pipe descriptor */
        read(p[0], buffer, N);
        printf("Child read: %s", buffer);
    } else {
        /* in parent */
        close(p[0]);    /* close read pipe descriptor */
        write(p[1], "Hello from parent\n", 18);
    }
}

```

Si el cmd1 escribe y el pipe se llena, el cmd1 deja de escribir hasta que el cmd2 lea.

P[0] es el descriptor de lectura del pipe.

Es buena práctica cerrar el extremo del pipe que no se va a usar, por ejemplo, el hijo hace un `close(p[1])` porque va a leer no escribir, y el padre hace lo opuesto.

Tiene un error el ejemplo este, el `write` del padre tiene que tener un 19 no un 18. Si quisiéramos que el hijo le responda al padre, tendríamos que crear otro pipe, porque los pipes son unidireccionales.

Si quieres comunicar dos procesos que no son padre e hijos que no estén en el mismo ejecutable tienes que hacerlo con *named pipes*, en donde se crea un archivo especial llamado FIFO, porque no se pueden comunicar con las pipes normales. Creo un archivo FIFO con el comando `mkfifo`.

Cabe aclarar que un pipe es un medio de comunicación unidireccional. Si el escritor también lee del pipe podría consumir su propia escritura.

Para lograr una comunicación bidireccional debemos usar dos pipes.

El pipe internamente es un buffer acotado y el kernel implementa el clásico patrón productor/consumidor. El proceso que escribe puede bloquearse porque el buffer se llenó y el lector puede bloquearse si el buffer está vacío.

Un escritor bloqueado se desbloqueará cuando el lector consuma datos del pipe. Conversamente, el lector se desbloqueará cuando el escritor produzca datos.

Se debe notar que un pipe sólo puede utilizarse por procesos relacionados padre/hijo. Dos procesos independientes pueden comunicarse por un named pipe o FIFO. Un FIFO es un archivo especial asociado a un pipe gestionado por el kernel. Uno de los procesos lo abre para escritura y el otro para lectura. Las operaciones `read` y `write` leerán o escribirán en el pipe asociado y se sincronizan de la misma manera.

Señales

Las señales son notificaciones de eventos.

Las señales son un mecanismo básico de comunicación entre procesos para implementar la notificación de eventos. El kernel en algunos casos abstrae una interrupción o excepción en una señal enviada a un proceso. También un proceso puede enviar una señal a otro proceso. Este mecanismo fue desarrollado en las primeras versiones de UNIX en la década de 1970.

La llamada al sistema `kill(pid, signal)` envía la señal `signal` al proceso identificado con `pid`. Cada señal tiene identificadores numéricos. El sistema envía señales a los procesos cuando el usuario en una terminal pulsa las combinaciones de teclas como por ejemplo `Ctrl-C`, la cual envía `SIGINT` (interrupt) o `Ctrl-Z`, que envía `SIGTSTP`.

Un proceso puede manejar una señal `s` instalando un signal handler por medio del syscall `signal(sig, handler)`, donde `sig` es la señal a manejar y `handler` es una función de la forma `void handler(int signal)`.

Un handler es una función que se va a ejecutar cuando el proceso reciba una señal.

Por omisión cada señal tiene un default handler implementado en el kernel. Su comportamiento depende de cada señal. Por ejemplo, para la señal `SIGINT` el default handler finaliza el proceso, mientras que el default handler para `SIGTSTP` causa que el proceso se suspenda (queda en estado `SLEEPING`).

Cuando un proceso recibe una señal, se altera su flujo de ejecución normal y se dispara su handler. Luego del retorno, el proceso continúa con su ejecución en el estado que estaba antes de la recepción de la señal.

La única señal que no puede ser atrapada (manejada) es `SIGKILL` la cual se usa para terminar un proceso (si es que el usuario es el usuario efectivo, es decir, quien lo ejecutó).

Alarmas

Las llamadas al sistema `alarm(seconds)` y `setitimer(...)` permiten definir una alarma, es decir que luego de transcurrido ese tiempo, el kernel lanzará la señal `SIGALRM` al proceso.

Una alarma generalmente se usa cuando una aplicación realiza una operación (por ejemplo, enviar un mensaje) y si no tiene respuesta dentro de un intervalo de tiempo (timeout), requiere realizar alguna acción de recuperación o re-intento.

Una alarma es como una especie de timeout.

Trazado de procesos

Un proceso puede controlar la ejecución de otro proceso. Al primero se lo conoce como el **trazador** y al otro el **trazado**. Comúnmente un debugger se implementa usando este mecanismo, entre otros.

La llamada al sistema `ptrace(request, pid, addr, data)`. El proceso *trazado* se suspende cada vez que envíe una señal o haga una llamada al sistema. El *trazador* será notificado en un `waitpid(pid, &status)`. En `status` se indica la causa de la suspensión del proceso trazado.

Luego, el *trazador* puede realizar alguna de las siguientes acciones sobre el *trazado*:

- Identificar qué llamada al sistema realizó.
- Leer o escribir en el área de código o datos. Así un debugger, puede por ejemplo, definir un breakpoint por software, reemplazando una instrucción por una llamada al sistema.
- Acceder al estado de la CPU (valores de los registros) del proceso bloqueado.
- Hacer que continúe la ejecución
- Otras. Para más detalles, hacer `man ptrace`.

A continuación se muestra un ejemplo simple que intercepta la primera llamada al sistema hecha por el proceso hijo (en GNU-Linux arquitectura `x86_64`).

```

#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <sys/user.h>
#include <sys/reg.h>
#include <sys/syscall.h>
#include <stdio.h>

int main()
{
    pid_t child;
    long rax;
    int status;
    child = fork();
    if(child == 0) {
        ptrace(PTRACE_TRACEME, 0, NULL, NULL);
        execl("/bin/ls", "ls", NULL);
    }

```

```

else {
    while (1) {
        // wait for a child syscall
        wait(&status);
        // finish tracer if child did an exit syscall
        if (WIFEXITED(status))
            break;
        // get value of saved rax register of child process
        orig_rax = ptrace(PTRACE_PEEKUSER, child, 8 * ORIG_RAX, NULL);
        printf("Child syscall number %ld\n", orig_rax);
        // continue tracing next syscall
        ptrace(PTRACE_SYSCALL, child, NULL, NULL);
    }
}
return 0;
}

```

El comando strace permite rastrear un proceso.

Algo random:

```

marcelo@marcelo-linux-desktop:tmp$ cat file.txt > file2.txt
marcelo@marcelo-linux-desktop:tmp$ diff file.txt file2.txt
marcelo@marcelo-linux-desktop:tmp$ cat file2.txt
hola mundo

```

Ahí se hizo una copia del file.txt y lo guardó en file2. diff te dice la diferencia entre ambos archivos.

Arquitectura RISC-V

Estándar de una arquitectura abierta de un conjunto de instrucciones (ISA) basado en Reduced Instruction Set Computer (RISC).

El proyecto se inició en 2010 en la Universidad de California, Berkeley. Actualmente soportado por el consorcio RISC-V International.

Características:

- Arquitectura *load-store*. Esto significa que no tenemos instrucciones que operan sobre direcciones de memoria, su operando no es una dirección de memoria, es un registro. Esto hace que la arquitectura sea más sencilla. El acceso a la memoria es mucho más lento que acceder a los registros.
- Conjunto de instrucciones base de 32 bits. Aquí las instrucciones son de tamaño fijo y de 32 bits. Cada instrucción empieza en direcciones de memoria que sean múltiplos de 4.
- Soporta extensiones (floating point, . . .). Es una arquitectura abierta que soporta extensiones.
- Variantes con espacios de direcciones de 32, 64 o 128 bits.

Cores: un core es un componente que tiene una unidad de fetch de instrucciones independiente. Un core es como si fuera una CPU. En risc-v a un core lo vamos a llamar hart. Conceptualmente es un core.

Hart: hardware thread. Un core puede soportar múltiples harts.

Coprocesadores: un core puede incluir extensiones. Cada coprocesador puede extender el conjunto de instrucciones base.

Registros:

Name	Asm name	Description	Saved by
x0	zero	Contiene 0	
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	
x4	tp	Thread pointer	
x5-x7	t0-t2	Temporaries	Caller
x8	fp	Frame pointer	Callee
x9	s1	Saved register	Callee
x10-x17	a0-a7	Arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller

Los registros pueden ser de 32 (RV-32) o 64 bits (RV-64).

Esos son los registros que tiene la CPU. En el Assembly vamos a usar los nombres *Asm name*.

El **zero** es un registro constante que tiene un cero siempre.

El **x1** la CPU lo usa como dirección de retorno, guarda el valor del program counter para después poder volver a esta instrucción que se ejecutó. Se hace un call a otro proceso y se guarda el *program counter* de la invocante en ese registro. Si la función no llama a nadie, no hace falta guardar el *program counter* ahí, porque no hace falta. Esto es una ventaja, una optimización porque no tenemos que apilar nada en la memoria al pedo. El PC siempre apunta a la instrucción que se está ejecutando, no a la siguiente como INTEL.

Global pointer se puede usar para tener un puntero a un dato global.

Cada CPU cuando inicia va a guardar el hart id en el **tp**.

Las funciones reciben sus argumentos en a_0-a_7 y retornan el valor en a_0-a_1 .

Instrucciones no privilegiadas:

Asm. Instruction	Description
li rd, value	Load immediate: $rd = value$
la rd, address	Load address: $rd = mem[pc - address]$
l{b h w d} rd, address	Load: $rd = mem[pc - address]$
s{b h w d} rs, addr, rb	Store: $mem[rb + pc - offset] = rs$
mv rd, rs	Copy register: $rd \leftarrow rs$
add rd, rs1, rs2	Add: $rd = rs1 + rs2$
addi rd, rs1, v	Add immediate: $rd = rs1 + v$
sub rd, rs1, rs2	Sub:: $rd = rs1 - rs2$
blt r1, r2, offset	Branch: if $rt < rs$, $pc = pc + offset$
beq r1, r2, offset	Branch: if $rt = rs$, $pc = pc + offset$
j offset	Jump: $pc = pc + offset$
jal offset	Jump and link: $ra = pc$; $pc = pc + offset$
ret	Return from subroutine: $pc = ra$
call offset	As jal (with long offset)
fence	Memory barrier: no reorder load/stores
amoswap.w.aq rd, rs, s	Atomic swap: $rs \leftrightarrow mem[s]$; $rd = old\ mem[s]$

li rd, value: el valor de la primera instrucción no puede ser de 32bits, porque la instrucción se codifica con 32 bits, entonces no puedes meter el value en esos 32bits, porque necesitas el resto de las cosas metidas ahí. Si queremos cargar un valor de 32bits vamos a tener que usar dos instrucciones.

La instrucción **la** es cargar una dirección de memoria relativa con respecto al program counter. Si fuera una dirección absoluta tendríamos que usar 32bits. Los registros son de 32bits.

Load (**l**) y store (**s**). En el **s** el **rb** es un offset relativo al program counter.

El **j** es equivalente a un goto, el **jal** es como un **call**.

El **fence** es como un flush de la cache. Esto se usa cuando tenemos múltiples cores y uno escribe en una variable pero esto se queda en la caché y no en la RAM entonces otro core se caga si quiere leerla porque va a leer algo viejo. El **fence** es como que lo flushes a la RAM. Aparte provoca que no se pueda cambiar el orden en el que se ejecutan las instrucciones.

Los argumentos no se van a apilar en la pila, se van a pasar por registros que son los de a_0 - a_7 . El valor retornado por una función debe quedar guardado en a_0 - a_1 .

Modos de ejecución: privilegios

Level	Description
0	User (U) or application
1	Supervisor (S) (kernel)
2	Hypervisor (H) (for VMs)
3	Machine (M)

Cada core inicia en M mode (más privilegiado). Cada modo tiene un conjunto de registros de control y estado (CSRs).

Nota: trabajaremos con un board RISC-V32 con modos U,S,M.

Al modo supervisor lo vamos a usar para ejecutar el código del kernel. El machine mode es el más privilegiado de todos. La máquina va a arrancar con ese modo y después lo cambiamos a supervisor. Todas las instrucciones **csr** son privilegiadas.

CSRs (en machine y supervisor) modes

Machine mode	Supervisor mode	Description
mstatus	sstatus	Status register
mie	sie	Interrupts enable/pending
mtvec	stvec	Trap handler/vector base address
mscratch	sscratch	Pointer to data area (for ISR)
mepc	sepc	Saved program counter on trap/irq
mcause	scause	Trap cause (interrupt number)
mtval	stval	Bad address or instruction
mip	sip	Interrupts pending
	satp	Address of root page table
pmpcfg0-63		Phys. memory protection address
medeleg		Exceptions delegation
mideleg		Interrupts delegation
mhartid		Hart (core) number

Cambios de privilegios

La instrucción `ecall` cambia a un nivel de privilegio mayor. Se usa comúnmente para implementar *syscalls*.

Pasos al pasar a modo supervisor desde modo usuario:

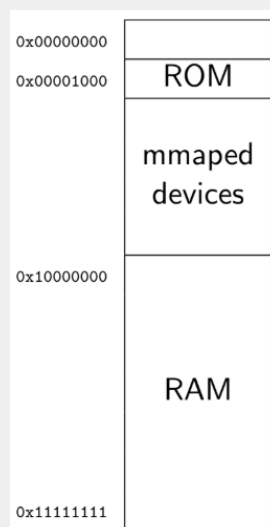
1. La CPU salva el pc en `sepc`
2. Cambia el pc a `stvec` (*trap vector address*)

La instrucción `sret` (ejecutada en el trap handler: retorna al nivel de privilegio anterior (configurado en `sstatus`). El pc toma el valor del registro `sepc`.

En machine mode funciona de manera similar pero se debe usar `mret`. El nivel de privilegio a retornar se configura en `mstatus`.

El pc toma el valor del registro `mepc`.

RV-32: espacios de memoria física



ROM: A partir de la dirección 0x00001000.

Luego, espacio para dispositivos mapeados en memoria (registros de los controladores de dispositivos).

RAM a partir de 0x10000000 (2GB).

Tres fuentes de instrucciones que vamos a tener: teclado de la consola, el reloj del sistema para que genere interrupciones periódicas, y el disco. Esto es porque son los únicos dispositivos que vamos a tener.

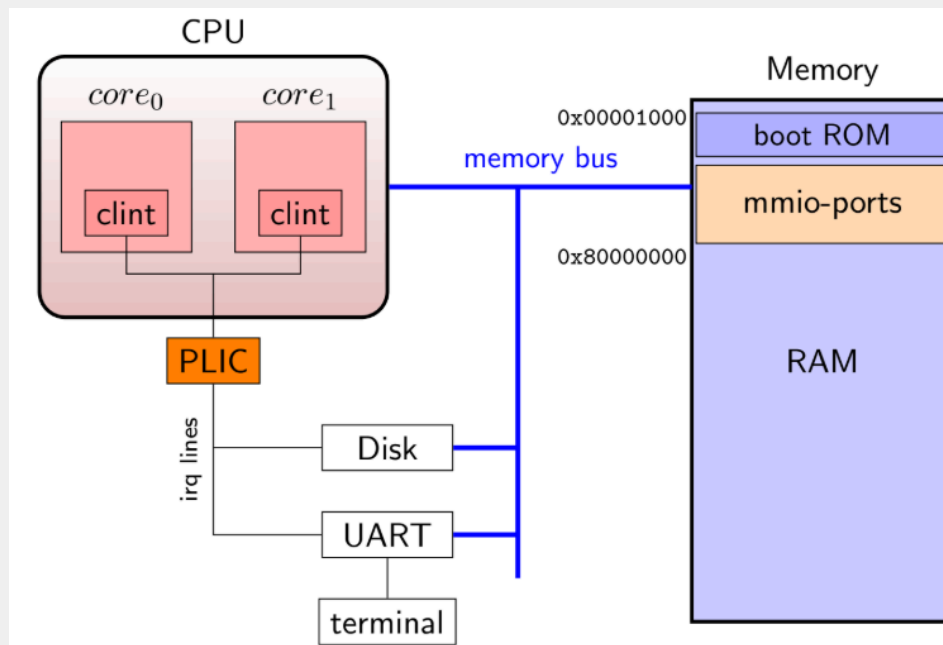
Interrupciones

- Los dispositivos se mapean en memoria. No existe un bus de I/O dedicado.
- Cada core tiene un core line interrupt controller (clint).
- Una plataforma (board) comúnmente tiene un Programmable Interrupt Controller (PLIC) en cual puede configurarse para rutear interrupciones a diferentes cores.
- Las interrupciones se atrapan en machine mode pero pueden delegarse a otros modos (comúnmente supervisor mode).
- La delegación se configura en los CSRs `medeleg` (excepciones) y `mideleg` (interrupciones)
- El csr `stvec` contiene la dirección base del trap handler o la base del vector de interrupciones (vector de punteros a diferentes handlers). En el ultimo caso, en una interrupción, en RV-32, `pc = stvec[4 * scause]`

Plataforma Qemu virt (RV-32)

- Arquitectura de 32 bits
- Bus de direcciones fisico de 32 bits (4GB)
- Múltiples cores (flag `-smp n`)
- Controlador de interrupciones PLIC
- Chip de Universal Asynchronous Transmitter-Receiver (UART) serial device
- Interfaces virtio-mmio
- Puente genérico de un host PCI
- Real Time Clock (RTC)
- Memoria flash (con boot firmware)

Esquema del board QEMU virt (RV32)



Dispositivos

CLINT: Core Level Interrupt controller

- Dispositivo mapeado a partir de la dirección `0x2000000`
- Registro `MTIMER` de 64 bits en `0x200BFF8`

- Por cada core un registro comparador MTIMECMP mapeado en $0x2004000 + i$ (para cada core i)
- Cada core tiene su correspondiente

Funcionamiento

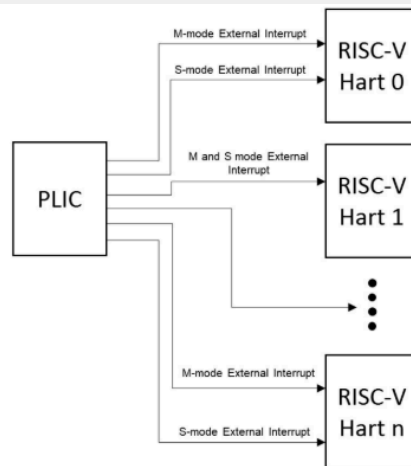
1. MTIMER contiene en número de ciclos desde el inicio (a una frecuencia fija pre-establecida).
2. Un core i pone en $MTIMECMP[i]$ un valor
3. Cuando $MTIMER \geq MTIMECMP[i]$ genera una interrupción pendiente en el core i (set un bit en $mpeip$)

PLIC: Platform Level Interrupt Controller

Registros:

1. $M/SENABLE[hart]$,
2. $M/SPRIORITY[hart]$
3. $M/SCLAIM[hart]$ (irq number)

- Ante una interrupción el CSR mip/sip queda con el bit setado con la línea de interrupción. También se setea en $m/scause$ el *interrupt bit* de y el *exception code*



PLIC: Diagrama de bloques.

PLIC: Operación

1. Un dispositivo genera una interrupción
2. PLIC notifica a un core (*hart*) seteando el bit correspondiente en m/sip (*interrupts pending*) y $m/scause$ (nro de interrupción)
3. El core dispara la ISR apuntada por $m/stvec$
4. La ISR lee (*claim*) el número o línea de *irq* desde $M/SCLAIM[hart]$ ¹
5. El PLIC retorna el número de *irq*
6. La ISR notifica que procesó la interrupción escribiendo en $M/SCLAIM[hart] \leftarrow irq_number$
7. El PLIC puede generar otra interrupción

Para más detalles, ver `plic.c`

Qemu RISC-V generic virtual platform devices

Universal Asynchronous Receiver-Transmitter (UART)

- Transmisión serial (de a bytes)
- Usado comúnmente para conexión de terminales (ttys)
- Detalles: <http://byterunner.com/16550.html>

Virtio devices

- Estándar propuesto para *dispositivos virtuales*
- Documentación: <https://docs.oasis-open.org/virtio/virtio/v1.1/virtio-v1.1.pdf>
- Sólo usaremos un driver para el disco.

Herramientas de desarrollo

Para el desarrollo de software en un lenguaje de programación se requiere de un conjunto de herramientas conocido como toolchain que contiene al menos:

- **Compilador:** encargado de traducir programas fuente en assembly.
- **Ensamblador (assembler):** genera archivos objeto (binarios) en un formato dado (ej: ELF). ELF es un formato de archivos objetos.
- **Enlazador (linker):** combina archivos objetos y bibliotecas para generar ejecutables o bibliotecas. El ejecutable lo genera el linker, no el assembler. El linker agarra los archivos objetos y los enlaza con las bibliotecas necesarias para luego generar un ejecutable.

Assembly es el lenguaje, assembler es el traductor/ensamblador.

Nota: en el caso de los lenguajes C/C++ un programa fuente se pre-procesa para expandir las macros y directivas incluidas. Es un preprocesador que expande todas esas macros y directivas (son los include que tienen el #, etc). Esto se hace antes de compilar. Este preprocesador se llama cpp.

Dados los archivos fuente:

```
/* main.c */
#include <stdio.h> /* for printf() */

extern char* hello(void);

int main(void)
{
    printf("%s\n", hello());
    return 0;
}
```

Y

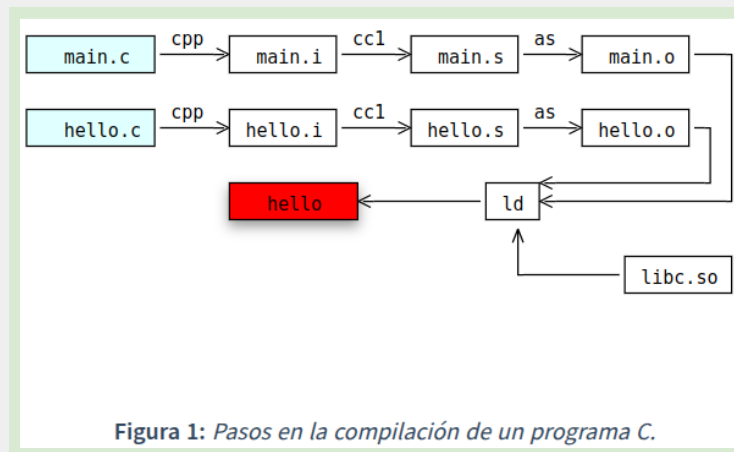

```

/* hello.c */
#define hi "Hello world"

char *hello(void)
{
    return hi;
}

```

El siguiente diagrama muestra el proceso de compilación de la aplicación mediante el comando `gcc -o hello main.c hello.c`. Este comando genera el programa (archivo ejecutable) `hello` desde los programas fuente `main.c` y `hello.c`.



El `cc1` es el compilador de `gcc` (que en realidad `gcc` es un driver) este genera el archivo assembly `main.s`. Todos estos archivos generados no los vemos porque se generan en un `/tmp`. El linker es el comando `ld`, el cual toma los dos archivos objetos y la biblioteca estándar `libc.so` (que tiene la función `printf`) y genera el ejecutable. Los `.o`, el `.so` y el ejecutable están en formato ELF.

Como se puede apreciar en la figura 1, la compilación de un programa C consiste de varios pasos producidos por el comando `gcc`, el cual es un driver del proceso:

1. **Pre-procesamiento:** se procesan las directivas del programa fuente y se expanden las macros encontradas. La salida es un archivo temporal (generalmente con extensión `.i`).
2. **Compilación:** el compilador (`cc1`) traduce de C a assembly.
3. **Assembler:** el ensamblador (`as`) produce los archivos objeto (binarios) correspondientes en el formato soportado por el SO como ELF en sistemas GNU-Linux.
4. **Enlazado (linking):** el linker (`ld`) enlaza (combina y resuelve las referencias externas) de los archivos objeto y bibliotecas (en este ejemplo, la biblioteca estándar `libc.so` y genera el ejecutable final).

Estos pasos pueden ser controlados mediante el comando `gcc`. Los siguientes flags permiten detener el proceso en sus diferentes etapas:

- `-E`: parar luego de la etapa de pre-procesamiento. Va a correr solo el preprocesador.
- `-S`: parar luego de generar el assembly.
- `-c`: parar luego de generar el archivo objeto (`.o`), es decir, no invocar al linker.

Los programas se compilan en direcciones virtuales de memoria, son virtuales porque sino todos los programas estarían en la misma memoria si empiezan desde la dirección de memoria 0, se reescribirían.

Cuando los compilas los programas aparecen desde la dirección de memoria 0, pero esa es una dirección virtual/lógica, no son direcciones reales porque no puedes cargar todos los programas a partir de la dirección 0. No las tenemos que tomar como direcciones físicas o reales. Estos programas en realidad se cargan en direcciones que están libres.

Archivos Objeto

Un archivo objeto (.o) es un binario en el formato definido por la ABI del sistema. En sistemas GNU-Linux el formato es ELF, en Mac-OS es Mach-O y en MS-Windows son los formatos Portable Executable (PE) y Common Object File Format (coff) (ver PE format).

Si bien estos formatos difieren en detalles, un archivo objeto o ejecutable tienen las siguientes partes (denominadas secciones y/o segmentos).

- Un header que describe el tipo de archivo, arquitectura, entry point, etc.
- Tipos de secciones:
 - Data: valores de constantes y variables globales. Estas secciones pueden a su vez estar divididas en datos inicializados y no inicializados.
 - Text: instrucciones de programa.
 - Symbol table: tabla de símbolos que relacionan identificadores (funciones, variables, constantes, etc) con sus direcciones de memoria.
 - Reallocation entries: direcciones en el programa de las referencias externas, es decir aquellos símbolos definidos en otros módulos (archivos objeto o bibliotecas). Estas direcciones generalmente corresponden a los operandos de instrucciones (call f, load/store d, ...) que referencian a símbolos externos y que luego deberán ser resueltos por el linker.

Enlazado (linking)

El linker (o link-editor) es el encargado de producir los binarios (ejecutables y bibliotecas) en base a la composición de varios archivos objeto y bibliotecas.

Esta combinación de archivos binarios consiste de dos pasos:

1. Concatenar cada archivo objeto y recalcular las direcciones de las variables, constantes y funciones.
2. Resolver las direcciones de las instrucciones tipo call address. Estas instrucciones son reallocation entries. Algunas pueden ser referencias externas (definidas en otro archivo objeto o biblioteca).

Dados los programas de ejemplo de arriba, el módulo main.o contiene dos referencias externas: hello y printf. Estos símbolos aparecen como *reallocation entries*. Es posible ver con objdump -d main.o el disassembly de las instrucciones en la sección de texto.

```
Text section:
_main:
00000000    ...
00000018    call 0x18          ; call hello (UNRESOLVED)
0000001c    call 0x1c          ; call printf (UNRESOLVED)
Reallocation Entries:
address      type      symbol
0x0000001c   call32   _printf
0x00000018   call32   _hello
```

Las instrucciones `call <address>` están sin resolver: `address` apunta a ellas mismas o 0, dependiendo de la plataforma. Estas instrucciones aparecen en la tabla de reallocation entries para que el linker las resuelva. Se puede ver que en los archivos objeto `main.o` y `hello.o`, las funciones `_main` y `_hello` tienen ambas dirección 0.

El comando `gcc -o myprog main.o hello.o` invoca al linker el cual enlaza los archivos objetos (y la biblioteca estándar) y genera el ejecutable `myprog`.

Al analizar la sección `text` de `myprog` (con `objdump -d myprog`) se puede observar que las direcciones de las instrucciones `call` se han resuelto.

```
...
_main:
...
00003f50    call _hello      ; call 0x3f68
00003f54    call 0x10003f74 ; stub for _printf
...
...      ret
_hello:
00003f68    ...
...      ret
```

El linker combinó ambos módulos, recalculó direcciones (`_hello` tiene una dirección a continuación de la última instrucción de `_main`) y resolvió la dirección del `call` a la dirección de comienzo de `_hello`.

Se debe notar que también se resolvió la dirección de la invocación a `_printf` la cual está definida en la biblioteca estándar (de enlace dinámico). A continuación se describen los detalles sobre bibliotecas y dynamic linking.

Cuando se ensambla y no se ejecuta el linker todavía, las cosas externas que se usan van a aparecer inconclusas, porque el linker es el que las tiene que resolver después. Esto se ve si hacemos el `objdump` y vemos que en los `call` de las funciones externas o se llaman a sí mismas o llaman a las instrucciones que sigue (lo cual está mal pero es porque no se resolvieron todavía).

Bibliotecas

Una **biblioteca** es una colección de módulos (código y datos) reusables. Una biblioteca generalmente contiene un conjunto de archivos objeto (`.o`). Pueden ser de dos tipos:

- Para *enlazado estático*.
- De *enlazado dinámico* (shared).

Una biblioteca de enlazado estático consiste en un contenedor (formato `ar` en sistemas tipo UNIX) de archivos objetos. Un contenedor incluye un índice con los nombres de los archivos objeto que contiene.

En sistemas tipo UNIX se puede generar una biblioteca estática con el comando:

```
ar rcs libmylib.a file1.o file2.o ..
```

Las bibliotecas estáticas usan como convención la extensión `.a`.

El comando `ar` es por `archiver`. Hacer `man ar` para más detalles. Generalmente una biblioteca `B` se nombra de la forma `libB.a`. Continuando con el ejemplo anterior, es posible crear `libhello.a` con

```
ar rcs libhello.a hello.o
```

y crear el ejecutable myprog con

```
gcc -o myprog -L $PWD -l hello
```

El flag -L \$PWD adiciona el directorio corriente al path de búsqueda de ld.

Enlazado Estático y Dinámico

En nuestro SO vamos a hacer siempre enlazado estático, los binarios que vamos a obtener van a ser autocontenidos, van a tener ya todo lo necesario. Todo lo que tenemos en el kernel en su binario está todo contenido ahí.

Enlazado dinámico: se enlaza durante la ejecución. Las bibliotecas están una sola vez y los programas la comparten, no está duplicada por cada programa que la usa. El enlazado dinámico se llama lazy linking también porque se demora todo lo posible hasta la invocación de la función para linkear.

Los sistemas operativos modernos soportan shared libraries o * que son enlazadas dinámicamente, es decir en tiempo de ejecución del proceso. Estas bibliotecas también se conocen como *dynamic linking libraries (DLLs).

La ventaja es que durante la ejecución se carga una única copia en memoria de la biblioteca que será enlazada y se comparte por todos los procesos que la usen. Esto hace que los archivos ejecutables sean más pequeños y no haya código repetido de las bibliotecas de uso común.

Como desventaja es que se produce una pequeña sobrecarga en ejecución ya que se debe realizar el enlazado antes de comenzar la ejecución de cada proceso. Veremos que esta sobrecarga se puede reducir significativamente con lazy linking.

Enlazado estático: los ejecutables son más grandes porque tienen todo ahí, la biblioteca estándar ponele si es que la usa. Cada ejecutable tiene su propia copia de las bibliotecas entonces eso se duplica.

El enlazado estático tiene como ventaja que el ejecutable resultante es autocontenido, es decir que todas sus dependencias están incluidas en el archivo.

La desventaja es que si todas las aplicaciones tendrían incluidas las bibliotecas de uso común (como la estándar), ese código estaría replicado en cada aplicación, consumiendo mayor espacio en el sistema de archivos y en la memoria cuando estén en ejecución.

Es posible generar la biblioteca libhello.so de enlazado dinámico haciendo:

```
gcc -c -fPIC hello.c
gcc -shared -o libhello.so hello.o
```

El primer comando genera un archivo objeto con position independent code. Esto es requerido ya que la biblioteca se enlazará con diferentes direcciones de memoria base a cada proceso.

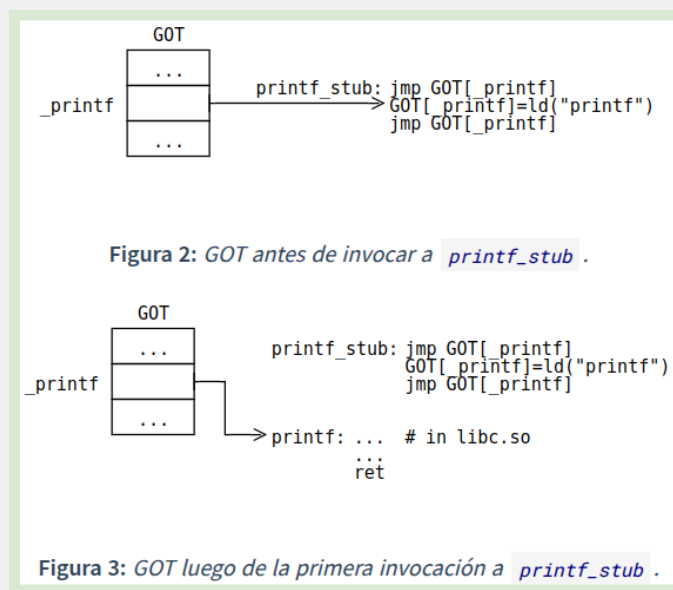
Es posible que una biblioteca tenga sus dos versiones: estática y dinámica. Generalmente el linker prefiere la versión dinámica a menos que se especifique el nombre específico (ej: hello.a).

Cuando se realiza dynamic linking el linker genera una función para invocar a la biblioteca del linker (ld.so) que determina cuáles son las bibliotecas dinámicas requeridas, las carga (bajo demanda) y resuelve los símbolos externos del programa.

Muchos sistemas operativos modernos realizan lazy linking, es decir el enlazado a una función se realiza en su primera invocación. Esto reduce la latencia inicial en la ejecución del proceso.

Con lazy linking el linker genera por cada función externa una pequeña rutina (stub) en el ejecutable la cual es responsable de resolver ese símbolo de función en su primera invocación. El linker resuelve estáticamente las invocaciones a estos stubs.

Por ejemplo, en una llamada a `printf` se invoca a `printf_stub`. Cada *stub* usa su entrada correspondiente en la Global Offset Table (GOT) que contiene las direcciones de las direcciones de las funciones a resolver. Inicialmente esas entradas apuntan a una instrucción especial del stub que invocará a la función de biblioteca del linker para resolver ese símbolo en particular. En el retorno, la entrada en la GOT se reemplaza por la dirección de la función resuelta. En las próximas invocaciones al stub, éste saltará a la función ya resuelta. Este proceso se muestra en las siguientes figuras para la función `printf`. El linker genera la GOT y los stubs. El stub correspondiente a `printf` se denota aquí como `printf_stub`. En el programa, en cada llamada a `printf` se compiló a una llamada a `printf_stub`. Inicialmente, la `GOT[_printf]` apunta a una instrucción de `printf_stub` que invocará al linker.



La figura 3 muestra que luego de la primera invocación a `printf_stub` la entrada `GOT[_printf]` apunta directamente a `printf` de la biblioteca estándar. En las siguientes invocaciones a `printf_stub` ésta salta a `printf` (ya resuelta).

La sobrecarga es que en cada invocación se requiere un salto adicional (o indirecto).

En linux, el linker te genera una stub por cada función, en cambio en mac te genera una sola en total. Los stubs en linux se llaman PLT (Procedure Linking Tables).

Build Systems

Un sistema de desarrollo incluye herramientas de automatización de los pasos de producción del software. Para la construcción de un componente de software se debe tener en cuenta sus dependencias y los comandos a aplicar en cada paso del proceso de compilación y enlazado hasta obtener los programas y/o bibliotecas finales.

GNU build system

Este sistema se basa en la herramienta make, la cual se basa en una especificación de los objetos o targets a construir y sus dependencias. La especificación se escribe en un archivo Makefile y consiste en una secuencia en reglas de la forma:

```
target: dependencies
<tab> build command
```

A modo de ejemplo, el programa myprog basado en los archivos fuente del capítulo anterior, puede especificarse en la siguiente regla:

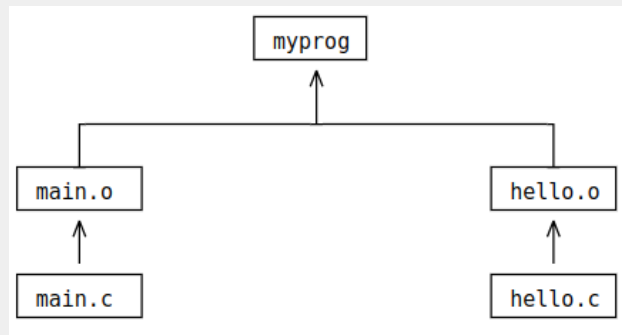
```
myprog: main.o hello.o
    gcc -o myprog main.o hello.o

main.o: main.c
    gcc -c main.c

hello.o: hello.c
    gcc -c hello.c
```

La primera regla determina que para generar el ejecutable myprog, el cual depende de los archivos objeto main.o y hello.o, se construye con el comando gcc -o \$@ \$?.

El programa make construye un grafo de dependencias para determinar qué debe reconstruirse dependiendo del timestamp (fecha+hora) de los archivos.



Por ejemplo, si el archivo main.o no existe o es más viejo que main.c, deberá dispararse la segunda regla para reconstruir main.o. Luego, se deberá ejecutar nuevamente la primera regla en la cual se ejecuta el linker para generar el ejecutable final.

Estas reglas pueden escribirse de manera más compacta usando variables automáticas.

```
myprog: main.o hello.o
    gcc -o $@ $^

%.o: %.c
    gcc -c $<
```

El \$@ se refiere al myprogram, el \$^ se refiere a lo que va después del myprogram:

`%o: %.c` esto dice que todos los `.o` dependen de los `.c` correspondientes. Por ejemplo, `main.o` depende de `main.c`. O sea tenemos:

`main.o: main.c`

(tab) `gcc -c main.c` (esta es la regla de cómo generar el `main.o`)

El `myprogram` depende del `main.o` y `hello.o`, el `main` depende del `main.c` y el `hello.o` depende del `hello.c`. Cuando vengo a ejecutar el `myprog`, se fija si el `main.o` y el `hello.o` están en sincronía con sus `.c`. Si no lo están (un `.c` se generó después que el `.o`, o sea cambio) tiene que generarlo de vuelta, y sino no hace nada porque ya está todo listo.

Para más detalles sobre las variables usadas ver `automatic variables`.

El GNU build system incluye otras herramientas que permiten automatizar el proceso de compilación y enlazado de paquetes de software en forma independiente de la plataforma. Estas herramientas se conocen como las `autotools` que contiene los siguientes componentes:

- Autoconf: crea archivos de configuración y el script `configure` desde la especificación `autoconf.ac`.
- Automake: se basa en las directivas en `Makefile.am` y `configure.ac` y genera el `Makefile.in` que será usado por `make`.

Con estas herramientas un desarrollador genera un paquete `my-package.tgz` (contenedor `tar` comprimido).

Aquí se muestra un ejemplo de cómo crear un paquete simple.

Este paquete se distribuye y un usuario lo puede instalar siguiendo los siguientes pasos:

```
# tar xzvf my-package.tgz
# ./configure && make && sudo make install
```

Procesos

Un `thread` (hilo) es la secuencia de instrucciones que la CPU está ejecutando o ejecutará. El kernel `multithreaded` crea varios hilos de ejecución, generalmente para realizar ciertas tareas que le podrían llevar mucho tiempo para ejecutarlos (planificarlos) cuando la CPU esté idle (no haya un procesos de usuario listo para asignarle la CPU).

Un proceso tiene al menos un hilo de ejecución y se corresponde con una aplicación o programa de usuario.

En particular un `thread` de un proceso se ejecutará en modo usuario (el menor modo de privilegio de la CPU).

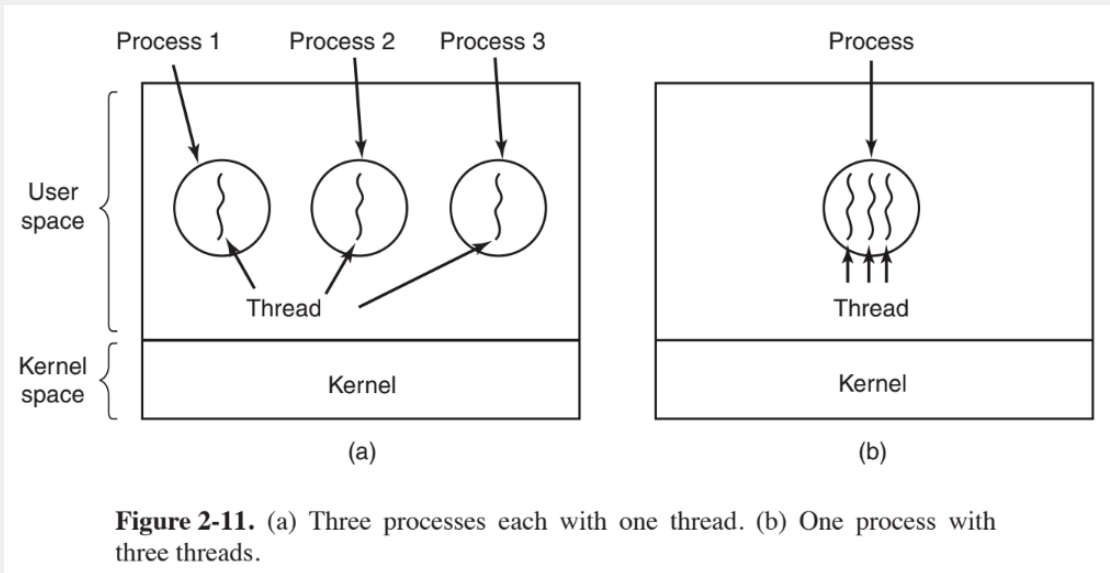
Una forma de ver a un proceso es considerarlo como una forma de agrupar recursos relacionados. Un proceso tiene un espacio de direcciones que contiene texto y datos del programa, así como otros recursos. Estos recursos pueden incluir archivos abiertos, procesos secundarios, alarmas pendientes, manejadores de señales y más. Al agruparlos en un proceso, se pueden gestionar con mayor facilidad.

Otro concepto que tiene un proceso es el de hilo de ejecución, un thread. El `thread` tiene un `program counter` que registra qué instrucción ejecutar a continuación. Tiene registros que almacenan sus variables de trabajo actuales. Tiene una `stack` que contiene el historial de ejecución, con un `frame` para cada procedimiento llamado pero aún no retornado. Aunque un `thread` debe ejecutarse en algún proceso, el `thread` y su proceso son conceptos diferentes y pueden tratarse por separado. Los procesos se utilizan para agrupar recursos; los `thread` son las entidades programadas para su ejecución en la CPU.

Lo que los `threads` aportan al modelo de procesos es permitir que se realicen múltiples ejecuciones en el mismo entorno del proceso, con gran independencia entre sí. Tener varios `threads` ejecutándose en paralelo en un proceso es similar a tener varios procesos ejecutándose en paralelo en una computadora. En el primer

caso, los threads comparten un espacio de direcciones y otros recursos. En el segundo, los procesos comparten memoria física, discos, impresoras y otros recursos.

El término **multithreading** se utiliza para describir la situación en la que se permiten varios threads en el mismo proceso.



In Fig. 2-11(a) we see three traditional processes. Each process has its own address space and a single thread of control. In contrast, in Fig. 2-11(b) we see a single process with three threads of control. Although in both cases we have three threads, in Fig. 2-11(a) each of them operates in a different address space, whereas in Fig. 2-11(b) all three of them share the same address space.

Todos los threads tienen exactamente el mismo espacio de direcciones, lo que significa que también comparten las mismas variables globales. Dado que cada thread puede acceder a todas las direcciones de memoria dentro del espacio de direcciones del proceso, un thread puede leer, escribir o incluso borrar la stack de otro thread. No hay protección entre threads porque (1) es imposible y (2) no debería ser necesaria. A diferencia de los procesos, que pueden provenir de diferentes usuarios y ser hostiles entre sí, un proceso siempre pertenece a un solo usuario, quien presumiblemente ha creado varios threads para que puedan cooperar, no pelearse entre sí. Además de compartir un espacio de direcciones, todos los threads pueden compartir el mismo conjunto de archivos abiertos, procesos secundarios, alarmas y señales, etc.

Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

Figure 2-12. The first column lists some items shared by all threads in a process. The second one lists some items private to each thread.

The items in the first column are process properties, not thread properties. For example, if one thread opens a file, that file is visible to the other threads in the process and they can read and write it.

Hilos de ejecución (threads)

Un ejemplo para entender mejor qué son los threads:

It is easiest to see why threads are useful by looking at some concrete examples. As a first example, consider a word processor. Word processors usually display the document being created on the screen formatted exactly as it will appear on the printed page. In particular, all the line breaks and page breaks are in their correct and final positions, so that the user can inspect them and change the document if need be (e.g., to eliminate widows and orphans—incomplete top and

Now consider what happens when the user suddenly deletes one sentence from page 1 of an 800-page book. After checking the changed page for correctness, he now wants to make another change on page 600 and types in a command telling the word processor to go to that page (possibly by searching for a phrase occurring only there). The word processor is now forced to reformat the entire book up to page 600 on the spot because it does not know what the first line of page 600 will be until it has processed all the previous pages. There may be a substantial delay before page 600 can be displayed, leading to an unhappy user.

Threads can help here. Suppose that the word processor is written as a two-threaded program. One thread interacts with the user and the other handles reformatting in the background. As soon as the sentence is deleted from page 1, the interactive thread tells the reformatting thread to reformat the whole book. Meanwhile, the interactive thread continues to listen to the keyboard and mouse and responds to simple commands like scrolling page 1 while the other thread is computing madly in the background. With a little luck, the reformatting will be completed before the user asks to see page 600, so it can be displayed instantly.

While we are at it, why not add a third thread? Many word processors have a feature of automatically saving the entire file to disk every few minutes to protect the user against losing a day's work in the event of a program crash, system crash, or power failure. The third thread can handle the disk backups without interfering with the other two. The situation with three threads is shown in Fig. 2-7.

If the program were single-threaded, then whenever a disk backup started, commands from the keyboard and mouse would be ignored until the backup was finished. The user would surely perceive this as sluggish performance. Alternatively, keyboard and mouse events could interrupt the disk backup, allowing good performance but leading to a complex interrupt-driven programming model. With three threads, the programming model is much simpler. The first thread just interacts with the user. The second thread reformats the document when told to. The third thread writes the contents of RAM to disk periodically.

It should be clear that having three separate processes would not work here because all three threads need to operate on the document. By having three threads instead of three processes, they share a common memory and thus all have access to the document being edited. With three processes this would be impossible.

Cada proceso (y el kernel) tienen al menos un hilo o thread de ejecución.

El estado de un thread en un punto dado de su ejecución está representado por:

- Registros de la cpu:
 - Program counter (pc): dirección de la próxima instrucción a ejecutar.
 - Stack pointer (sp): dirección del tope de la pila.

- Frame (o base) pointer (fp): dirección base del registro de activación del tope.
- Un stack: pila de registros de activaciones de funciones. (esto después lo sacó el profe de las notas de la teoría)

La pila contiene registros de activación para el control de la secuencia de invocaciones a funciones y sus retornos. Además comúnmente contienen los datos automáticos (variables locales y argumentos) de las funciones activas. De ese modo se crean y eliminan automáticamente los ambientes locales.

La representación de un registro de activación se basa en el calling convention de la ABI de la plataforma. El *frame pointer* se usa en las instrucciones del programa como dirección base para acceder en forma relativa a las variables locales y argumentos dentro del registro de activación.

Los procesos no comparten memoria, los threads si.

It is important to realize that each thread has its own stack. Each thread's stack contains one frame for each procedure called but not yet returned from. This frame contains the procedure's local variables and the return address to use when the procedure call has finished. For example, if procedure X calls procedure Y and Y calls procedure Z, then while Z is executing, the frames for X, Y, and Z will all be on the stack. Each thread will generally call different procedures and thus have a different execution history. This is why each thread needs its own stack.

Los pasos en una invocación a una función *f* son los siguientes:

1. *Invocante*: reserva espacio para el valor de retorno (de *f*), si es que se dejará en la pila. Si se retorna en un registro, este paso se omite.
2. *Invocante*: apila (o pone en registros) los valores de los argumentos.
3. *Invocante*: ejecuta la instrucción `call f`, que comúnmente apila el valor del program counter y luego le asigna la dirección de la primera instrucción de *f*.
4. En algunas arquitecturas (como RISC-V), el compilador generó instrucciones para apilar la dirección de retorno si `call` no lo hace.
5. *Invocada* (*f*): apila (salva) el valor del frame pointer y le asigna el valor del stack pointer. (Cada función invocada tiene un stack frame. El frame pointer estaba apuntando al stack frame de la función invocante. *f* (apila) ese valor viejo del fp en la pila, para no perderlo. Luego, *f* le asigna el valor del sp al fp, para marcar el inicio de su propio stack frame).
6. *Invocada* (*f*): posiblemente reserva espacio para las variables locales, restando un valor al stack pointer. Estos pasos (4 y 6) se conocen como el prólogo. Se apila restando la cantidad de bytes que van a ocupar al sp.
7. *Invocada* (*f*): ejecuta las instrucciones del cuerpo de *f*.
8. *Invocada* (*f*): comienza el retorno (epílogo). Se eliminan las variables locales y temporarios (`sp=fp`).
9. *Invocada* (*f*): recupera el base pointer salvado (`pop fp`).
10. *Invocada* (*f*): retorna (ejecuta la instrucción `ret`). En algunas arquitecturas, como X86, `ret` espera la dirección de retorno en el tope del stack y lo desapila. En otras, como RISC-V `ret` copia en el pc el valor de ra. En el caso de haberse apilado, el compilador habrá generado instrucciones de desapilado en ra previo a `ret`.
11. *Invocante*: desapila los argumentos, si hubiera, y toma el valor retornado (en la pila o desde registros de la CPU). Se desapilan sumando la cantidad de bytes que ocupan al sp.

Esta secuencia de operaciones deja un activation record con el siguiente formato:

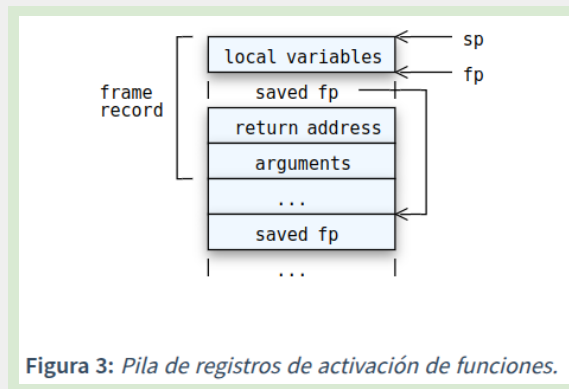


Figura 3: Pila de registros de activación de funciones.

Lo del frame record es de la rutina invocada y lo que está abajo es la del invocante. Arriba de las variables pueden haber valores temporales, eso faltó en la imagen.

Los argumentos se apilan de derecha a izquierda porque así al primer argumento que accedemos se accede con un desplazamiento fijo. Si tienes un `printf("%d ... %s", v, str)`, en la pila se apila así:

```
"%d ... %s"
v
str
```

Cuando apilas primero se apila abajo de todo y se va apilando arriba de eso para arriba. Crece desde abajo para arriba.

Si tienes una función que recibe como parámetro una estructura, este se guarda como un puntero a esa estructura en los registros de activación (creo).

El *calling convention* en una ABI de un SO o plataforma de hardware especifica los detalles de estos pasos. En particular, cómo se pasan los argumentos (en registros, pila o ambos) y el valor retornado (en registros o en la pila). Por ejemplo, Linux para x86-64 usa la especificación System V ABI.

RISC-V

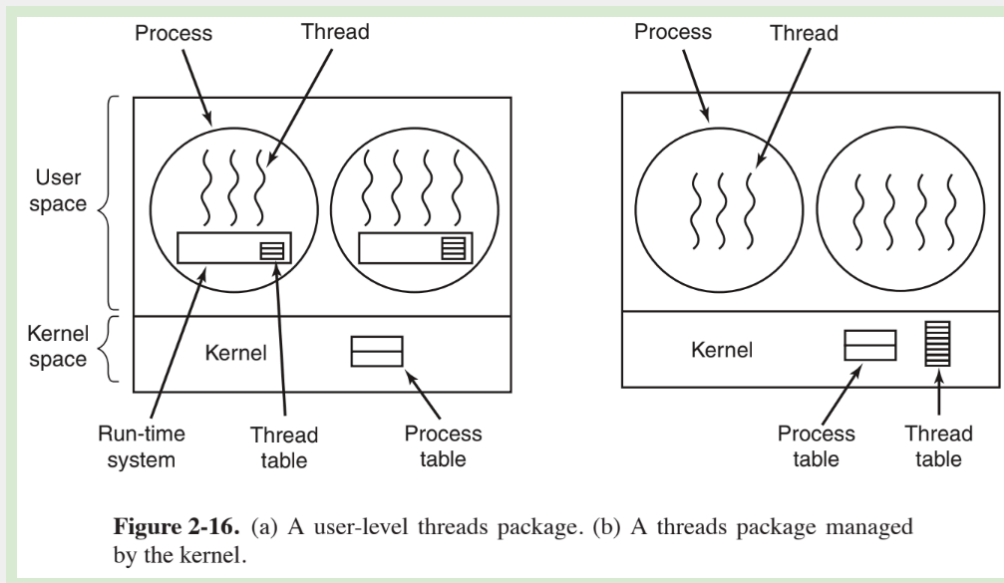
Una cpu *risc-v* tiene 32 registros de propósitos generales más el *program counter* (`pc`). Algunos de sus registros se usan según la siguiente convención:

- `x1` o `ra` : Return address (set by `call` or `jmp` instructions)
- `x2` o `sp` : Stack pointer.
- `x8` o `fp` : Frame pointer.
- `x10-x17` (`a0-a7`): Argumentos. En caso de funciones con más de 8 argumentos se deben apilar en el stack.
- El valor de retorno debe quedar en `a0` (y `a1`, opcionalmente). Los
- Los registros `s0-s11` deben ser salvados (en la pila) por la función invocada.

En el kernel del SO existe un thread por CPU que luego de la inicialización del sistema estarán ejecutando el cuerpo de `scheduler()` (en un ciclo infinito).

Un kernel de un SO podría ser *multithreaded*.

Un hilo puede estar ejecutado en dos modos:



1. **User mode:** la CPU está ejecutando instrucciones del proceso. The first method is to put the threads package entirely in user space. The kernel knows nothing about them. As far as the kernel is concerned, it is managing ordinary, single-threaded processes.

When threads are managed in user space, each process needs its own private thread table to keep track of the threads in that process. This table is analogous to the kernel's process table, except that it keeps track only of the per-thread properties, such as each thread's program counter, stack pointer, registers, state, and so forth.

2. **Kernel mode:** The kernel knows about and manages the threads. No run-time system is needed in each. Also, there is no thread table in each process. Instead, the kernel has a thread table that keeps track of all the threads in the system. When a thread wants to create a new thread or destroy an existing thread, it makes a kernel call, which then does the creation or destruction by updating the kernel thread table. The kernel's thread table holds each thread's registers, state, and other information. The information is the same as with user-level threads, but now kept in the kernel instead of in user space (inside the run-time system). This information is a subset of the information that traditional kernels maintain about their single threaded processes, that is, the process state. In addition, the kernel also maintains the traditional process table to keep track of processes.

Se ha alterado el flujo de ejecución normal del proceso debido a la ocurrencia de un trap:

El kernel, en su inicialización, definió un vector de interrupciones, el cual define un trap handler para cada tipo de trap. Para más detalles sobre manejo de interrupciones y modos de ejecución ver el capítulo de taller de xv6.

Al ocurrir un trap, el hardware y el trap handler correspondiente realizan:

1. La CPU pasa a kernel mode.
2. Se salta al trap handler correspondiente.
12. Si el trap se produjo en user mode, cambia al mapa de memoria del kernel y cambia al stack en modo kernel del proceso corriente.
3. Salva el estado de la CPU (valores de los registros) en el trapframe correspondiente.
4. Procesa el trap: llama a diferentes funciones del kernel.
5. En el retorno, recupera el estado de la CPU salvado en el trapframe.
6. Retorna del trap ejecutando una instrucción de retorno de interrupción o retorno del modo supervisor (ej: `iret` en x86, `sret` en risc-v).

El campo `kstack` de `struct proc` (definida en `proc.h`) apunta a la base del stack en modo kernel (de 4KB de tamaño).

En el caso que el trap ocurra en user mode, el uso de una pila diferente impide que un proceso de usuario pueda filtrar datos del kernel. Un proceso ejecutando en modo usuario no tiene acceso al kernel mode stack.

Creación de threads y Cambios de contexto

Para crear un nuevo thread se debe crear el espacio de memoria para su pila. En la pila se crea el contexto inicial que consiste en los valores de los registros de la CPU que deberán usarse al inicio del thread. En particular, el valor del program counter (pc), que deberá apuntar a la primera instrucción a ejecutar, en general el inicio de una función.

La función de creación de un thread puede ser de la forma:

```
uint* create_thread(void(*pc)(void), uint* stack_ptr)
{
    // push callee saved register values on stack
    push(&stack_ptr, 0); // s0 in RISC-V
    // ...
    push(&stack_ptr, 0); // s11 in RISC-V
    // push the first instruction address to execute
    push(&stack_ptr, pc);
    // return the updated stack pointer
    return stack_ptr;
}
```

La función de cambio de contexto comúnmente tiene la forma `switch_context(¤t_ctx, &new_ctx)` y deberá realizar los siguientes pasos:

1. Salvar (apilar) los valores de los registros de la CPU, en particular la dirección de retorno (ra en RISC-V), en la pila corriente (apuntada por sp) y actualizar `current_ctx` al nuevo valor de sp.
2. Cambiar al stack del nuevo thread: `sp <-- *new_ctx`.
3. Restaurar (pop) los valores salvados de los registros de la CPU (incluyendo el pc) desde el tope de la pila apuntado por `new_ctx` y actualizar `new_context`.

Abajo se muestra una implementación en assembly para RV32 (RISC-V 32 bits).

```

# context_switch(uint* old_sp, uint* new_sp)
# a0=old_sp, a1=new_sp
context_switch:
    # push callee saved registers and task pc (ra)
    addi sp, sp, -13 * 4
    sw ra, 0 * 4(sp) # push return address
    sw s0, 1 * 4(sp) # push callee saved registers
    sw s1, 2 * 4(sp)
    sw s2, 3 * 4(sp)
    sw s3, 4 * 4(sp)
    sw s4, 5 * 4(sp)
    sw s5, 6 * 4(sp)
    sw s6, 7 * 4(sp)
    sw s7, 8 * 4(sp)
    sw s8, 9 * 4(sp)
    sw s9, 10 * 4(sp)
    sw s10, 11 * 4(sp)
    sw s11, 12 * 4(sp)

    # update old_sp and set sp to new_sp
    sw sp, (a0)      # *old_sp <- sp;
    lw sp, (a1)      # sp <- *new_sp

```

```

# restore register values from new stack
lw ra, 0 * 4(sp) # restore return address
lw s0, 1 * 4(sp) # restore callee saved registers
lw s1, 2 * 4(sp)
lw s2, 3 * 4(sp)
lw s3, 4 * 4(sp)
lw s4, 5 * 4(sp)
lw s5, 6 * 4(sp)
lw s6, 7 * 4(sp)
lw s7, 8 * 4(sp)
lw s8, 9 * 4(sp)
lw s9, 10 * 4(sp)
lw s10, 11 * 4(sp)
lw s11, 12 * 4(sp)
addi sp, sp, 13 * 4 # simulate pop
ret

```

Se debe notar que el primer parámetro es de salida (se actualiza al nuevo tope del stack del `current_thread`). El parámetro `new_ctx` es por referencia, ya que debe actualizarse a medida que se van desapilando los valores del stack del nuevo thread en los registros de la CPU.

Lo que pasa acá básicamente es lo siguiente (según yo):

Cada thread tiene su propia stack, su propia memoria en donde se almacena su estado, sus registros, etc. Cuando este thread llamemoslo A se va a ejecutar, se hace un context switch, es decir, todos sus registros, todo su estado se carga a los registros reales de la CPU para que lo pueda ejecutar. Una vez que se terminó de ejecutar ese thread o pinto el context switch porque se va a ejecutar con thread B ahora, el estado que tenemos en la CPU se tiene que guardar en la stack propia del thread A para no perderlo, cosa de que cuando

se vuelva a ejecutar ese thread, se siga ejecutando desde donde dejó. Luego, lo mismo que hicimos al principio para el thread A lo hacemos para el thread B, cargamos todo su estado en la CPU para poder ejecutarlo.

Y esa función de `context_switch` hace esto: el `sp` es el de la CPU que siempre apunta al tope de la stack del thread que se está ejecutando. Entonces al estar apuntando al principio al `sp` de la stack del thread A, lo que va a hacer es reservar espacio en la stackA para poder guardarle los registros que están en la CPU, es decir su estado actual (se reserva espacio porque la stack crece). Después, el `sp` del thread A se actualiza a ser el `sp` de la CPU, o sea que su stack y su `sp` ya están actualizados con el nuevo estado y la próxima vez que se vuelva a ejecutar este thread, se seguirá su ejecución desde ese estado. Luego, el `sp` de la CPU se actualiza con el del thread B para poder cargar su estado (sus registros) a los de la CPU para así poder ejecutarse.

La CPU tiene los registros físicos y el `sp` que va a estar apuntando a la stack de cada thread que se ejecute en ese momento, la cpu no tiene una stack, solo apunta las stacks de los threads que se van a ejecutar.

Gráficamente esta función de `context_switch` se puede ver así:

- Thread A está ejecutando, por lo que `sp` apunta a la cima de la pila de Thread A.
- La pila de Thread B ya tiene los registros guardados de una ejecución previa (cuando se cambió de Thread B a Thread A).
- Las variables `*current_sp` y `*next_sp` están en memoria, pero aún no se han actualizado.

text

... Copy

Pila de Thread A		Pila de Thread B
+-----+		+-----+
...	<--- sp	ra (guardado) <--- *next_sp
(datos previos)		s0 (guardado)
		s1 (guardado)
		...
		s11 (guardado)
+-----+		+-----+

- `sp` apunta a la pila de Thread A (el hilo actual).
- `*next_sp` apunta a la cima de la pila de Thread B, donde están los registros guardados de Thread B.

```
addi sp, sp, -13 * 4
```

... Copy

- **Qué hace:** Decrementa `sp` en $13 * 4 = 52$ bytes para reservar espacio en la pila de Thread A.
- **Por qué:** Se necesitan 52 bytes (13 registros $\times$ 4 bytes) para guardar los valores actuales de los registros `ra`, `s0` a `s11` de Thread A.

Estado de las pilas:

text

... Copy

Pila de Thread A	Pila de Thread B
+-----+	+-----+
...	ra (guardado) <--- *next_sp
(datos previos)	s0 (guardado)
	s1 (guardado)
<espacio libre> <--- sp	...
<espacio libre>	s11 (guardado)
<espacio libre>	+-----+
... (52 bytes)	
+-----+	

- La pila de Thread A ahora tiene 52 bytes reservados (13 celdas de 4 bytes).
- `sp` se mueve hacia abajo en la pila de Thread A.
- La pila de Thread B no se toca en este paso.

Lo que está en la stack del thread B es su saved context que se llama. Los demás registros no se le guardaron al thread B porque la convención es que el invocante los guarda, no se tiene que hacer cargo el thread B.

Si haces un `addi sp, sp, -13 * 4`, estas agregándole espacio a la stack, y esto va a crecer para arriba, y el `ra` se guarda en el primer lugar porque haces un `0 * 4(sp)`, y después vas moviendo para abajo. Cuando agregas más espacio el `sp` queda arriba de todo.

```
sw ra, 0 * 4(sp)
sw s0, 1 * 4(sp)
...
sw s11, 12 * 4(sp)
```

... Copy

- **Qué hace:** Guarda los valores actuales de los registros `ra`, `s0` a `s11` de Thread A en el espacio reservado en su pila.
- **Por qué:** Estos registros contienen el estado de Thread A (como el contador de programa en `ra` y variables en `s0` a `s11`), que debe preservarse para cuando Thread A se reanude.

Estado de las pilas:

text

✖ Collapse

⇒ Wrap

Copy

Pila de Thread A	Pila de Thread B
+-----+	+-----+
...	ra (guardado) <--- *next_sp
(datos previos)	s0 (guardado)
ra <--- sp	s1 (guardado)
s0	...
s1	s11 (guardado)
...	+-----+
s11	
+-----+	

```
sw sp, (a0)    # *current_sp = sp
lw sp, (a1)    # sp = *next_sp
```

Pila de Thread A	Pila de Thread B
+-----+	+-----+
...	ra (guardado) <--- sp
(datos previos)	s0 (guardado)
ra <--- *current_sp	s1 (guardado)
s0	...
s1	s11 (guardado)
...	+-----+
s11	
+-----+	

- `*current_sp` ahora apunta a la cima de la pila de Thread A (donde están sus registros guardados).
- `sp` ahora apunta a la cima de la pila de Thread B (donde están los registros guardados de Thread B).

```
addi sp, sp, 13 * 4
```

- **Qué hace:** Incrementa `sp` en `13 * 4 = 52` bytes, moviendo el puntero de pila de Thread B hacia arriba.
- **Por qué:** Los valores de los registros de Thread B ya fueron restaurados, por lo que el espacio que ocupaban en la pila ya no es necesario. Esto libera los 52 bytes, dejando la pila de Thread B lista para que el hilo continúe su ejecución.

Estado de las pilas:

text

... Copy

Pila de Thread A	Pila de Thread B
+-----+	+-----+
...	... <--- sp
(datos previos)	(datos previos)
ra <--- *current_sp	
s0	
s1	
...	
s11	+-----+
+-----+	

Se hace este “pop” para “limpiar” la pila del thread B, porque el estado viejo (estado en donde se quedó el thread luego de ser interrumpido) no sirve más porque ahora está en los registros de la CPU. Es por eso que para el thread A se tuvieron que pushear todos los registros del nuevo estado.

El `ret` después salta a la dirección almacenada en `ra`, que es el program counter de Thread B, provoca esto. Esto reanuda la ejecución de Thread B desde el punto donde se interrumpió.

A continuación se muestra un fragmento de código sobre el uso de estas funciones.

```
// a caller thread creating a new thread starting in function f()
uint* main_thread_sp;
unsigned char new_thread_stack[STACK_SIZE];
uint* new_thread_sp = (uint*) (new_thread_stack + STACK_SIZE);

void f(void)
{
    // ...
    // resume execution of main thread
    context_switch(&new_thread_sp, &main_thread_sp);
    // ...
}

int main(void)
{
    new_thread_sp = create_thread(f, &new_thread_sp);
    // start new thread: run f() body
    context_switch(&main_thread_sp, &new_thread_sp);
    // ...
    // main thread running again
    // ...
    // resume f thread again
    context_switch(&main_thread_sp, &new_thread_sp);
}
```

Estos cambios de contexto son explícitos. Cada thread especifica a qué otro thread desea darle el control de la CPU. En éste caso se denominan **corrutinas**.

En un SO, un thread al abandonar la CPU usará una función `yield()` que transfiere el control al thread principal que está `scheduler()`. Esta última función decide a qué thread le dará el control de la CPU.

Las funciones `yield()` y `scheduler()` tienen la siguiente forma:

```
struct task* current;
...
void yield()
{
    current->state = RUNNABLE;
    switch_context(&current, &scheduler_ctx);  -----+
}
|
|
void scheduler() {
    |
    ...
    while (1) {
        |
        current = NULL;
        |
        ...
        struct task* = select_runnable_task();
        |
        if (task) {
            |
            context_switch(&scheduler_ctx, &(task->sp));
            |
        }
        <-----+
    }
    ...
}
```

En el caso que los threads invoquen explícitamente a `yield()` se dice que el sistema soporta **multitarea cooperativa** ya que cada thread colabora cediendo la CPU periódicamente.

Un thread que no invoque a `yield()` monopolizará la CPU. Para prevenir esto, principalmente para la implementación de procesos de usuario, el kernel usa un timer que genera interrupciones periódicamente. En cada interrupción, el kernel le puede quitar la CPU al thread corriente (invocando a `yield()`). Esto es **preemptive multitasking**.

mos-steps

En la carpeta **05-tasks** se muestra la implementación de estas funciones en el kernel y el uso de *kernel threads* (o *tasks*) cooperativos. En el paso **06-preemptive** se implementa *preemptive multitasking*.

Procesos de Usuario

Un proceso es una instancia de un programa en ejecución, incluyendo los valores actuales del program counter, los registros y variables.

Un **proceso** representa un programa de usuario en ejecución. Generalmente se representa con al menos los siguientes atributos:

- Un identificador (process id o pid).
- Su estado: ver la figura 2.
- Razón o recurso por el cual está en estado SLEEPING.
- Su mapa (áreas) de memoria a las cuales puede acceder en modo usuario.
 - Código (text segment).
 - Datos globales (estático): área de datos inicializados (data section) y no inicializados (bss section).

Al menos un thread de ejecución, representado por

- Stack en modo usuario. Stack que usa el thread mientras ejecuta en modo usuario.
- Stack en modo kernel. En este stack se almacena el trapframe: valores de los registros de la CPU al ocurrir una interrupción en éste thread. Esta pila también se usa para el control de invocación a funciones dentro del kernel ejecutándose en el contexto del proceso. Finalmente, también se usa para guardar el contexto (caller saved CPU registers) del thread cuando se produzca un context switch.
- Conjuntos de recursos adquiridos: archivos/pipes/sockets abiertos, semáforos, timers, áreas de memoria compartida, manejadores de señales y otros.
- El comando o programa del cual es una instancia.

Para implementar un hilo de ejecución (thread) lo primero que tiene que hacer un kernel es definir y configurar una stack.

Vamos a necesitar dos stacks en el SO: stack en modo usuario que va a usar el programa que se ejecuta, el thread mientras se va a estar ejecutando en modo usuario y no va a poder hacer instrucciones privilegiadas al estar en modo usuario.

Cuando ocurra una interrupción, el kernel va a tomar el control y la ejecución del thread en modo usuario se interrumpe. El kernel entonces cambia de stack por una stack en modo kernel. En esta stack se salva el estado del thread, o sea guardar todos los valores de los registros de la CPU, para que se puedan restaurar una vez que se vuelva a ejecutar el thread como si no hubiera pasado nada.

Un proceso es básicamente un programa en ejecución. Asociado a cada proceso se encuentra su espacio de direcciones, una lista de ubicaciones de memoria desde 0 hasta un máximo determinado, que el proceso puede leer y escribir. El espacio de direcciones contiene el programa ejecutable, los datos del programa y su pila. También se asocia a cada proceso un conjunto de recursos, que generalmente incluyen registros (incluidos el contador de programa y el puntero de pila), una lista de archivos abiertos, alarmas pendientes, listas de procesos relacionados y toda la demás información necesaria para ejecutar el programa. Un proceso es fundamentalmente un contenedor que contiene toda la información necesaria para ejecutar un programa.

Linux

GNU-Linux representa a un proceso por medio de una `struct task_struct`, el cual contiene los atributos básicos del thread y punteros a estructuras de datos que representan los demás recursos (archivos, memoria, etc). En un proceso *multithread* cada `struct task_struct` apuntan a los mismos recursos, ya que pertenecen al mismo proceso y tienen el mismo *thread group*=PID aunque diferente *thread id*.

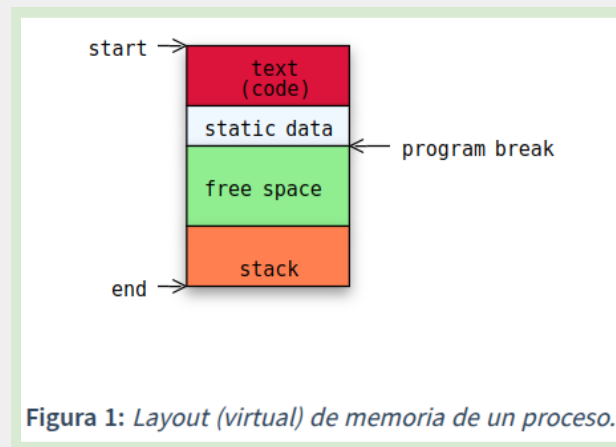
Cuando un proceso se carga en memoria, mediante el `syscall exec(path, args)`, sus secciones de código (text) y datos (data) se cargan desde el archivo ejecutable (en formato ELF, por ejemplo), se le asigna espacio para el stack del thread principal y se inicializa un stackframe conteniendo los valores de los registros de la CPU requeridos al inicio de la ejecución. En particular los valores que deberán tomar el program counter (pc) y el stack pointer (sp).

El valor inicial del sp deberá apuntar a la primera instrucción de la función de inicio del proceso, comúnmente la función `_start` de la biblioteca estándar (enlazada con cada programa de usuario). Esta función comúnmente se define como:

```
void _start(void)
{
    exit(main());
}
```

El stack se inicializa de tal forma que `main(int argc, char* argv[])` reciba los parámetros que correspondan.

La siguiente figura muestra el esquema (layout) en memoria de un proceso. A un proceso en ejecución que está en memoria lo vamos a ver así:



En linux ese start está en la dirección 0, pero esa dirección es virtual.

El espacio de direcciones de un proceso es $(start, end)$. En muchos sistemas, $start = 0$.

En los SO tipo UNIX, la llamada al sistema `sbrk(n)` permite extender/reducir el segmento de datos del proceso asignando o liberando memoria y actualiza el program break.

El subsistema de gestión de procesos se encarga de la creación, cambios de estado, cambios de contexto, sincronización y finalización de procesos. Más abajo se describe en detalle el uso de los atributos mencionados. También se encarga de mantener el registro de los recursos adquiridos por cada proceso.

Llamadas al Sistema de gestión de procesos

Algunas de las llamadas al sistema para la gestión de procesos se describen a continuación:

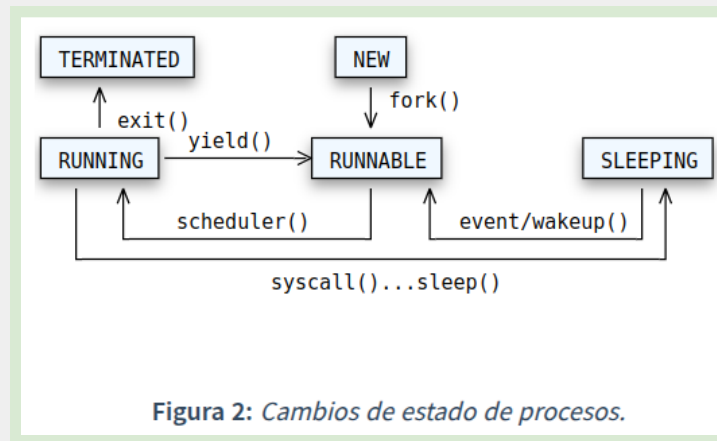
- `fork()`: crea un nuevo proceso (hijo) el cual es una copia del proceso invocante. El proceso hijo hereda los descriptores de los archivos abiertos del padre.
- `exit(exitstatus)`: finaliza el proceso.
- `wait(&childstatus)`: espera (se bloquea) hasta que finalice un proceso hijo.
- `getpid()`: retorna el identificador del proceso corriente.
- `kill(pid)`: envía una señal (terminación) al proceso con id `pid`.
- `sleep(n)`: espera (se bloquea) por `n` segundos.
- `exec(filename, args)`: reemplaza la imagen de código y datos del proceso corriente por el código y datos de `filename`. Al final de esta sección se describe en mayor detalle.
- `sbrk(n)`: asigna `n` bytes de memoria extra al proceso invocante.

Estados de un proceso

Un proceso puede estar en uno de los siguientes estados:

- **RUNNING**: está ejecutando actualmente, tiene asignada una CPU.
- **RUNNABLE** o **READY**: está esperando a que el kernel le asigne una CPU. Temporalmente parado para dejar que otro proceso use la CPU.
- **SLEEPING** o **WAITING**: está bloqueado hasta la ocurrencia de un evento que lo despertará. El proceso no puede ejecutarse, inclusive si la CPU está ociosa y no tiene otra cosa que hacer.

La figura 3 muestra los cambios de estado que puede tener un proceso y las funciones de un kernel involucradas en las transiciones.



Pueden existir otros estados. Por ejemplo en los sistemas tipo UNIX, un proceso que ha terminado su ejecución (vía `exit()`) pero aún existe en el sistema (está todavía en la tabla de procesos, lo tenemos que dejar ahí todavía) se denomina **zombie**. Esto ocurre comúnmente cuando el proceso padre aún no ha ejecutado `wait(&status)` y se requiere mantener su estado de terminación en el descriptor del proceso hasta que el proceso padre ejecute el `wait()` correspondiente.

En los sistemas operativos tipo UNIX, cada proceso, excepto `init` (el primer proceso lanzado por el kernel luego del booting) tiene un padre. El kernel debe mantener esta invariante.

¿Qué sucede si un proceso padre termina antes que sus hijos? En UNIX, los procesos hijos son adoptados por `init`. A estos procesos se los denomina huérfanos. El `init` está en un ciclo infinito haciendo `wait()`. Hace este `wait()` infinitamente porque eso le permite adoptar a los huérfanos.

Representación de Procesos

Según el libro, para implementar el modelo de proceso, el sistema operativo mantiene una tabla (un conjunto de estructuras), denominada tabla de procesos, con una entrada por proceso. (Algunos autores denominan a estas entradas bloques de control de proceso PCB). Esta entrada contiene información importante sobre el estado del proceso, incluyendo su program counter, stack pointer, asignación de memoria, el estado de sus archivos abiertos, información de scheduling, y todo lo demás sobre el proceso que debe guardarse cuando este pasa del estado en ejecución al estado **RUNNABLE/READY** o **SLEEPING** para que pueda reiniciarse posteriormente como si nunca se hubiera detenido.

Una de las estructuras de datos principales es el conjunto de process control blocks (PCB) o descriptores de procesos. Este conjunto puede estar implementado como un arreglo (tabla), diccionario o listas.

Un descriptor de proceso es una estructura o registro que contiene campos para representar los atributos ya mencionados.

Linux

Los descriptors de procesos están representados en la estructura de datos

Las operaciones que realizan los cambios de estado de un proceso se muestran en la figura 3. No confundir estas funciones con aquellas que implementan las llamadas al sistema con el mismo nombre, como `sleep` o `fork`. Las funciones que implementan los syscalls comúnmente invocan a estas funciones.

En `xv6`, los syscalls están implementados en las funciones `sys_fork()`, `sys_exec()`, ... en `sysproc.c` y `sysfile.c`.

En los sistemas tipo UNIX, cada proceso (excepto `init`) tiene un padre: el proceso que invocó el syscall `fork` correspondiente. Esto induce a una estructura de árbol como representación de todos los procesos del sistema.

Sobre la tabla de procesos comúnmente también se representan colas o listas para representar subconjuntos de procesos. Por ejemplo, un scheduler que implementa una política FIFO, implementará el conjunto de procesos `RUNNABLE` como una cola, lista o red-black tree.

Un ejemplo interesante es la implementación de listas doblemente encadenadas intrusivas en GNU-Linux.

mos-steps

`scheduler()` recorre toda la tabla de procesos hasta encontrar un proceso `RUNNABLE`.

Cuando un proceso está en estado `SLEEPING`, el sistema deberá conocer su motivo. Un diseño e implementación común es asociar el conjunto de procesos que esperan al evento que los despertará en una `wait queue`.

Otro aspecto a tener en cuenta es qué sucede si el scheduler no encuentra un proceso `RUNNABLE` (situación muy común). Básicamente se pueden implementar alguna de las siguientes alternativas:

1. Hacer que `scheduler()` quede en un ciclo hasta encontrar uno.
2. Usar un proceso `idle`, creado inicialmente por el sistema y que ejecute `while (true) yield();`. Este proceso al obtener la CPU la libera inmediatamente. Esto hace que siempre haya un proceso `RUNNABLE`.

En cualquier caso ésto es una oportunidad para contabilizar la carga de trabajo (o idleness) del sistema para realizar acciones como por ejemplo, ahorro de energía o disparar operaciones diferidas (como procesar paquetes de red recibidos) en un kernel thread.

Recordemos que el kernel tomará el control cada vez que ocurra un `trap`.

Un `trap` es una abstracción de los siguientes eventos:

- El proceso ejecutó un syscall (instrucción `ecall` en `risc-v`), o
- La CPU disparó una excepción porque el proceso intenta ejecutar una instrucción inválida (privilegiada, división por cero, etc) o porque intenta acceder a un área de memoria inválida, o
- Un dispositivo generó una interrupción. Un kernel con preemptive multi-tasking usa un `timer` o `clock` que genera interrupciones periódicamente.

Otra estructura de datos importante es la representación de la **cpu**, la cual comúnmente contiene los siguientes atributos:

- Identificador de la CPU (por si hay más de una).
- Una referencia al proceso o thread corriente en ejecución por esa CPU.
- Un contexto salvado: estado de la CPU del último cambio de contexto.
- Estado de las interrupciones (habilitadas/deshabilitadas) y otros.

En un sistema que soporte múltiples CPUs (ej: multicore) deberá tener una estructura por CPU.

Para ver en detalle las estructuras de datos para la gestión de procesos en xv6, ver `proc.h`.

En el paso 05-tasks de mos-steps se definen algunas de estas estructuras.

Llamadas al Sistema

Comúnmente, a cada llamada al sistema se le asigna un identificador único. Es común asignarles números naturales (en xv6, ver `kernel/syscall.h`).

Las llamadas al sistema se implementan en dos partes:

- Invocación desde un proceso (user mode): generalmente se implementan como funciones en la biblioteca estándar que se enlace con cada programa de usuario. Cada función se implementa comúnmente poniendo el código de syscall en un registro designado de la cpu (a7 en risc-v) y luego ejecutar una instrucción de trap o software interrupt (int en x86 o ecall en risc-v). En xv6 están implementadas en `user/usys.S`.
- Funciones de servicios en el kernel: al ejecutarse una instrucción de trap correspondiente a un syscall, se dispara el mecanismo de manejo de interrupciones y se ejecuta el trap handler correspondiente en el kernel. En este caso, se invocará a un dispatcher que analiza el código del syscall (que está en un registro de la cpu) e invoca a la función de servicio correspondiente.
En xv6, éstas funciones están definidas en `sysproc.c` y `sysfile.c`. Por cada syscall hay una función `sys_<syscall>()` que implementa el servicio. El dispatcher es la función `syscall()` (en `syscall.c`), que invoca a la rutina apuntada por `syscalls[n]`, donde n es el código del syscall.

Generalmente las syscalls retornan algún valor o código de error a los procesos. En el caso de una API POSIX, los valores negativos son códigos de error.

Los syscalls exit y wait

Estas dos llamadas al sistema POSIX están relacionadas.

- `exit(exitcode)`: finaliza el proceso corriente con código de salida `exitcode`.
- `wait(&exitcode)`: el proceso corriente espera (se bloquea) hasta que finalice un hijo. El código de salida del hijo se copia en el parámetro de salida `exitcode`.

Esto ofrece un mecanismo de sincronización y comunicación básico entre procesos. El proceso padre puede saber cómo terminó el hijo.

Por convención se asume que el código o status de salida 0 es terminación normal (o exitosa). Un código de salida diferente a cero representa una finalización anormal y el código indica el motivo. Cada aplicación define el significado de los códigos de salida anormales.

Esto motiva el estado ZOMBIE de un proceso que ejecutó `exit()` antes que el padre ejecute `wait()`, ya que al menos se debe mantener el código de salida hasta que el padre ejecute `wait()`.

En el caso que el proceso padre finalice sin hacer un `wait()`, los sistemas tipo UNIX comúnmente hacen un `reparent` del proceso hijo, comúnmente haciendo que lo adopte `init`.

En un shell el código de salida del último proceso queda en la variable de ambiente `$?`.

Cambios de Contexto de procesos

Todo esto que está entre los símbolos `{ }` lo saque de la teoría pero después el profe lo cambio y lo saco, lo que le sigue abajo es lo actualizado.

`{` Es posible ver a los threads de cada proceso y el scheduler como corrutinas.

Una **corrutina** es un componente en un programa que puede ser suspendido por medio de las operaciones `yield` o `sleep` y luego continuar (`resume`) desde su punto anterior preservando su estado local. Comúnmente se usan para implementar multitarea cooperativa. Las corrutinas son programas en ejecución que le dan el control explícitamente a otro proceso.

En xv6, la función `sched()` (mediante `swtch`) implementa una operación del tipo `yield()` o `resume(scheduler)` mientras que el `swtch` en `scheduler()` implementa un `resume(p)` donde `p` es el proceso (corrutina) seleccionado como se muestra en el siguiente diagrama.

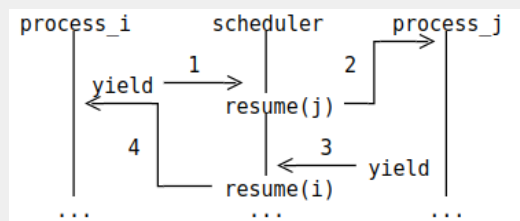


Figura 4: Visión de threads como corrutinas.

La figura anterior muestra que el proceso (corrutina) *i* ejecutándose abandona la cpu mediante `yield` lo que hace que el control continúe en `scheduler`, el cual decide hacer continuar al `process j`. Este último luego abandona la cpu y `scheduler` hace que el proceso *i* continúe o reasuma desde el punto a continuación de `yield`.

Como ya se describió arriba, el kernel toma el control al ocurrir un trap.

Xv6

Los *trap handlers* (o puntos de entrada) de xv6 son:

- `uservec()` en `kernel/trampoline.S` para traps que ocurren en *user mode*.
- `kernelvec()` en `kernel/kernelvec.S` : para traps que ocurren en *kernel mode*.
- `timervec()` en `kernel/kernelvec.S` : para interrupciones del *timer*. En *risc-v* ejecuta en *machine mode*.

Estos *handlers* de bajo nivel terminan invocando a `usertrap()` o `kerneltrap()` , definidas en `trap.c` . Estas funciones determinan la acción a realizar.

Un handler se ejecuta en modo kernel y dependiendo en qué contexto ocurrió el evento puede pertenecer al hilo de ejecución del proceso interrumpido (process context) o en kernel context si estaba ejecutando código del kernel (ejemplo, ciclando en el scheduler).

Luego, en el kernel generalmente ocurren dos tipos de cambios de contexto (threads switch):

1. Del contexto del proceso al contexto del kernel (al scheduler).
2. Del contexto del kernel al contexto de un proceso (desde el scheduler).

El primer caso se da cuando un proceso abandona el estado RUNNING. El segundo se da cuando el scheduler selecciona un proceso RUNNABLE y lo pasa a RUNNING, asignándole la cpu.

El kernel decide que el proceso corriente debe abandonar la CPU en base a:

1. El proceso completó su quantum o time slice de uso de CPU. Esto se produce cuando el kernel recibe un timer interrupt. En este caso el kernel invoca a una función del tipo `yield()`. Esto es típico de un SO con preemptive multitasking.
2. El proceso realizó una llamada al sistema, como `read()`, `write()`, `sleep()` u otra, cuyo procesamiento puede llevar demasiado tiempo por lo que conviene suspender el proceso (pasar al estado SLEEPING). El kernel invoca a una función del tipo `sleep(wait_queue)` o similar e invoca al scheduler para que éste asigne la CPU a otro proceso.

Un proceso sleeping se despertará (pasará a RUNNABLE) cuando ocurra un evento como por ejemplo, la finalización de la operación de entrada-salida por la que está esperando.

Cuando un proceso deja el estado RUNNING, el kernel realiza los siguientes pasos:

1. Salva el estado del thread corriente (contexto). Al menos debe guardar el program counter y el stack pointer.
2. Recupera el contexto del scheduler (salvado previamente). En xv6 a esto lo realiza `swtch(&p->context,&cpu->context)` en `sched()`. Esto hace que la cpu salte al punto de continuación de `scheduler()`. En xv6, este punto es debajo de la invocación a `swtch(&cpu->context,&p->context)`. Ver el listado 1. En este punto el kernel está ejecutando en el contexto del scheduler, no de un proceso. El scheduler selecciona un proceso RUNNABLE `p`.
3. Se salva el contexto actual (del scheduler) y se cambia al contexto de `p` (en xv6, `swtch(&cpu->context,&p->context)`).

Esto hace que la CPU continúe su ejecución en el punto en el que abandonó la CPU en el pasado (en xv6, salta al punto 2 del listado 1). La implementación de una función como `swtch(old,new)` es dependiente de la arquitectura y comúnmente se implementa en assembly.

Xv6

La función `swtch(old,new)`, está definida en `swtch.S`. El control (*resume*) transiciona entre (las *corrutinas*) `sched()` y `scheduler()` como se muestra en la siguiente figura.

```

scheduler()
{
    ...
    for(;;) {
        ...
        // to process context: resume(sched)
        swtch(&cpu->context, &p->context);
        ... (1) <-----+
    }
}

sched() {
    ...
    // to scheduler context: resume(scheduler)
    swtch(&p->context, &mycpu()->context);
    ... (2) <-----+
}

```

Listado 1: Cambios de contexto en xv6.

}.

Un hilo de un proceso de usuario puede estar ejecutando en dos modos:

1. **User mode:** la CPU está ejecutando instrucciones del proceso.
2. **Kernel mode:** se ha alterado el flujo de ejecución normal del proceso debido a la ocurrencia de un trap:

El kernel, en su inicialización, definió un vector de interrupciones, el cual define un trap handler para atrapar o manejar los traps (interrupciones, excepciones generadas por la CPU e interrupciones por software).

Al ocurrir un trap, el hardware y el trap handler correspondiente realizan:

1. La CPU pasa a kernel mode.
2. Se salta al trap handler correspondiente.
3. Si el trap se produjo en user mode, cambia al mapa de memoria del kernel y cambia al stack en modo kernel del proceso corriente.
4. Salva el estado de la CPU (valores de todos los registros) en el stack en modo kernel. Este estado se denomina trapframe.
5. Procesa el trap llamando a diferentes funciones del kernel. En el camino puede decidir darle el control a otro proceso (context switch), cambiando el current stack.
6. Antes del retorno de la interrupción, el trap handler recupera el estado de la CPU salvado en el trapframe (desde el stack corriente).
7. Retorna del trap ejecutando una instrucción de retorno de interrupción o retorno del modo supervisor (ej: iret en x86, sret en risc-v). Luego del retorno del kernel el thread del nuevo proceso seleccionado continuará su ejecución desde el punto en que había sido interrumpido en el pasado.

En el caso que el trap ocurra en user mode, el uso de una pila diferente impide que un proceso de usuario pueda filtrar datos en la pila dejados por el código del kernel, lo cual sería un problema serio de seguridad. Un proceso ejecutando en modo usuario no tiene acceso al kernel mode stack.

El kernel decide que el proceso corriente debe abandonar la CPU en base a:

1. El proceso completó su quantum o time slice de uso de CPU. Esto se produce cuando el kernel recibe un timer interrupt. En este caso el kernel invoca a una función del tipo `yield()`, la cual cambia al contexto del scheduler. Esto es típico de un SO con preemptive multitasking.
2. El proceso realizó una llamada al sistema, como `read()`, `write()`, `sleep()` u otra, cuyo procesamiento puede llevar demasiado tiempo por lo que conviene suspender el proceso (pasar al estado SLEEPING). El kernel invoca a una función del tipo `sleep(wait_queue)` e invoca a `yield()` para que éste asigne la CPU a otro proceso.

Un proceso sleeping se despertará (pasará a RUNNABLE) cuando ocurra un evento como por ejemplo, la finalización de la operación de entrada-salida por la que está esperando.

Ejecución de programas de usuario

Un proceso en modo usuario se representa como una task en el kernel. La única diferencia es que cada proceso de usuario tiene, además del stack en modo kernel, un stack para su ejecución en modo usuario.

1. Busca e inicializa un descriptor de tarea.
2. Invoca a `exec("program")` el cual asigna memoria, carga las secciones de ejecutable desde el disco, arma la tabla de páginas y configura el trapframe en el stack en modo kernel. Además en la pila en modo usuario apila los valores de los argumentos de `main`.
3. Se configura el contexto de la tarea vía `init_context(userret, task->sp)`. La función `userret()` configura la CPU para luego retornar desde un trap (vía `trapret`) a modo usuario. En RISC-V, esto requiere que antes de retornar a modo usuario debamos hacer:
 - Setear el modo previo de ejecución a user mode: `sstatus.SPP = 0`,
 - Habilitar interrupciones en modo usuario: `sstatus.SPIE = 1`,
 - Definir el retorno al inicio o punto de interrupción previo del programa: `sepc = trapframe->ra`.

Luego, `scheduler()` al encontrar a este proceso como RUNNABLE hará el context switch saltando a un camino de retorno de un trap en modo supervisor hacia modo usuario. Más abajo se describe `exec` en forma más detallada.

Cuando un proceso de usuario esté en ejecución, éste podrá ser interrumpido (por un dispositivo, excepción de la CPU o hace una llamada al sistema).

En éste punto el trap handler del kernel deberá cambiar del stack en modo usuario al stack en modo kernel, en donde se apilarán los valores de los registros de la CPU salvados (trapframe).

👤 Stack en modo usuario

Cuando un programa está corriendo normalmente (por ejemplo, tu navegador, un editor de texto, un juego), lo hace en **modo usuario**. En este modo, tiene su **propia stack** que:

- Está ubicada en el espacio de memoria del proceso (no accesible directamente por otros procesos)
- Se usa mientras el programa ejecuta su código "normal"
- Contiene las variables locales, parámetros, y direcciones de retorno de las funciones del programa

🛡️ Stack en modo kernel

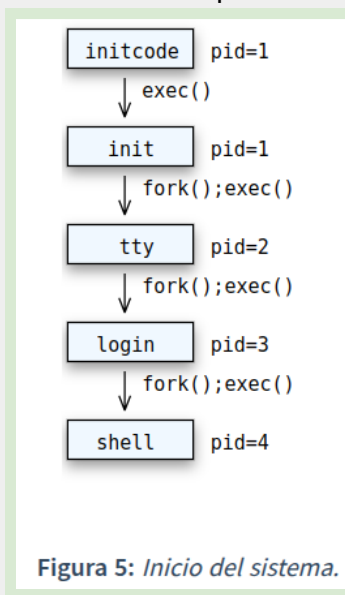
Cuando el programa necesita realizar algo que requiere privilegios del sistema operativo (como leer un archivo, escribir en la red, pedir más memoria...), hace una **llamada al sistema (syscall)**. En ese momento:

1. El procesador **cambia al modo kernel**
2. Se salta a una **rutina del sistema operativo**
3. **NO** se sigue usando la stack del programa usuario
4. En cambio, el sistema operativo usa una **stack especial en modo kernel**, que:
 - Está en una zona protegida de la memoria (accesible solo por el kernel)
 - Es privada para cada proceso (cada proceso tiene su stack del kernel)
 - Guarda los datos necesarios para atender la llamada del sistema, como registros, argumentos del syscall, etc.

El primer proceso de un sistema

En los sistemas tipo UNIX, el primer proceso lanzado es **init**.

El proceso **init** generalmente se vincula a las terminales y dispara el proceso **login** en cada una. Finalmente **login** creará un **shell** para que un usuario comience su interacción con el sistema. La siguiente figura muestra el árbol de procesos inicial en un sistema tipo UNIX.



En edos, haremos que **init** simplemente ejecute el **shell** (haciendo **fork()** y en el hijo **exec("shell", [])**).

La llamada al sistema **exec**

El syscall **exec(cmd, args)** realiza básicamente los siguientes pasos:

1. Reserva memoria para el código, datos y user-mode stack y carga el código y los datos del programa (ejecutable) `cmd` en la memoria asignada. Al mismo tiempo, construye la tabla de páginas para implementar protección y memoria virtual. Este será el nuevo mapa de memoria del proceso. Configura la pila en modo usuario (apilando los argumentos de `main`) y el `trapframe` para simular un retorno al punto de entrada del programa (dirección de memoria `entry` en el archivo ELF).
2. Configura la pila en modo usuario (apilando los valores de los argumentos de `main`).
3. Configura el `trapframe` en la pila en modo kernel. En `trapframe->ra` (valor del pc salvado en la interrupción previa) será el punto de entrada del programa (dirección de memoria `entry` en el archivo ELF). El valor de `trapframe->sp` será el del tope de la pila en modo usuario.
4. Libera la imagen de memoria anterior del proceso.
5. Configura la cpu con el nuevo mapa de memoria creado en el paso 1 (en RISC-V, `task->vm` apunta a la tabla de páginas creada).

En xv6 esta función está implementada en `exec.c`.

El proceso, luego al ser planificado, retornará de modo kernel a modo usuario ejecutando la función correspondiente al punto de entrada (comúnmente `main()` o `start()`) del programa.

 xv6

`exec()` asigna una *página* encima de la página del stack sin permisos de acceso en modo usuario para detectar *stack overflow*.

La llamada al sistema `fork()`

Esta llamada al sistema crea un nuevo proceso (hijo) que es una copia o clon del proceso invocante (padre).

Básicamente realiza los siguientes pasos:

1. Obtiene un nuevo descriptor para el nuevo proceso (hijo) y un nuevo `pid`.
2. Copia el contexto del padre en el descriptor del hijo. Ambos hilos de ejecución deberán continuar en el mismo estado.
3. Asigna memoria y copia los bloques de memoria de código, datos estáticos y stack en modo usuario. En el proceso crea el mapa de memoria correspondiente (tabla de páginas).
4. En el stack en modo kernel del hijo setea el valor de retorno del `syscall` en cero (en RISC-V, `trapframe->a0 = 0`). En el `trapframe` del padre pone como valor de retorno el `pid` del hijo (en RISC-V, `trapframe->a0=tasks[child].pid`).
5. Setea el estado del proceso hijo en `RUNNABLE`.

En cada proceso, al retornar del modo supervisor, éstos continuarán en el punto de retorno de `fork()`. En el padre retornará el `pid` del hijo y en el hijo 0.

Planificación de uso de CPU

El sistema operativo debe seleccionar o planificar a qué proceso `RUNNABLE` se le asignará la cpu en diferentes momentos. Dependiendo de qué tipo de sistema y aplicaciones se ejecutan, ésta selección deberá hacerse en base a diferentes criterios.

En los sistemas por lotes (batch systems), los programas se envían y se almacenan en un `jobs spool` para su ejecución y no necesariamente se ejecutan inmediatamente.

Un **long term scheduler** selecciona un proceso desde el jobs pool y lo ejecuta. En sistemas como clusters de computadoras o sistemas multiprocesadores de alto desempeño es común usar un long term scheduler. En estos sistemas los usuarios submit jobs. Un job consiste de al menos un programa más información sobre los recursos requeridos, como número de cpus y memoria y posiblemente tiempo estimado de ejecución. Estos schedulers planifican el mejor plan de ejecución generalmente con el objetivo de minimizar el tiempo medio de espera.

Algunos algoritmos utilizados para lograr ese objetivo es el de ordenar los trabajos por menor tiempo de ejecución: Shortest job first.

En un sistema time sharing (como muchos SO modernos) no usan long term schedulers, permitiendo que un proceso comience su ejecución luego de su creación y luego usan **short term schedulers** que seleccionan un proceso en base a alguna política o criterio.

Existen diferentes criterios para la planificación del uso de cpu por parte de los procesos:

- Maximizar la utilización de la CPU
- Rendimiento (throughput): número de procesos completados por unidad de tiempo.
- Tiempo de ejecución de un proceso (turnaround).
- Minimizar tiempos de espera por la cpu.
- Minimizar tiempos de respuesta: tiempo de procesamiento de un requerimiento.

Obviamente varios de estos criterios se contraponen entre sí. Por ejemplo, maximizar el rendimiento puede afectar negativamente a lograr bajos tiempos de respuesta.

Algoritmos de planificación:

En ésta sección se tienen en cuenta los procesos en estado READY o RUNNABLE, es decir aquellos que están esperando por usar la cpu.

First-Come, First-Served (FCFS)

Este es el algoritmo más simple. Se implementa comúnmente con una cola (FIFO). Cuando un proceso se torna RUNNABLE se agrega al final de la cola y se le otorgará la cpu cuando esté primero. Su desventaja principal es que comúnmente produce tiempos promedio de espera grandes.

Este algoritmo no es interrumpible (preemptive), es decir que hasta que el proceso no termina, no libera la CPU. Esto hace prácticamente imposible su uso en time-sharing systems en los que la cpu se comparte por intervalos.

Una variación consiste en que los procesos realicen cooperative multitasking, liberando la CPU voluntariamente mediante algún syscall.

Round-Robin (RR)

Esta técnica se implementa generalmente en sistemas time sharing. Es similar a FCFS pero con preemption. Al asignarse la CPU a un proceso se le asigna un quantum.

Quantum: intervalo máximo de tiempo de uso (ráfaga) de CPU. Generalmente se establece en una cantidad de ticks. Un tick es un evento que ocurre en cada interrupción del timer, comúnmente cada algunos milisegundos.

Se implementa con una cola FIFO, donde se agrega un proceso READY al final y se selecciona el primero.

Si el proceso usó la cpu todo el quantum, es decir no hizo ninguna llamada al sistema que permita que el kernel tome el control, será interrumpido por el timer. En este caso se le quita la cpu y se lo encola nuevamente como RUNNABLE.

eos-steps

Ver el paso *07-preemptive* del proyecto *eos-steps*.

Si una task no usó su quantum puede ser porque se durmió antes por alguna razón.

Las tasks que hacen muchas operaciones de entrada y salida se les da más prioridad que aquellas que usan mucho la CPU para hacer muchos cálculos o algo por el estilo. Se le da más quantum a las operaciones que usan más las CPU, y menos quantum a las de E/S que no la usan tanto. Además se les da prioridad a estas últimas para mejorar la experiencia del usuario.

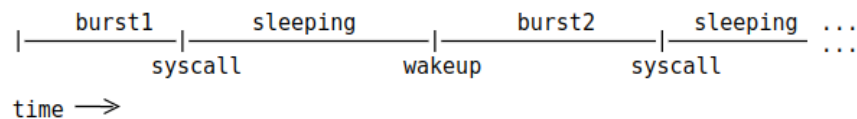
Shortest-Job first

Este algoritmo, comúnmente usado en long term schedulers o en batch systems. En estos casos el programador debe incluir en la descripción del job la duración estimada de ejecución (además de los requisitos de los recursos que requiere).

Esta idea también se puede aplicar en **short-term schedulers** asumiendo como tiempo de ejecución la duración de la próxima ráfaga de cpu. En éste caso, el algoritmo debería llamarse shortest cpu-burst first.

Ráfaga de cpu

Una ráfaga (*burst*) de cpu es el tiempo que un proceso usó la CPU desde que fue planificado hasta que libera la CPU.



Se elige el proceso con menor tiempo de la próxima ráfaga. El principal problema es que la duración de la próxima ráfaga no se conoce.

Una forma de estimar el tiempo de la próxima ráfaga (burst) es tener en cuenta su comportamiento pasado. Es común que un proceso reproduzca su comportamiento al menos temporalmente. Comúnmente se usa una función de promedio exponencial. Sea t_n el tiempo de la última ráfaga y τ_n la última información. La duración de la próxima ráfaga se puede estimar como:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$$

El parametro $0 \leq \alpha \leq 1$ controla el *peso relativo* al valor reciente y la *historia pasada*. Si $\alpha = 0$, entonces $\tau_{n+1} = \tau_n$ y la historia reciente (t_n) no se tiene en cuenta. Si $\alpha = 1$ solo se considera la historia creciente. Un uso común es $\alpha = 0.5$.

Uso de prioridades

Es posible asignar prioridades a los procesos y elegir uno con mayor prioridad. Generalmente se asignan números y su relación de orden puede ser arbitraria (a menor valor corresponde mayor prioridad o viceversa). La asignación de prioridades puede ser estática (at process creation time) o dinámica (en run time).

Este algoritmo puede ser preemptive o no. Uno de los mayores problemas con este algoritmo es starvation: la posibilidad que un proceso quede potencialmente sin planificar, por lo que se considera un algoritmo no justo (unfair).

Una solución a este problema es aplicar envejecimiento (aging): incrementar gradualmente la prioridad de procesos que han estado esperando por un tiempo considerable.

Multilevel queues

En este esquema, los procesos se clasifican en grupos. A cada grupo se le asigna una prioridad, o sea que es un algoritmo basado en prioridades. Los procesos pueden calificar en un grupo debido a su comportamiento y se clasifican dinámicamente. Por ejemplo, un sistema podría considerar dos grupos de procesos:

1. Interactivos.
2. En background (no interactivos) u orientados a uso de CPU.

donde generalmente se les da prioridad a los interactivos con el objetivo de minimizar el tiempo de respuesta ante los usuarios interactuando con el sistema.

Una implementación común usa RR en cada nivel (grupo) aunque se podrían usar diferentes algoritmos en cada nivel. El scheduler selecciona el primer proceso del grupo con mayor prioridad, como se muestra en la siguiente figura.

Cada nivel podría asociar un quantum diferente.

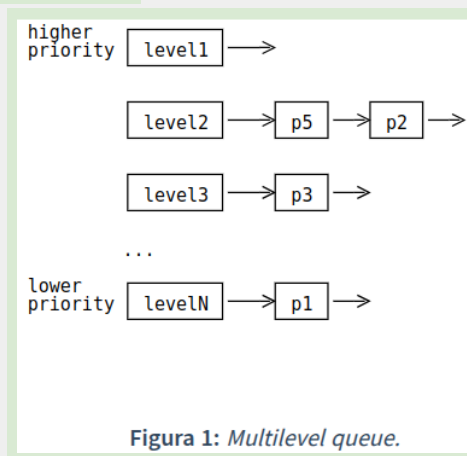


Figura 1: Multilevel queue.

La variante **multilevel feedback** (mlf) permite que los procesos se muevan o cambien su calificación dinámicamente. Por ejemplo, un proceso que no usó todo el quantum se considera interactivo y se sube de nivel, mientras que el que lo consumió completamente se considera en background u orientado a cpu y se lo baja. Esta estrategia prioriza a los procesos orientados a entrada-salida y contribuye a mejorar los tiempos de respuesta.

Planificación de threads

Algunos SO usan kernel threads para realizar tareas diferidas que generalmente se ejecutan fuera del contexto de una interrupción, como por ejemplo el migration kernel thread en Linux que se encarga de medir la carga de cpus y migrar los procesos a otras cpus para balancear la carga.

Otros kernel threads pueden usarse para procesar paquetes de red recibidos u otras tareas de larga duración, que no conviene realizar en el contexto de una interrupción.

Comúnmente los kernel threads tienen mayor prioridad que los procesos de usuario.

Caso de estudio: Linux scheduler

Los procesos y threads se representan en `struct task_struct`. Un proceso multi-threaded se modela con varias instancias de `task_struct`, donde una representa el hilo principal del proceso y los demás como hilos hijos. En este caso todos comparten el mismo `pid`.

Linux usa un scheduler basado en prioridades en el rango de 0 a 139. La prioridad de un proceso de usuario se determina en base a su valor `nice` (un atributo de un proceso) en el rango -20 a +19.

```
priority = nice + 20
```

La prioridad de un proceso lo categoriza en dos grandes clases:

- Real-time: prioridades 0-99. Se usan políticas FIFO (non-preemptive) o RR con quantum fijo.
- Normal: prioridades 100-140. Se usan políticas RR (con quantum variable), BATCH (para procesos no interactivos) e IDLE (los de menor prioridad).

Varios procesos del sistema y kernel threads ejecutan con prioridades de real-time.

Un proceso o **tarea de tiempo real** requiere que el sistema le asigne los recursos (CPU, memoria, etc) en tiempos acotados (vencimientos). Ejemplo: una aplicación que debe grabar un DVD requiere mantener el buffer de datos leídos de la fuente de origen con una cantidad mínima permanentemente ya que la grabación en un medio óptico es un proceso continuo que no se puede interrumpir (el láser no puede parar de grabar en el medio).

Los procesos normales inician con prioridad 0.

Las últimas versiones incluyen el completely fair scheduler (CFS) el cual usa un red-black-tree para ordenar los procesos por tiempo de uso en base a una línea de tiempo calculada. De esta forma el scheduler tiene costo constante por lo que se lo conoce como el $O(1)$ scheduler.

La llamada al sistema `nice(value)` permite asignar el valor de `nice`. Los usuarios comunes pueden invocar a `nice` con valores 0-19. Sólo el usuario root puede usar `nice` con valores entre -20 y 19.

Linux le da prioridad a los procesos interactivos que se la pasan esperando y no usando la cpu, esto es para minimizar la espera del usuario y mejorar su experiencia.

Multiprocesadores

Algunas arquitecturas de hardware basados en multiprocesadores se desarrollaron según el concepto de asymmetric multiprocessing (amp). En estas arquitecturas una CPU (master) generalmente ejecuta el código del kernel y sus servicios mientras que los procesos de usuarios se ejecutan en las demás cpus (workers). El scheduler corre en el master, por lo cual simplifica los problemas de paralelismo y concurrencia.

Las arquitecturas modernas son multi-procesadores simétricos (smp), es decir que todas las cpus ejecutan código de usuario y del kernel.

En una arquitectura smp se debe poner especial cuidado en los problemas de concurrencia generados por el paralelismo ya que varias cpus pueden estar ejecutando el mismo código del cliente en paralelo.

En un multiprocesador uno de los desafíos es lograr un buen balance de carga (load balancing): emparejar la carga de trabajo entre todas las cpus.

Esto requiere que el sistema pueda contabilizar la carga y asignar a las cpus menos cargadas. Así en cada context switch es posible que un proceso migre de cpu, es decir, que continúe su ejecución en otra cpu.

La migración puede afectar al rendimiento, ya que puede que la memoria caché de la cpu anterior debe quedar invalidada y la caché de la nueva cpu debe ir llenándose al continuar la ejecución del proceso.

Es posible ver que estos dos objetivos se contraponen, por lo que hay que lograr una solución de compromiso, por lo cual muchos schedulers implementan processor affinity, que tratan de minimizar la

migración. En algunos sistemas, como Linux, proveen llamadas al sistema para pedirle al sistema que no migre de cpu al proceso (hard affinity). Para más detalles, ver `set_affinity` syscall.

Una arquitectura multi-core replica varias cpus en un mismo chip. A su vez cada core puede usar múltiples pipelines para permitir ejecutar varios threads o hilos de ejecución en paralelo para minimizar los memory stalls de un core. Una contención se produce cuando un core tiene que esperar por datos en memoria que aún no se han cargado (prefetch) en la caché (cache miss).

Hyperthreading: capacidad de que un core pueda ejecutar más de un thread en paralelo. Comúnmente se implementa asignando varios pipelines al core. Así, una CPU de dos cores con dos pipelines cada uno puede verse como un multiprocesador con 4 CPUs.

In RISC-V, un hart es una abstracción de un hardware thread o logical processor: una unidad lógica de procesamiento con su propio conjunto de registros. Una implementación puede hacerlo mediante cores que usan múltiples pipelines.

Concurrencia e IPC (Inter Process Communication)

En un kernel de un SO hay que considerar los problemas generados por la concurrencia.

Dos o más threads o procesos cooperativos se comunican mediante el uso de algún recurso en forma compartida. En el caso que dos threads compartan datos en su mismo espacio de memoria. Este modelo de comunicación se conoce como **shared memory**. En el caso de procesos en diferentes espacios de memoria (o en diferentes computadoras en una red) será necesario algún mecanismo de pasaje de mensajes.

Los mecanismos de sincronización y comunicación ofrecidos por un SO se conoce como inter-process communication (IPC).

En un kernel reentrante aún con una única cpu hay que considerar que el código puede tener interleaving (diferentes posibles secuencias de ejecución con otro código) producido por el mecanismo de interrupciones.

Lo mismo ocurre en procesos de usuarios en donde el programa puede capturar eventos como señales o excepciones.

Por ejemplo, sea el siguiente código ejecutándose con las interrupciones habilitadas.


```

struct queue q;

void interrupt_handler() {
    ...
    enqueue(q, data);
    ...
}

void enqueue(int value)
{
    struct node * new = new_node(value);
    if (!q.first) {
        q.first = q.last = new;
    }
    ...
}

int dequeue() {
    ...
    if (!q.last)
        // (1)
        error("queue is empty");
    ...
}

```

Si la cpu está ejecutando un thread de un proceso en la línea (1) en dequeue() y ocurre una interrupción, el flujo de ejecución se interrumpe y se salta al interrupt_handler(). Esta función llama a enqueue() y modifica q.last. Luego, al retorno de la interrupción, la cpu continuará ejecutando (1), que asume un cierto estado de q.last que ha cambiado.

Otro escenario similar ocurre en presencia de paralelismo en multiprocesadores. Si dos o más cpus invocan a enqueue() cuasi-simultáneamente, podría ocurrir una secuencia de operaciones que rompe el invariante de la estructura de datos.

Un tercer escenario similar ocurre cuando a un thread ejecutando en enqueue(), el SO le quita la CPU (en el punto 1, por ejemplo) para darle el control a otro thread que también invoca a enqueue() o dequeue().

Concurrencia: ocurrencia de múltiples intercalados (interleaving) de instrucciones debido a paralelismo en multiprocesadores, interrupciones o thread-switching. La concurrencia es el interleaving que puede ocurrir por el hecho de que hay un planificador de tareas que va quitando y asignando la cpu, se interrumpen esos procesos y por eso se puede dar el interleaving. El mecanismo de interrupciones es otro generador de problemas de concurrencia.

Paralelismo: se da cuando tenemos múltiples procesadores.

Cuando existe la posibilidad que dos o más threads intentan modificar un recurso compartido se dice que existe una condición de carrera o race. O sea, las condiciones de carrera se dan en los casos en donde dos o más procesos leen o escriben datos compartidos y el resultado final depende de quién los ejecuta y cuándo.

La forma de solucionar esto es mediante la exclusión mutua, es decir, si un proceso está usando un recurso compartido, entonces el resto de los procesos no podrán hacer lo mismo, serán excluidos de usarlo.

Surge entonces la necesidad de impedir que se ejecuten trazas que puedan romper invariantes. En el ejemplo anterior, debería asegurarse que las funciones enqueue() y dequeue() ejecuten al menos parte de su cuerpo sin interrupciones, es decir que no sea posible ejecutar una traza que intercale (interleaving) sus operaciones.

Para evitar o tener bajo control estas races se requiere de mecanismos de control de la concurrencia. Si bien la exclusión mutua evita las condiciones de carrera, esto no es suficiente para tener procesos paralelos cooperando correcta y eficientemente mientras usan recursos compartidos. Se necesitan 4 condiciones para tener una buena solución:

1. No puede haber dos procesos simultáneamente en la región crítica.
2. No se deben hacer asunciones sobre la velocidad o el número de CPUs.
3. Ningún proceso que esté ejecutándose fuera de la región crítica puede bloquear otro proceso.
4. Ningún proceso debería tener que esperar para siempre para poder ingresar a la región crítica.

En un sistema de un solo procesador, la solución más sencilla es que cada proceso deshabilite todas las interrupciones justo después de entrar en su región crítica y las vuelva a habilitar justo antes de salir de ella. Con las interrupciones deshabilitadas, no pueden ocurrir interrupciones de reloj. Después de todo, la CPU solo cambia de un proceso a otro como resultado del reloj u otras interrupciones, y con las interrupciones desactivadas, la CPU no cambiará a otro proceso. Por lo tanto, una vez que un proceso ha deshabilitado las interrupciones, puede examinar y actualizar la memoria compartida sin temor a que intervenga ningún otro proceso.

Este enfoque generalmente no es atractivo, ya que no es prudente dar a los procesos de usuario la capacidad de desactivar las interrupciones. ¿Qué pasaría si uno de ellos lo hiciera y nunca las volviera a activar? Eso podría ser el fin del sistema. Además, si el sistema es un multiprocesador (con dos o más CPU), deshabilitar las interrupciones sólo afecta a la CPU que ejecutó la instrucción de deshabilitación. Los demás continuarán ejecutándose y podrán acceder a la memoria compartida.

Locks

Un lock provee exclusión mutua, asegurando que cuando un thread adquirió el lock los demás que lo requieran deberán esperar a que el primero lo libere. Esto permite excluir trazas no deseadas. Es común el uso del término **mutex** como sinónimo de lock.

Los locks son recursos que usamos para proteger el recurso compartido y así prevenir las race conditions. Nos van a permitir mantener los invariantes en los recursos compartidos.

Región crítica se llama la región que los locks delimitan. El objetivo de los locks es lograr la exclusión mutua, que es que un solo thread esté en la región crítica, no varios al mismo tiempo.

Su uso consiste en asociar a cada recurso compartido usado por los threads con un lock de protección. Cada operación sobre el recurso (ej: en funciones enqueue() y dequeue()) deberá adquirir el lock antes de su acceso y liberarlo luego de su uso.

El uso de locks tiene consecuencias adversas como perder rendimiento porque las operaciones se secuencializan y produce contención en los threads que compiten por el recurso.

Retomando el ejemplo anterior, sería posible mantener el invariante de representación de la cola q usando locks de la siguiente forma.

```

struct queue q;
int lq = 0;

void enqueue(int *value)
{
    struct node * new = new_node(value);
    ...
    acquire(&lq);
    ...
    release(&lq);
}

int dequeue() {
    acquire(&lq);
    ...
    release(&lq);
    return value;
}

```

Listado 1: Ejemplo de uso de locks.

Los locks producen contención (esperas) y hay que tratar de minimizarlas. En el ejemplo del [listado 1][Listado-1] si el `acquire()` en `enqueue()` estaría por encima de la línea que crea el nuevo nodo, ésta operación secuencializaría la operación de creación, la cual podría ser ejecutada concurrentemente entre varios threads ya que operan sobre valores locales independientes. Lo mismo ocurre si se mueve el `release()` en `dequeue()` a la línea inmediata antes del `return`.

El código protegido por un lock se conoce como región crítica.

Deshabilitar instrucciones en un multiprocesador no sirve, porque es como si no estuviera eso hecho, puedes tener ambos threads en dos procesadores diferentes ejecutando la misma región crítica ponele. Y el deshabilitar todas las interrupciones provoca que no se aproveche la concurrencia así que no es tan buena solución esto para ese problema.

Spinlocks

Un spinlock hace que cada thread espere ocupando la cpu (en un ciclo). Los spinlocks se usan para esperas cortas. Los otros tipos de locks se llaman sleeplocks que sirven para esperas largas. Un primer intento de implementación podría ser el siguiente.

```

void acquire(int *lk)
{
    while (*lk)
        ;           // (1)
    *lk = 1;        // (2)
}

void release(int *lk)
{
    *lk = 0;
}

```

Listado 2: Primer intento de implementación de spinlocks.

Sea el siguiente escenario de dos threads usando un lock 1;

```

int l = 0;

// thread 1                thread 2
...                        acquire(&l);
acquire(&l);                critical region
critical region            release(&l);
release(&l);                ...
...

```

Suponiendo que la cpu está ejecutando el thread 1 dentro de acquire() en la línea (1), sea el valor de l cero y antes de la línea (2) ocurre una interrupción de reloj y el kernel decide hacer un context switch al thread 2.

Ahora el thread 2 también invoca a acquire() y el valor de l aún es cero, por lo que ejecutará la línea (2) y continuará su ejecución.

En este punto, el kernel vuelve a hacer un context switch al thread 1. El cual continúa en la línea (2). En este punto, ambos threads están dentro de la región crítica, es decir que no se ha podido garantizar la exclusión mutua.

El problema es que el lock en sí mismo es un recurso compartido y las operaciones en acquire() y release() deberían ejecutarse atómicamente. En éste caso se debería impedir que el scheduler pueda realizar un cambio de contexto. Porque puede pasar lo siguiente:

Supongamos que un proceso lee el lock y ve que es 0. Antes de poder establecer el lock en 1, se ejecuta otro proceso y se establece el lock en 1. Cuando el primer proceso se ejecuta de nuevo, también establecerá el lock en 1, y dos procesos estarán en sus regiones críticas al mismo tiempo.

Se puede inhabilitar el scheduler deshabilitando las interrupciones en la cpu.

```

void acquire(int *lk)
{
    cli(); // clear (disable or ignore) interrupts
    while (*lk)
        ;           // (1)
    *lk = 1;        // (2)
}

void release(int *lk)
{
    *lk = 0;
    sti(); // enable interrupts
}

```

Listado 3: Segundo intento de implementación de spinlocks.

Solución de Peterson

Peterson propuso en 1981 un algoritmo para resolver el problema de la región crítica o exclusión mutua basado en un modelo de memoria compartida. La propuesta inicial sólo contempla el problema de la exclusión mutua entre dos threads como se puede ver en el siguiente listado.

```

bool flag[2] = {false, false};
int turn;

lock(i)                                unlock(i)
{                                        {
    flag[i] = true;                      flag[i] = 0;
    other = i==0? 1: 0;                  }
    turn = other;
    while (flag[other] && turn == other) ;
}

// thread 0                                thread 1
lock(0);                                    lock(1);
// critical section                        // critical section
// ...                                    // ...
// end critical section                    // end critical section
unlock(0);                                unlock(0);

```

Listado 4: Solución de Peterson.

Esta solución se basa en usar la variable `flag[i]` para representar la intención del thread `i` de ingresar en la región crítica y la variable `turn` para indicar qué thread tendría la prioridad para ingresar.

Si bien es posible extender el algoritmo a N threads, como lo hace el algoritmo filter. Este algoritmo funciona sobre N threads identificados por valores [0..N-1] y usa una tabla `level[N]` donde `level[i]` representa el nivel o posición en una cola de espera para ingresar a la región crítica. El nivel cero indica la posición inmediata para entrar en la región crítica.

El `flag[0] = true` significa que el thread tiene la intención de entrar a la región crítica. `turn = 1` significa que lo cede al entrar a la región crítica. `flag[1] = true` significa que el otro thread tiene intención de entrar. La cantidad de flags va a aumentar en base a la cantidad de threads que haya.

El problema de esta solución es que para usarla debemos conocer a priori la cantidad de procesos que se van a ejecutar, y eso es difícil de saberlo en la práctica.

Si no deshabilitamos las interrupciones en la región crítica la cpu nos puede traer problemas de concurrencia, al menos en el mismo procesador necesitamos que dentro de la región crítica no ocurra ninguna interrupción.

Si dos threads intentan ingresar a la región crítica a la vez, podemos tener esto:

Ambos ponen su `flag[i] = true`. Ambos ceden el turno al otro: `turn = other`, entonces, el turno termina con el valor del último hilo que ejecutó eso. En el while, uno de los dos hilos encontrará que `turn != other`, y podrá avanzar. El otro se queda esperando hasta que el primero haga `flag[i] = false`.

```
int level[N], last_to_enter[N];
enter(i)
{
    for (int l = 0; l < N; l++) {
        level[l] = 1;
        last_to_enter[l] = i;
        while (last_to_enter[l] == i && same_or_higher(i,l))
            ;
    }
}

same_or_higher(i,j)
{
    for (k=0; k<N; k++) {
        if (k != i && level[k] >= j)
            return true
    }
    return false;
}

leave(i)
{
    level[i] = -1;
}
```

Listado 5: Algoritmo filter.

El principal problema es que el algoritmo filter no garantiza una espera limitada.

Spinlocks en multiprocesadores

Estas soluciones funcionan en una arquitectura uniprocador. En un multiprocesador, las interrupciones se deshabilitan por cpu, por lo que dos cpus diferentes podrían ejecutar en paralelo `acquire(&1)` y podrían darse dos trazas paralelas las cuales ambas entrarían en la región crítica.

Si comúnmente implementar un big lock por medio de deshabilitar las interrupciones en todo el sistema, ésto generalmente es más costoso y puede producir esperas innecesarias.

El principal problema es que en `acquire()` la condición de espera consiste en al menos dos operaciones que pueden ser interrumpidas en el medio.

1. Test: se carga de la memoria y se analiza el valor del lock.
2. Set: se escribe el valor 1 en el lock.

Es necesario lograr estos dos pasos de manera atómica.

Las cpus modernas proveen instrucciones atómicas para poder implementar locks. Las instrucciones más comunes se conocen como swap o test-and-set.

El compilador gcc provee la función `__sync_lock_test_and_set(&lock, value)` que básicamente es equivalente a la siguiente operación atómica:

```
int __sync_lock_test_and_set(int *lock, int value)
{
    int tmp = *lock;
    *lock = value;
    return tmp;
}
```

En risc-v gcc emite la instrucción `amoswap`. Con esta operación atómica, es posible implementar spinlocks en multiprocesadores.

```
void acquire(int *lk)
{
    cli(); // clear (disable) interrupts
    while (__sync_lock_test_and_set(lk, 1) != 0)
        ;
    __sync_synchronize();
    *lk = 1;
}

void release(int *lk)
{
    __sync_lock_release(lk); // *lk = 0;
    __sync_synchronize();
    sti(); // enable interrupts
}
```

Listado 4: Implementación de spinlocks en multiprocesadores.

Si el valor anterior era 0, el lock estaba libre → el proceso lo adquiere.

Si el valor era 1, el lock estaba ocupado → se queda girando en el bucle hasta que se libere.

Además de las dificultades de implementar locks efectivamente hay que tener en cuenta que tanto el compilador como las cpus pueden alterar el orden de las instrucciones generadas o ejecutadas con respecto a cómo figuran en el código fuente. Comúnmente esto se hace para mejorar el rendimiento o hacer un mejor aprovechamiento de los registros.

Esto se hace en instrucciones que se consideran independientes, es decir, que no tienen dependencias entre sus operandos.

En estos casos es necesario instruir al compilador y al hardware que no reordene.

Además en los multiprocesadores modernos comúnmente se usan modelos de memoria relajados, en el cual una escritura en memoria por una cpu puede no ser visible en otra. El objetivo es reducir el número de acceso a RAM y aprovechar los datos en las cachés locales a cada cpu (caché L1).

Esto puede ser un problema para implementar estas primitivas ya que si un cambio en el valor del lock no es visible por otra cpu, no funcionaría.

Estas arquitecturas ofrecen comúnmente una instrucción que actúa como una barrera de memoria o fence, que al ejecutarse, las escrituras se deben reflejar en las otras cpus (generalmente invalidando las líneas de caché locales a cada cpu). Generalmente estas instrucciones también indican que cualquier reordenamiento no debe cruzar un fence.

En gcc la función `__sync_synchronize()` actúa como una barrera de memoria. En la plataforma risc-v gcc genera la instrucción fence.

Ver la implementación de spinlocks dados en el paso 02-hello-smp de eos-steps.

Si bien estas implementaciones son correctas, presentan un problema: cuando un proceso quiere entrar en su región crítica, comprueba si la entrada está permitida. Si no lo está, el proceso simplemente permanece en un bucle esperando a que se le permita entrar.

Este enfoque no solo desperdicia tiempo de CPU, sino que también puede tener efectos inesperados. Consideremos una computadora con dos procesos: H, con alta prioridad, y L, con baja prioridad. Las reglas de programación establecen que H se ejecuta siempre que esté listo. En un momento determinado, con L en su región crítica, H está listo para ejecutarse (por ejemplo, al completarse una operación de I/O). H comienza entonces a esperar, pero como L nunca se programa mientras H se ejecuta, L nunca tiene la oportunidad de salir de su región crítica, por lo que H entra en un bucle infinito.

Ahora vamos a ver algunas primitivas de comunicación entre procesos que bloquean en lugar de desperdiciar tiempo de CPU cuando no se les permite ingresar a sus regiones críticas.

Sleep locks

Un spinlock sólo es útil para realizar esperas cortas, es decir cuando la región crítica toma muy poco tiempo ya que se está usando la CPU en la espera.

En el caso que la espera dependa de una condición que puede cambiar dentro de demasiado tiempo es preferible suspender la ejecución del thread o proceso y reiniciarlo cuando la condición haya cambiado. Esto es típico cuando en una región crítica se deben hacer operaciones de entrada-salida o computaciones largas.

Un sleep lock realiza esta tarea. Son similares a las variables de condición, las cuales asocian un conjunto de procesos a una condición de espera (el lock). Una API de sleep locks podría ser de la siguiente forma:

- `void sleep(task t, wait_queue q)`: pasa t a estado SLEEPING dejándolo encolado en la cola q.
- `task* wakeup(wait_queue* q)`: desencola una tarea de q pasándola a estado RUNNABLE.

Obviamente, la implementación de estas primitivas requiere el uso de primitivas de locks como los spinlocks ya que la cola es un recurso compartido y `sleep` y `wakeup` pueden ejecutarse concurrentemente.

En los sleeplocks vamos a mandar a dormir a la tarea que quiera acceder a la región crítica pero no puede, el scheduler no la va a elegir para que se ejecute porque no va a estar en estado RUNNABLE. Vamos a tener que usar también una cola de espera en donde se guardan todas las tareas dormidas. Esa cola de espera representa el por qué están esperando esas tareas, puede haber una cola para acceder al disco por ejemplo.

Uso de locks

El uso de locks debe ser cuidadoso porque pueden generar problemas como deadlock.

Uno de los principales problemas es cuando el uso de locks no está encapsulado en una única función, sino que una lo adquiere y se debe liberar en otra. Esto muestra que los locks no son modulares. El uso de locks crea efectos colaterales que deben ser documentados.

Si un thread intenta adquirir un recurso previamente adquirido por él mismo, se bloqueará por siempre (**self lock**). Este problema puede resolverse implementando locks recursivos.

Otro problema es el uso de múltiples locks. Suponiendo que se usan locks como se muestra a continuación.

Los locks no son modulares, podemos tener problemas por culpa de esto. Si haces un acquire adentro de un acquire ese thread se mata a sí mismo.

```
f() {                                g() {
    ...                               ...
    acquire(&l1);                     acquire(&l2);
    ...                               ...      (1)
    acquire(&l2);                     acquire(&l1);
    ...                               ...
    release(&l2);                     release(&l1);
    ...                               ...
    release(&l1);                     release(&l2);
}
```

Listado 2: Uso de locks.

En este caso hay un potencial deadlock. Ya que un thread ejecutando `f()` en el punto (1) y otro thread ejecutando `g()` en el mismo punto quedarán esperándose mutuamente (deadlock).

Este escenario produce una espera circular. La forma de evitarlo es siempre manteniendo el mismo orden en el uso de los locks. En un sistema complejo esto puede ser muy difícil de lograr.

Deadlock

Un **deadlock** entre un grupo de procesos ocurre cuando se dan las siguientes condiciones simultáneamente:

1. **Exclusión mutua:** Los procesos han adquirido un recurso en forma exclusiva.
2. **Espera y retención:** Un proceso retiene un recurso mientras espera por otro.
3. **No quita (preemption):** Los procesos sólo liberan sus recursos voluntariamente.
4. **Espera circular:** Existe una dependencia circular de recursos. Sean los procesos $P_0, P_1, \dots, P_{n-1}, P_i$, para todo i tal que $0 \leq i < n$, espera por un recurso adquirido por $P_{(i+1) \bmod n}$.

Generalmente en un SO se debe aplicar una disciplina particular en el uso de locks. Por ejemplo, se debe definir el orden de uso de locks en todos los módulos del sistema.

Semáforos

Un semáforo comúnmente representa una cantidad de recursos disponibles. Es una primitiva de sincronización más general que los locks.

Un semáforo n-ario (o *counting semaphore) permite que hasta n threads accedan a recursos compartidos. Cualquier thread adicional que intente acceder al recurso deberá esperar hasta que algún otro lo libere. Comúnmente un semáforo se implementa con un contador más un sleeplock. Una API de semáforos generalmente define al menos las siguientes operaciones:

1. `sem_init(s, value)`: inicializa el valor del semáforo s.
2. `sem_wait(s)`: si el valor de s está en cero, suspende al invocante. Sino, decrementa el valor del semáforo.
3. `sem_signal(s)`: incrementa el valor de s y despierta a los threads o procesos bloqueados en s.

Es fácil de ver que si $n = 1$ sólo 1 thread podrá acceder al recurso garantizando exclusión mutua, lo que lo hace equivalente a un mutex.

El siguiente programa muestra el ejemplo clásico del problema del productor-consumidor con semáforos. El problema consiste en que hay un (o más) productor que produce valores en un buffer acotado de capacidad N y uno o más consumidores que extraen valores del buffer y los procesan.

Los productores y consumidores se ejecutan concurrentemente. Comúnmente el buffer se maneja como una cola (disciplina FIFO).

Los semáforos nos permiten acceder a un recurso hasta n procesos y no aseguran la exclusión mutua creó. Cada semáforo tiene una cola de espera asociada. Podemos implementar exclusión mutua con semáforos binarios, inicializando n en 1 para que solo un thread pueda acceder a la región crítica.

```
buffer buf[N];
semaphore mutex = 1,
             full = N,
             empty = 0;

void producer()
{
    while (true) {
        value = gen_value();
        sem_wait(&full);
        add_to_buffer(value);
        sem_notify(&empty);
    }
}

void consumer()
{
    while (true) {
        sem_wait(&empty);
        value = get_from_buffer();
        sem_signal(&full);
        process(value);
    }
}

void add_to_buffer(value)
{
    sem_wait(&mutex);
    // add value to buffer
    sem_signal(&mutex);
}

item_type get_from_buffer()
{
    sem_wait(&mutex);
    // extract item from buffer
    sem_signal(&mutex);
}
```

Listado 3: Productor-consumidor con semáforos.

El semáforo `full` representa el número de valores en el buffer y el semáforo `empty` representa el número de slots libres en el buffer.

Las operaciones sobre el buffer usan un semáforo binario o mutex para asegurar sus invariantes de representación ante la concurrencia.

Monitores

Como ya se analizó, el uso de locks y semáforos no es simple. El principal problema es que no son modulares. En 1973 Hansen y Hoare en 1974 propusieron un mecanismo de alto nivel llamado monitor. Un monitor es una construcción sintáctica de un lenguaje de programación que encapsula datos y las operaciones sobre ellos. Los procesos o threads sólo pueden manipular y acceder a los datos sólo por medio de las operaciones del monitor. Es posible ver un monitor como un objeto, clase o módulo que es thread safe, es decir que los invariantes de sus operaciones están protegidos en presencia de concurrencia.

Su implementación común es usar variables de condición y al menos un mutex para asegurar la exclusión mutua entre las operaciones concurrentes sobre los mismos datos. Una variable de condición representa una cola de threads asociada a un mutex interno esperando a que una condición *c* se torne verdadera. Comúnmente se implementa con dos operaciones *wait(c)* y *signal(c)*.

Un monitor es un módulo en donde todas sus operaciones cuando son invocadas desde un thread está garantizada la exclusión mutua.

El siguiente listado muestra el ejemplo del productor-consumidor usando un monitor en un lenguaje hipotético:

```
monitor PC {
    condition full, empty;
    data buffer[N];
    int count = 0;

    void put(data item) {
        if (count == N) wait(full);
        insert(buffer, item);
        if (++count == 1) signal(empty);
    }

    data get() {
        if (count == 0) wait(empty);
        data item = remove(buffer);
        if (--count == N-1) signal(full);
    }
};

void producer() {
    while (true) {
        data item = gen_item();
        PC.put(item);
    }
}

void consumer() {
    while (true) {
        data item = PC.get();
        process(item);
    }
}
```

Listado 4: Productor-consumidor con un monitor.

Algunos lenguajes con soporte de concurrencia soportan monitores. El lenguaje Java permite definir un monitor mediante el uso de métodos sincronizados. Una clase que declara métodos con el modificador *synchronized* asegura que su ejecución no tendrá interleavings con otros métodos sincronizados sobre el mismo objeto.

Java tiene las operaciones `wait()` y `notify()` que operan sobre el monitor del objeto. Son similares a `wait(channel)` y `wakeup(channel)` ya vistas en donde `channel` es implícito (o equivalente a `this`).

También Java provee `notifyAll()` que despierta a todos los threads esperando en el monitor (`notify()` despierta sólo uno seleccionado aleatoriamente).

El uso de monitores permite escribir programas concurrentes de una forma más modular. Esta idea se puede aplicar en lenguajes sin monitores, usando una disciplina de modularización equivalente, encapsulando y ocultando el uso de locks.

Mensajes

Los mecanismos analizados anteriormente se basan en el modelo de concurrencia basado en `shared memory`. En el caso que dos o más procesos (cada uno ejecutando en su propio espacio de memoria) quieran comunicarse deberán usar algún mecanismo de pasaje de mensajes.

El modelo `message-passing` se basa en al menos dos primitivas del tipo:

- `send(destination, message)`
- `receive(source, &message)`

La operación `send` envía datos a un proceso identificado como `destination` mientras que `receive` recibe un mensaje del emisor `source` (que puede ser `any`).

Estos mecanismos permitirían comunicar procesos locales o remotos (entre procesos ejecutándose en diferentes computadoras conectadas en red).

La comunicación puede implementarse usando buffers de mensajes o `mailboxes` o sin ellos.

En el primer caso la comunicación se dice *asíncronica* ya que el emisor no tiene que bloquearse ya que el mensaje quedará en el `mailbox`. Un receptor sólo se bloqueará si el `mailbox` está vacío.

Si no se usan buffers, la comunicación será *síncronica* ya que los procesos deberán coincidir (esperarse) en las operaciones `send/receive`. Esto se conoce como un *rendezvous*.

Si usas un buffer, no es necesaria la sincronización entre las tareas (que una tarea esté lista para mandar y la otra para recibir), se puede hacer una comunicación *asíncronica*.

En un SO con diseño *micro-kernels* como `MINIX`, los subsistemas (gestor de procesos, memoria, sistemas de archivos, etc) se implementan como servers que reciben requerimientos de los demás subsistemas por medio de un mecanismo de mensajes y tienen la siguiente forma:

```
int main(void)
{
    message m;
    init_subsystem();
    while (TRUE) {
        receive(ANY, &m);
        switch (m.type) {
            case REQUEST_1:
                ...
                send(other_subsystem, m2);
                ...
                break;
            ...
        }
    }
}
```

Es común que un sistema basado en pasaje de mensajes ofrezca operaciones de alto nivel basados en diferentes patrones de comunicación para grupos de procesos como los siguientes:

- `broadcast(sender, receivers, data)`: envía un dato a un conjunto de procesos.
- `scatter(sender, receivers, array)`: distribuye partes del array de datos a los receptores.
- `gather(receiver, senders, &array)`: el receptor reúne en array las partes de datos enviados por los emisores.
- `value = mapreduce(sender, receivers, op, array)`: el emisor distribuye (map) los datos a los receptores. Cada uno aplica la operación `op` sobre los datos recibidos (en paralelo). El emisor recibe los valores computados y también aplica la operación `op` para producir el valor final. La operación `op` debe ser asociativa y conmutativa para poder explotar el máximo paralelismo.
- `barrier(group)`: actúa como una barrera de sincronización. Los procesos pueden continuar luego que todos ejecuten (alcancen) la barrera.

El estándar Message-Passing Interface (MPI) define una API con operaciones de este tipo. Es ampliamente usado en redes de computadoras o clusters y permite abstraer la red como un sistema multiprocesador para realizar computación de alto desempeño (high performance computing (HPC)).

Remote procedure call

El mecanismo de invocación a procedimientos o funciones remotas o RPC abstrae el mecanismo de pasaje de mensajes subyacente para que un programador en lugar de usar primitivas del tipo `send/receive` realice un pedido de un servicio a un proceso remoto usando el concepto familiar de invocaciones a funciones.

Un servidor ofrece una API en forma de un conjunto de tipos de datos y funciones. Esto comúnmente se define en un interface definition language (IDL) para independizarse del lenguaje de programación de implementación.

Esta remote procedure call sirve para que como programador usemos una función como si estuviéramos en un monolítico donde esa función por debajo maneja mensajes. Internamente esta función transforma cosas en un mensaje.

```
interface memory {  
    int malloc(int size);  
    int free(void *address);  
}
```

Listado 5: Interface de ejemplo de un memory allocator.

Un IDL compiler genera desde la interface los stubs para el cliente y el servidor en un lenguaje de programación específico.

```

rpc_server s = rpc_get_server();
void* malloc(int size)
{
    rpc_call m = {.f = MALLOC_MSG, .args = [rpc_marshall(size), 0]};
    send(rpc_server, m);
    rpc_response r;
    receive(rpc_server, &r);
    return rpc_unmarshall(r.result);
}

void free(void *address)
{
    ...
}

```

Listado 6: Ejemplo de un stub del cliente.

El programa cliente se enlaza con el client stub y podrá invocar al servicio remoto simplemente invocando a `data_ptr = malloc(sizeof data);`.

```

extern void * malloc(int);
extern void free(void *);

void dispatcher(void) {
    while (true) {
        rpc_call m;
        receive(ANY, &m);
        switch (m.f) {
            case MALLOC_MSG:
                void * result = malloc(rpc_unmarshall(m.args[0]));
                rpc_response r;
                r.result = rpc_marshall(result);
                send(m.sender, r);
                break;
            case FREE_MSG:
                ...
                break;
        }
    }
}

```

Listado 7: Ejemplo de un stub del servidor.

El server stub se enlaza con el programa servidor (el cual debe implementar las funciones de servicio de la API) y consiste básicamente en un dispatcher que recibe los mensajes que representan invocaciones remotas de los clientes e invoca a las funciones que implementan cada servicio.

Las operaciones de marshalling y unmarshalling serializan y deserializan datos. La serialización codifica un dato en alguna forma (binaria o textual) para ser transmitido. La deserialización es el proceso inverso: obtiene el valor recibido en un mensaje y se representa como un dato de un determinado tipo en el lenguaje de programación usado.

Por ejemplo, un valor de tipo entero puede representarse en una estructura de datos de la forma `{type: INT, value: 5}` para transmitirse como un parámetro o un resultado en una invocación remota.

Futuros y Promesas

Un futuro o promesa refiere a una abstracción que representa un valor que será computado por algún otro componente (thread) de ejecución concurrente. Se usan comúnmente para implementar data-flow variables en entornos concurrentes: variables que ante una lectura y ésta no tiene asignado un valor, el thread debe "esperar" por que otro le asigne algún valor. Las operaciones de lectura actúan como un mecanismo de sincronización.

En este modelo es común que se dispongan de las siguientes operaciones:

- `future f = async g(args)`: invocación asincrónica de `g()`. El invocante continúa con la ejecución. Su implementación comúnmente ejecuta `g` en un nuevo thread. Retorna un future: una variable data-flow. En vez de disparar el thread explícitamente, el thread se dispara implícitamente.
- `value = f.get()`: esta operación, comúnmente ejecutada por el invocante de un future `f = async g(...)`, obtiene el valor (encapsulado en `f`) posiblemente bloqueando al invocante hasta que el valor sea computado por `g()`.

Este mecanismo es built-in en algunos lenguajes de programación como en Oz en los cuales la operación `get()` es implícita. En otros lenguajes como en C++, se implementan en bibliotecas y el acceso a un future es explícito (como `f.get()`).

Concurrencia sin locks (lock free)

Una estructura de datos que puede usarse concurrentemente y que no usa operaciones de bloqueo se denomina **lock-free**. Obviamente su ventaja es que es posible mejorar el rendimiento de un sistema y seguridad: si un thread finaliza con un lock adquirido, los demás threads intentando acceder no podrán progresar.

Una estructura de datos **wait-free** si cada thread que la usa puede completar sus operaciones en un número acotado de pasos.

Así, un spinlock califica como lock-free pero no como wait-free. El diseño de estas estructuras de datos no es simple. Su implementación generalmente requiere de instrucciones atómicas del tipo `compare_and_swap(&target, old, new)` la cual realiza los siguientes pasos atómicamente.

1. Compara los valores de `target` con el valor `old`.
2. Si son iguales, a `target` le asigna `new` y retorna `true`.
3. Sino, retorna `false`.

El listado 8 muestra un fragmento de una lista enlazada lock-free.

```
/* insert at front */
push(list* head, T value)
{
    list_node * n = create_node(value);
    n->next = head->first; // (1)
    while (!compare_and_swap(&(head->first), n->next, n)) ; // (2)
}
```

Listado 8: Ejemplo de `push()` lock-free.

La operación atómica `compare_and_swap()` permite verificar si otro thread modificó `head->first` entre las líneas (1) y (2), en cuyo caso la operación `head->first = n` debería reintentarse.

La estrategia consiste en identificar los puntos de programa en los que puede haber una race condition y asegurarse que se logró la actualización preservando el invariante. Se debe notar que es muy difícil de lograr cuando están involucrados varios pasos.

Los locks generan esperas, podríamos hacer concurrencia sin espera con lock free.

Se basan en la hipótesis de que las race conditions tienen una baja probabilidad de ocurrir. Vamos a hacer algoritmos optimistas, si caemos en una race condition, se ejecuta de vuelta hasta que no caigamos más ahí. El 99% de las veces va a andar, tenemos que tener la mala suerte de que no funcione al tener otro thread intentando modificar lo mismo que nosotros.

IPC en UNIX

Los SO tipo UNIX se caracterizan por ofrecer una buena variedad de mecanismos de sincronización y comunicación entre procesos como los que se listan en la siguiente tabla.

Mecanismo	Llamadas al sistema
File	<i>open, read, write, close, ...</i>
Pipes	<i>pipe, read, write, close, ...</i>
Named pipes	<i>mkfifo, read, write, close, ...</i>
Signals	<i>kill, signal, alarm, ...</i>
Sockets	<i>listen, connect, accept, send, recv, ...</i>
Message queues	<i>msgget, msgrcv, msgctl, ...</i>
Semaphores	<i>semget, semctl, semop, ...</i>
Shared memory	<i>shmget, shmat, shmctl, ...</i>
Memory mapped files	<i>mmap, munmap, ...</i>

Tabla 1: Mecanismos de IPC en sistemas UNIX.

Todos estos mecanismos permiten la comunicación y sincronización entre procesos locales (en una misma computadora) a excepción de los sockets que permiten comunicar procesos en diferentes computadoras (nodos) de una red.

Sockets: descriptores que vamos a usar para comunicarse con otros procesos.

Gestión de la memoria

La memoria principal o Random Access Memory (RAM) consiste en un arreglo de bytes. Cada byte se identifica por su posición o dirección en el arreglo. Esta memoria es volátil, es decir que los datos almacenados se pierden cuando se corta la energía. La cpu interactúa directamente con la RAM en cada ciclo de ejecución de cada instrucción. En las arquitecturas del tipo Von-Newman, las instrucciones de programa y los datos se almacenan en la misma memoria.

Tanto los programas como los datos están en la RAM.

Memorias dinámicas: periódicamente hacen un refresco y es por esto que son más lentas.

En un sistema multitarea la memoria se debe particionar lógicamente para separar las instrucciones y datos de cada programa en ejecución. Esto requiere que mínimamente un SO deba gestionar cómo se asigna memoria al kernel y a los procesos y además utilizar mecanismos de protección para que cada proceso se

ejecute confinado, es decir que sólo pueda acceder a sus áreas de memoria con el fin de evitar que un proceso pueda invadir las áreas de los otros procesos o del kernel.

La cpu ejecuta repetidamente el ciclo de instrucción que comúnmente tiene tres pasos:

1. Fetch (lectura) de la instrucción: se carga en un registro interno la instrucción con la dirección de memoria del program counter (pc).
2. Decodificación: se identifica el tipo de instrucción y sus operandos.
3. Ejecución de la instrucción: esto puede requerir leer o escribir en la memoria.

Si la memoria física de la computadora es lo suficientemente grande como para que entren todos los procesos, va a estar joya. Sin embargo, en la práctica, la cantidad total de RAM que necesitan todos los procesos suele ser mucho mayor de la que cabe en la memoria.

Swapping: cuando un SO necesita asignarle memoria a un proceso pero no hay más, lo que se hace es desalojar un bloque de la memoria (que seguro está siendo usado por otro proceso) y guardarlo a disco y este bloque se lo asignamos al primer proceso que intentó ejecutarse. Cuando el proceso al que se le quitó ese bloque quiere acceder a su bloque se va a producir una excepción y el SO va a traer sus datos desde el disco, que si no hay lugar va a tener que hacer el mismo proceso de desalojar un bloque. Esto pasa cuando el SO está ejecutando procesos que necesitan más memoria de la que hay. Es por esto que el disco pertenece al esquema de la jerarquía de memoria (esquema 1), porque es una extensión de la memoria.

Cada proceso tiene su propio espacio de direcciones, independiente del de otros procesos (excepto en circunstancias especiales donde los procesos desean compartir sus espacios de direcciones). Algo difícil es cómo dar a cada programa su propio espacio de direcciones, la dirección 28 en un programa significa una ubicación física diferente a la dirección 28 en otro programa.

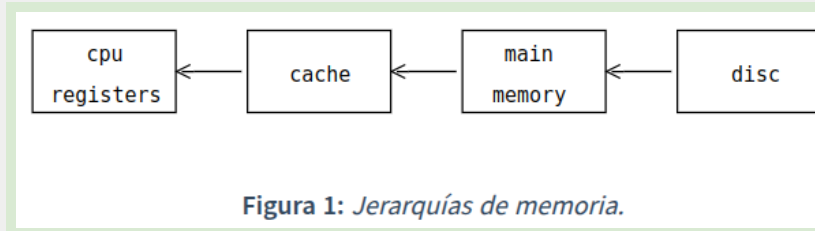
Jerarquía de memorias

La cpu comúnmente usa registros (memoria de alta velocidad) para la ejecución de las instrucciones. Es común que una operación de lectura o escritura desde/hacia la memoria tome varios ciclos de reloj, haciendo que la cpu deba frenarse (stall). Para atenuar estas diferencias de velocidades se usan memorias caché: memorias de menor capacidad pero de más alta velocidad de acceso que la RAM convencional.

Una memoria caché está diseñada para el acceso a los datos por contenido en lugar de direcciones. Cuando la cpu hace referencia a una dirección de memoria, si el contenido de esa dirección está en la caché, se recupera desde allí. Si no se encuentra en la caché, se accede a la RAM y se almacena en la caché para accesos futuros.

La cpu internamente tiene memoria que son sus registros. Para mejorar la velocidad entre la RAM y los registros de la CPU tenemos una caché. Las caché son de alta velocidad (estáticas) pero con menor capacidad que la RAM y andan a velocidades más cercanas a la que camina la CPU. Cada cosa que se carga en la RAM queda en la caché.

Finalmente, para almacenar programas y datos en forma permanente se usan dispositivos de almacenamiento como discos magnéticos, de estado sólido u otra tecnología. Esta arquitectura define una jerarquía de memorias como se muestra en la siguiente figura ordenadas por velocidad de acceso de izquierda a derecha.



Asignación de memoria

El kernel deberá asignar y liberar bloques de memoria a los procesos y para las estructuras de datos del kernel mismo. Una vez que el kernel se cargó en memoria durante el proceso de carga (boot), deberá reconocer el área de memoria libre disponible para asignar. Esta área de memoria se conoce como heap. El kernel deberá mantener un heap (pool) de memoria para poder asignar y liberar bloques antes las demandas de los procesos y el mismo kernel.

Comúnmente se usan algunas de las dos principales estrategias de gestión del heap. El heap requiere mantener conjuntos de bloques libres y asignados (ocupados).

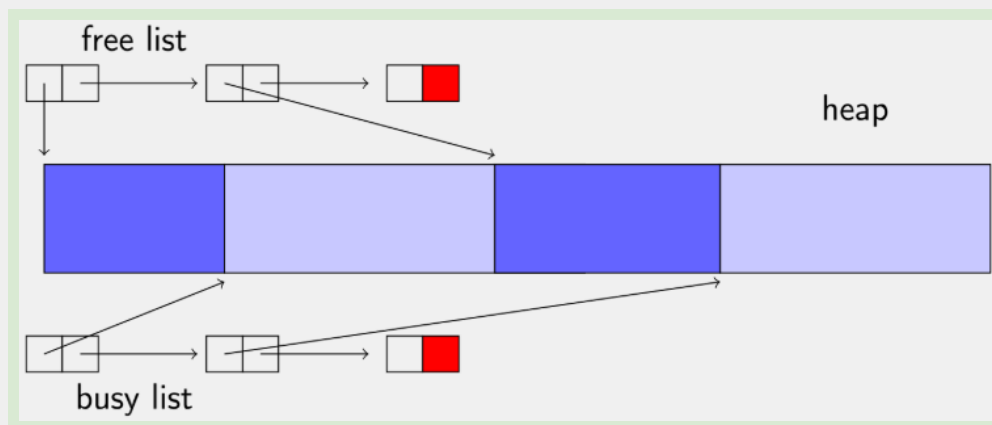
1. Asignación de bloques de tamaño variable.
2. Asignación de bloques de tamaño fijo.

En el primer caso el heap se particiona lógicamente y se asignan bloques de tamaño variable. En el segundo se particiona y asignan bloques de tamaño fijo.

Heap con bloques de tamaño variable

La primera estrategia comúnmente se implementa con una lista de bloques libres de la siguiente forma.

- Inicialmente existe un único bloque libre del tamaño del heap.
- Ante una solicitud de un bloque de tamaño n , se busca el bloque libre b con tamaño $b.size \geq n$. Si $b.size > n$, se particiona y se asigna el bloque de tamaño n quedando un bloque libre b' con $b'.size = b.size - n$. Si no existe un bloque tal, se retorna un error.
- Ante una liberación de un bloque, se inserta en la lista de bloques libres.



Obviamente, el algoritmo de búsqueda de un bloque que satisfaga el requerimiento afectará el rendimiento del sistema. Algunas estrategias para reducir la complejidad (tiempo) de la búsqueda pueden ser:

1. **First fit:** se asigna el primer bloque con tamaño $\geq n$. Se mantiene ordenada la lista de los bloques libres del bloque más grande al más chico, cuando se quiere almacenar algo primero se fija en el primer bloque que va a ser el más grande, si lo que quiero almacenar es más grande que eso ya está no tenes otro bloque más grande que ese. Lo malo de esto es que va a provocar que cada vez

tengamos bloques más chicos y dijimos que siempre es mejor tener bloques lo más grandes posibles. Este algoritmo es rápido porque busca lo menos posible.

2. **Best fit:** se asigna el bloque que mejor ajusta, es decir, un bloque b con $b.size = m$, $m \geq n$ y $\forall b' \in freelist \mid b'.size \geq b.size$. Si lo ordeno de menor a mayor en el peor de los casos va a ser lineal, perdemos un poco de rendimiento porque vamos a tener que recorrer la lista hasta encontrar uno lo suficientemente grande para lo que queremos guardar, pero tiene la ventaja de que favorecemos la fragmentación ya que usamos los bloques más chicos primero, el bloque que mejor encaja. Este algoritmo busca en toda la lista, de principio a fin, y selecciona el espacio más pequeño que sea adecuado. En lugar de dividir un espacio grande que podría necesitarse más adelante, el best fit busca un espacio cercano al tamaño real necesario para que se ajuste mejor a la solicitud y a los espacios disponibles. El best fit es más lento que el first fit porque debe buscar en toda la lista cada vez que se llama. Sorprendentemente, también genera más memoria desperdiciada que el first fit, ya que tiende a llenar la memoria con pequeños huecos inútiles. El first fit genera huecos más grandes en promedio.
3. **Worst fit:** se asigna el bloque libre más grande. Esta estrategia no se usa básicamente.
4. **Siguiente ajuste:** memorización del último bloque asignado.

A medida que se hacen los requerimientos y liberaciones de bloques, el heap se puede ir fragmentando o particionando en un gran número de bloques libres pequeños. Esto impediría que se puedan satisfacer requerimientos de bloques grandes. Este problema se conoce como **fragmentación externa**, en donde hay muchos bloques libres pequeños.

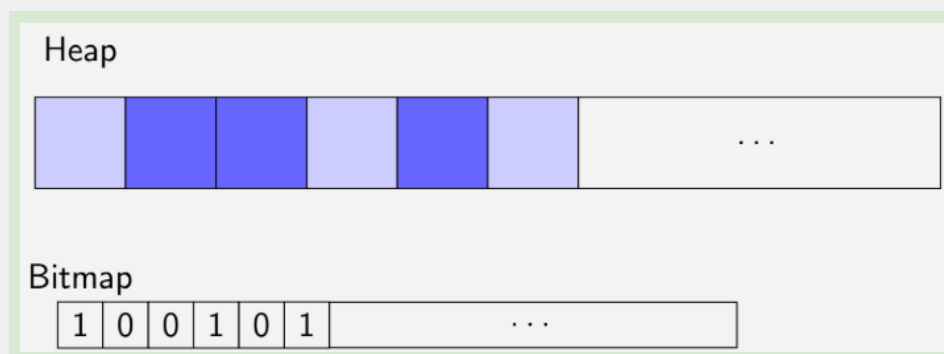
Una estrategia que puede aliviar la fragmentación es que ante una liberación de un bloque, si existen bloques libres adyacentes, se fusionen o compacten en un único bloque libre de mayor tamaño. Esta estrategia se conoce como semi-compactación (fusión de bloques libres adyacentes).

Heap con bloques de tamaño fijo

En este caso el heap se particiona en m bloques de un tamaño fijo s .

Ante un requerimiento de un bloque de tamaño n :

- Si $n \leq s$ se asigna cualquier bloque libre.
- Sino, se deben asignar el mínimo de k bloques contiguos tal que $k \times s \geq n$.



Comúnmente las listas (conjuntos) de bloques libres y usados se representan como un bit vector, lo que es una gran ventaja en términos de uso de memoria.

En éste esquema, el tamaño s de los bloques es crítico. Un valor grande podría aumentar la fragmentación interna, es decir el remanente de los bloques asignados que no se usarán. Un valor muy pequeño podría requerir encontrar varios bloques libres contiguos. Si no se pueden encontrar, no se puede satisfacer el requerimiento a pesar que puede haber muchos bloques libres en el heap.

Esta estrategia tiene el problema de que cuando nos piden algo de tamaño menor que los bloques terminamos desperdiciando espacio, esto se llama fragmentación interna. Tenemos pedazos de bloques que no se van a usar nunca. La pregunta es con qué tamaño de bloques dividir el heap. Un tamaño razonable que se usa es de 4K bytes para cada bloque. Si tienes algo que ocupa más espacio que un solo bloque, vas a tener que asignarle dos bloques contiguos a ese dato, o los necesarios para que entre.

¿Qué haces cuando te están pidiendo más memoria de lo que tiene ese bloque? Esta es una desventaja, puedes hacer fragmentación interna en donde le das dos bloques continuos, aunque le puedes terminar dando más memoria de lo que necesita así.

La asignación de bloques de tamaño variable no pasa esto pero si es más difícil de gestionar.

Es mejor ir dejando bloques más grandes.

La mayoría de los heaps de bloques de tamaño variable usan la estrategia de semi compactación.

En el heap con bloques del mismo tamaño podemos llevar la lista de los bloques libres como un conjunto de bits. Podemos tener un arreglo bitset que cada posición representa un bloque, será 0 si el bloque está libre y 1 si no lo está por ejemplo. Esto está bueno porque en una sola estructura tengo los bloques libres y los ocupados. Si tengo $n = 128$ bloques, voy a necesitar un bitset de 16 bytes ($128/8 = 16$).

Este bitset lo podemos tener en memoria porque no va a ocupar mucho lugar, es algo bien compacto.

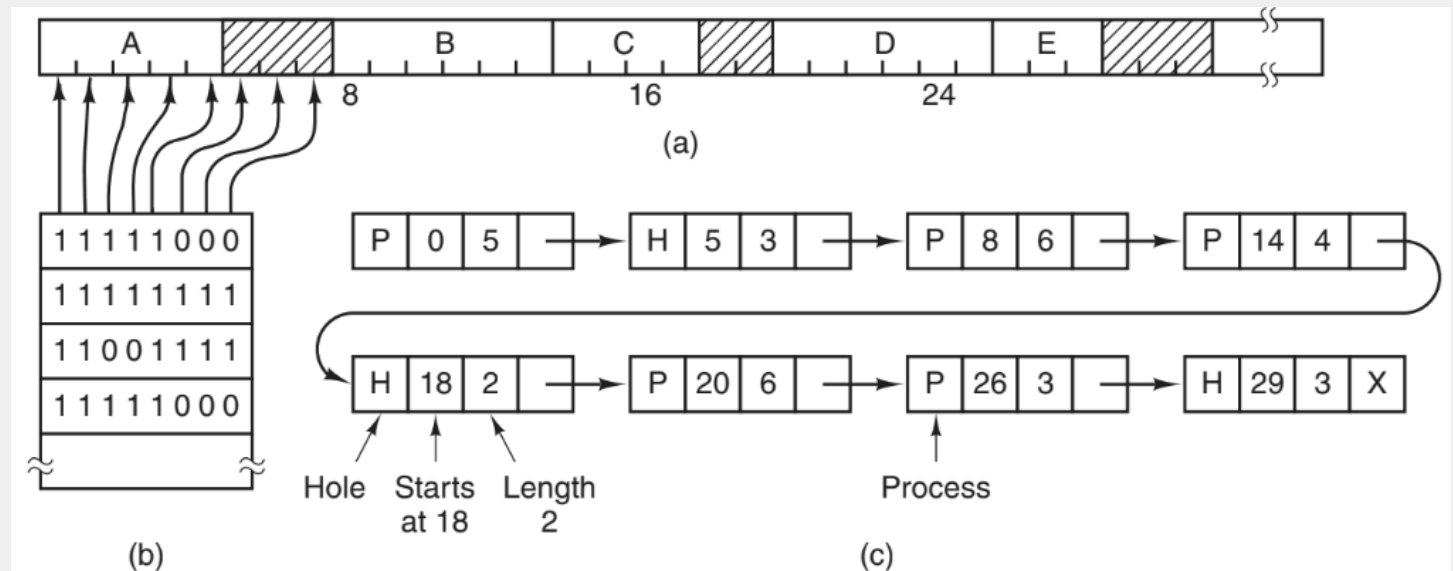


Figure 3-6. (a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.

Ventaja de esta estrategia: simplicidad. Desventaja: fragmentación interna.

El sistema compañero (buddy system)

Para poder lograr los beneficios de las dos estrategias mencionadas, es posible diseñar alguna solución híbrida combinandolas.

El buddy system se basa en particionar el heap en bloques libres de tamaño desde 2^l hasta 2^h (Kb). Por ejemplo, con $l = 16$ y $h = 20$ los bloques pueden ser 64K, 128K, 256K, 512K y 1024K bytes. El algoritmo funciona de la siguiente manera:

1. Inicialmente el heap se particiona en bloques de tamaño 2^h (1024KB).
2. Ante un requerimiento de tamaño n
 - a. Seleccionar un b con mejor ajuste.
 - b. Si $b.size \geq 2n$ dividir b en dos bloques b_0 y b_1 de tamaño $b.size/2$.
Sea $b = b_0$, repetir este paso hasta que sea $b.size = 2^l$.
3. Asignar el bloque b .

En una liberación de un bloque se aplica el algoritmo a la inversa. Si quedan bloques contiguos (compañeros) de igual tamaño menores a 2^h , se unen en un bloque más grande.

1. Inicialmente hay bloques libres de tamaño 2^h
2. En una solicitud de tamaño m :
 - 2.1 Buscar un bloque libre de mejor ajuste ($b_i : m \geq 2^i$)
 - 2.2 Si existe, asignarlo
 - 2.3 Sino:
 - 2.4 Dividir a la mitad un bloque de tamaño 2^{i+1}
 - 2.5 Si se alcanzó tamaño de mejor ajuste, asignarlo
 - 2.6 Sino, repetir con $2^{i+2}, \dots$

Ejemplo con $l = 16(2^l = 64Kb)$, $h = 19(2^h = 512Kb)$

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	512K							
$P_0 : 34K$	P_0	64K	128K		256K			
$P_1 : 95K$	P_0	64K	P_1		256K			
$P_2 : 55K$	P_0	P_2	P_1		256K			
$P_3 : 90K$	P_0	P_2	P_1		P_3		128K	

• En una liberación de un bloque:

1. Marcar el bloque como libre
2. Si hay un vecino libre:
 - 2.1 Combinarlos y volver al paso 2

Continuación del ejemplo...

Req.	64K	64K	64K	64K	64K	64K	64K	64K
	P_0	P_2	P_1		P_3		128K	
$freeP_2$	P_0	64K	P_1		P_3		128K	
$freeP_0$	128K		P_1		P_3		128K	
$freeP_1$	256K				P_3		128K	
$freeP_3$	512K							

O sea, vamos a tener bloques de distintos tamaños pero ese tamaño va a estar dentro de un rango, en donde el tamaño es de potencia de 2. Inicialmente el heap tiene los bloques con el mayor tamaño posible. Si lo que nos están pidiendo es la mitad de un bloque, este bloque se divide a la mitad y se sigue respetando el tamaño de bloque de potencia de 2.

Un bloque compañero es un bloque contiguo del mismo tamaño.

Asignador de potencias de 2

- Listas de bloques de tamaños $= 2^k$ ($2^l \leq k \leq 2^h$)
- Listas libres por separado
- No se combinan bloques
- Dado un requerimiento de bloque de tamaño m :
 1. Se localiza la menor lista i tal que $2^i \geq m$ y tenga un bloque libre
 2. Si no existe ninguna, el requerimiento falla

Interface del manejador del heap

- Alto nivel: `void* malloc(size_t size)`, `void free(void* ptr)`
- Bajo nivel: `void* alloc_frame()`, `void free_frame(void* ptr)`

Protección (segmentación)

El objetivo de la segmentación es el de reubicabilidad y protección para evitar accesos a memoria no permitidos.

Es necesario que un proceso sólo pueda acceder a su área de memoria. Suponiendo que se asigna la memoria a un proceso en forma contigua, haría falta una maquinaria que permita la cpu detectar accesos fuera de rango. Este mecanismo podría basarse en un registro de control que contenga la dirección base y la dirección límite. Además podría tener flags de permisos de acceso, como lectura, escritura, ejecución y otros.

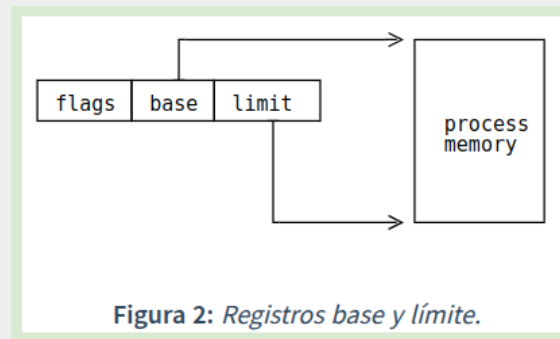


Figura 2: Registros base y límite.

Cuando se ejecuta un proceso, el registro base se carga con la dirección física donde comienza su programa en la memoria y el registro límite se carga con la longitud del programa.

Cuando el SO realiza un context switch a un proceso asigna las direcciones correspondientes a estos registros. La cpu en cada acceso a memoria verifica que la dirección se encuentre en el área especificada por los registros: $base \leq address \leq limit$.

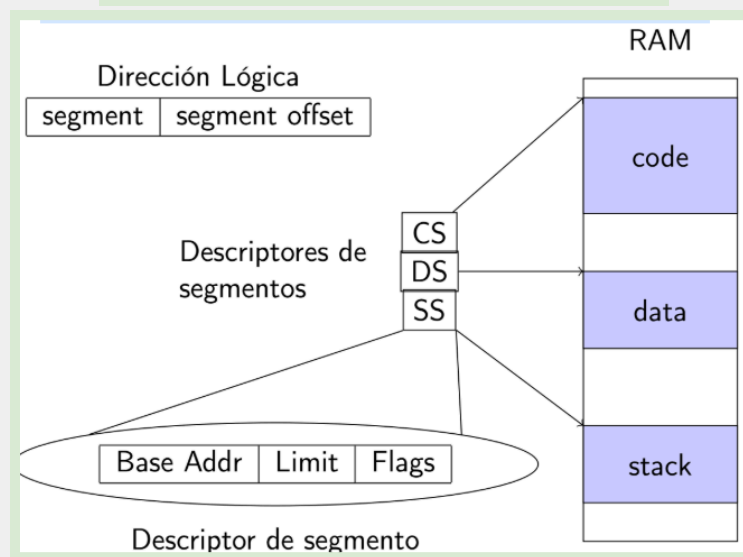
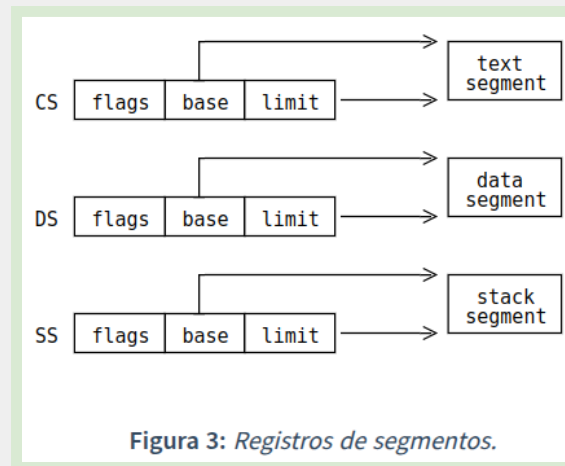
En caso que en una ejecución de una instrucción refiera a una dirección fuera de los límites, o que la operación de lectura o escritura no cumpla con los permisos la cpu genera una memory fault exception.

Comúnmente, el SO seguramente atraparará la excepción mediante algún exception handler y si no es recuperable, eliminará el proceso. Este error comúnmente se observa como un segmentation fault en sistemas tipo UNIX.

Este mecanismo se puede extender al uso de varios de estos registros, para que las áreas o segmentos (texto, datos, stack y heap) de un proceso no necesariamente se deban almacenar en forma contigua (no siempre se va a encontrar un bloque libre tan grande). En este caso el mecanismo se conoce como segmentación.

Un segmento es una unidad lógica de un proceso. Vamos a tener un conjunto de descriptores de segmentos por proceso.

A modo de ejemplo, la arquitectura Intel IA32 (x86) soporta los siguientes registros de segmentos (base+límit): code segment (CS), data segment (DS), stack segment (SS), extra segment (ES) y otros (GS, FS), como se muestra en la siguiente figura.



Tenemos los siguientes segmentos:

1. Segmento de código (instrucciones del programa).
2. Segmento de datos (variables globales).
3. Stack: control de llamadas a subrutinas (y retornos), variables locales (automáticas), valores temporarios (cálculos de expresiones).
4. Heap (variables creadas dinámicamente).

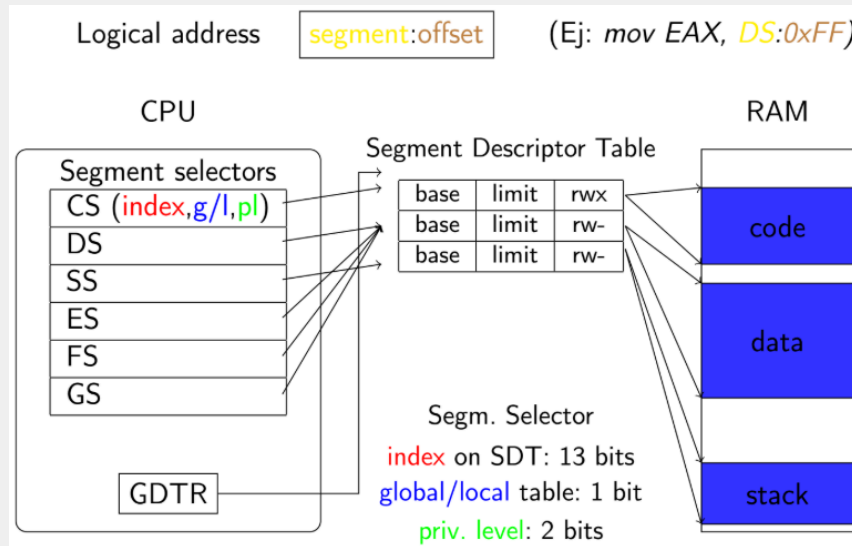
El *Code Segment* tendría que tener el permiso de lectura y ejecución (x) pero no escritura.

El *Data Segment* (variables globales) tendría que tener el permiso de lectura y escritura.

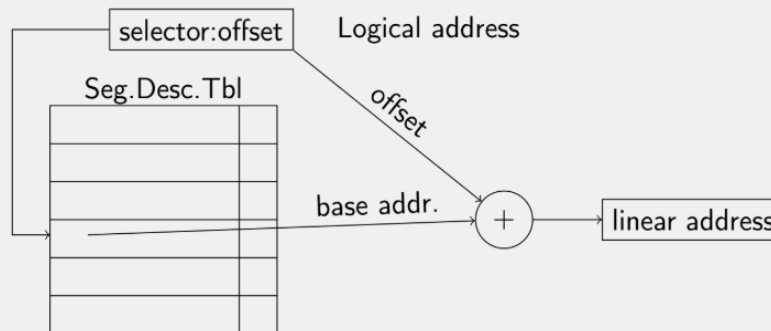
El *Stack Segment* debería tener el permiso de lectura y escritura pero no de ejecución (esto es una medida de seguridad que no lo tenga).

Una desventaja de este mecanismo es la necesidad de realizar una suma y una comparación en cada referencia de memoria. Las comparaciones pueden realizarse rápidamente, pero las sumas son lentas debido al tiempo de propagación del acarreo, a menos que se utilicen circuitos de suma especiales.

Segmentación en IA32:



Direccionamiento:



Protección: la CPU genera una Segmentation fault en un acceso fuera de los límites de un segmento o en un acceso a un segmento sin los permisos necesarios.

Con los segmentos estamos dividiendo la memoria en segmentos de tamaño variable. ¿Qué pasa si queremos dividir la memoria en bloques de tamaño fijo? Esto es lo que hace el paginado.

Multiprogramación y reubicabilidad

Ejecución de código independiente de su posición

Múltiples programas cargados en memoria

Los programas de usuario comúnmente se compilan para ser ejecutados a partir de cierta dirección de memoria: cada función o variable global tiene una dirección asignada. Tiene el problema de que varios programas esperan ser cargados desde la misma dirección base.

Posibles soluciones:

1. **Compilación a Position independent code (PIC):**

Con esta maquinaria, es posible lograr que el código de un programa pueda ejecutarse independientemente de la dirección base de carga. Esto puede lograrse con código independiente de su posición y básicamente

consiste en que las direcciones de memoria (operandos de las instrucciones) representan direcciones relativas (con respecto a una dirección base) en lugar de direcciones absolutas.

Los registros de segmentos contienen las direcciones base.

Así la cpu debe transformar una dirección relativa en una dirección física sumándole el valor de la dirección base del segmento.

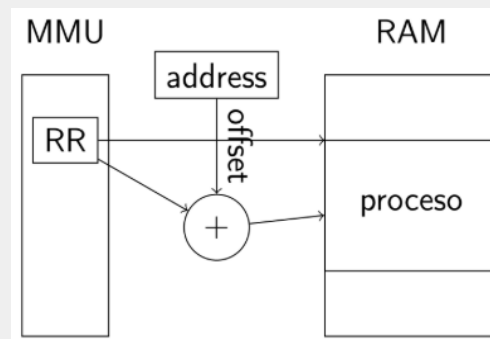
A modo de ejemplo, en x86, una dirección de memoria en una instrucción se denota como `segment:offset`. Una instrucción de salto del tipo `jmp address`, es equivalente a `jmp cs:address`, una instrucción de lectura/escritura desde/a memoria (ej: `mov %eax, address`) equivale a `mov %eax, ds:address`. Las instrucciones sobre la pila (`push/pop`) refieren por omisión al segmento `ss`.

PIC: uso de un registro de link/load que se inicializa por el SO con su dirección base de carga.

Las instrucciones usan direccionamiento relativo a ese registro u otro (ej: al program counter). En CPUs sin esta maquinaria, el linker/loader deberá recalcular las direcciones de los operandos de las instrucciones en tiempo de carga, antes de iniciar su ejecución. En CPUs con MMUs, las direcciones de un programa son lógicas. La MMU traduce las direcciones lógicas en reales (físicas).

2. Uso de Memory Management Unit (MMU):

Registros base: uso de un registro de reubicabilidad RR. Cada dirección de memoria se interpreta como un offset. La MMU implementa la función $phys_addr = RR + offset$



Paginado

La idea básica del paginado es que cada programa tiene su propio espacio de direcciones, dividido en fragmentos llamados páginas. Cada página es un rango contiguo de direcciones. Estas páginas se asignan a la memoria física, pero no todas las páginas tienen que estar en la memoria física al mismo tiempo para ejecutar el programa. El espacio de direcciones virtuales consta de unidades de tamaño fijo llamadas páginas. Las unidades correspondientes en la memoria física se denominan page frames. Las páginas y los page frames suelen tener el mismo tamaño. Cuando el programa hace referencia a una parte de su espacio de direcciones que está en la memoria física, el hardware realiza la asignación necesaria sobre la marcha. Cuando el programa hace referencia a una parte de su espacio de direcciones que no está en la memoria física, el sistema operativo recibe una alerta para que busque la parte faltante y re ejecute la instrucción fallida.

Con el paginado, en lugar de tener una reubicación separada solo para los segmentos de texto y datos, todo el espacio de direcciones se puede asignar a la memoria física en unidades bastante pequeñas.

La RAM física se divide en bloques del mismo tamaño, llamados page frames, por ejemplo de 4KB cada uno. Cada proceso tiene su propio espacio de direcciones virtuales. La memoria virtual del proceso se divide en páginas (por ejemplo, 4KB también), del mismo tamaño que los page frames.

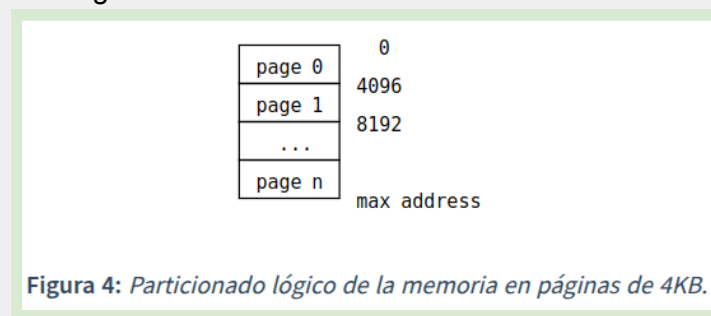
Características del paginado:

- Memoria particionada en bloques de tamaño fijo (páginas)
- Tamaños de páginas comúnmente usadas: 4KB o 4MB
- Ventaja sobre segmentación:
 1. Gestión simple del heap.
 2. Particionado de memoria más fina.
 3. Permite implementar técnicas de virtual memory.

Vamos a implementarlo con una tabla de páginas.

- Virtual/logical address (va): Par (*vpn*, *offset*).
- Tabla de páginas *pt*: *array[0..N-1]* of *pte*
- Page table entry (pte): Par (*ppn*, *flags*)

Como se verá en el siguiente capítulo, la implementación de algunas políticas de memoria virtual se simplifican si la memoria se particiona en bloques pequeños (páginas) del mismo tamaño, típicamente de 4KB como se muestra en la siguiente figura.



Con paginado es posible definir espacios de memoria para cada proceso (y el kernel). Un espacio de memoria está representado por el conjunto de las páginas a las que puede acceder. Estos conjuntos generalmente se representan en la memoria como arreglos y se conocen como tablas de páginas.

Las páginas asignadas a un proceso no necesitan estar físicamente contiguas, aunque se haya definido un espacio de direcciones lógicas contiguas.

Esto requiere de un mecanismo de traducciones de direcciones lógicas o virtuales a direcciones físicas o reales. Esto puede verse como una función *map*: $va \rightarrow pa$, donde *va* es una dirección lógica (virtual address) y *pa* es una dirección física (physical address).

Comúnmente el hardware con soporte de paginado implementa esta función de mapping con una tabla de páginas que representa la función de traducción de direcciones virtuales a físicas por extensión (o por casos). Por cada proceso y para el kernel, el SO construye sus propios espacios de memoria asociando una tabla de páginas a cada uno. Cada entrada en una page table se denomina page table entry (pte) y básicamente es un par de la forma (*ppn*, *flags*). El número de página *ppn* (physical page number) refiere al número de página física y los flags son permisos de acceso y de control.

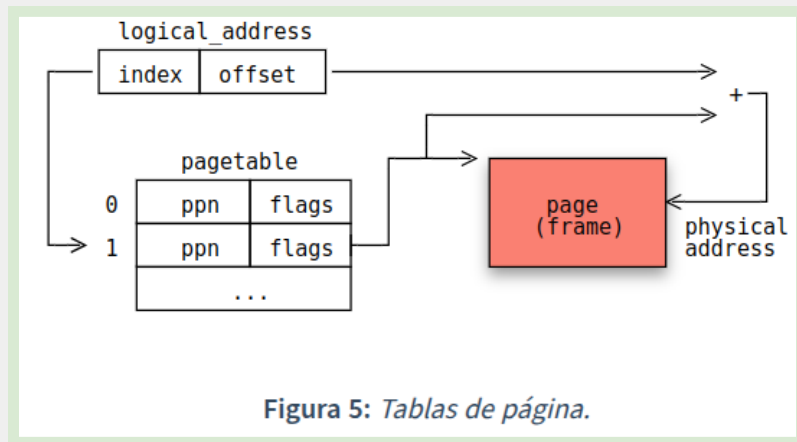
A la memoria la vemos como una secuencia de páginas donde cada página tiene una cantidad de bloques de tamaño fijo. Vamos a tener direcciones virtuales/lógicas. Cada dirección va a tener dos partes (el número de páginas físicas donde pertenece, y el otro es el desplazamiento). Pasamos de dividir la memoria en segmentos a páginas de bloques.

Direcciones virtuales: en paginado, una dirección en una instrucción de programa es en realidad una dirección lógica y generalmente se interpreta como un par (*index*, *offset*).

El *index* refiere a una entrada en la tabla de páginas. La dirección física se obtiene computando:

$$pa = pagetable[index].ppn * PS + offset$$

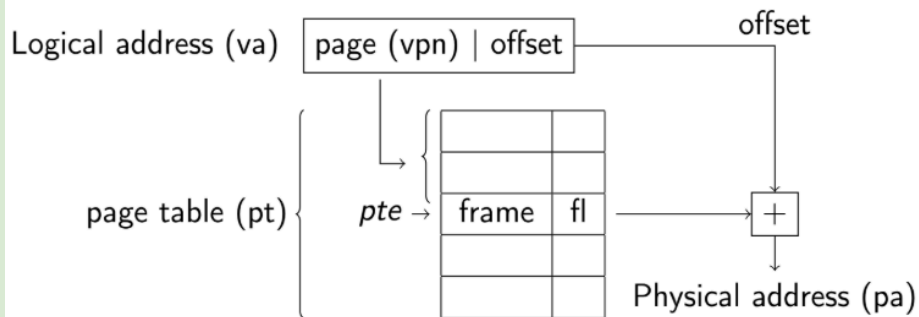
como se muestra en la siguiente figura, donde PS es el page size.



El offset contiene tantos bits como sea necesario dependiendo del tamaño de la página. Por ejemplo, si el tamaño de página es 4096, offset debería tener 12 bits ($2^{12} = 4096$).

La cpu genera una excepción del tipo page fault en el caso que una dirección lógica indexa en una entrada sin permisos de acceso o inválida.

- Tabla de páginas: $pt : logical_frame \rightarrow physical_frame$
- Mapping: $pa = pt[va.vpn].ppn + va.offset^1$



En dirección lógica vos tenes el index (vpn) que hace referencia a una entrada en la tabla de páginas. Esa entrada esta compuesta por su ppn y un conjunto de flags. Ese ppn es la dirección base del frame al que corresponde en la RAM, y con el offset de la dirección lógica, lo usas para moverte dentro de ese frame.

Cuando un programa intenta acceder por ejemplo a la dirección 0 usando la instrucción:

MOV REG, 0

la dirección virtual 0 se envía a la MMU. La MMU detecta que esta dirección virtual se encuentra en la página 0 (0 a 4095), que, según su mapeo, corresponde al page frame 2 (8192 a 12287). Por lo tanto, transforma la dirección a 8192 y la envía al bus. Por lo tanto, la MMU ha mapeado efectivamente todas las direcciones virtuales entre 0 y 4095 a las direcciones físicas 8192 a 12287.

¿Qué ocurre si el programa referencia una dirección no asignada? La MMU detecta que la página no está asignada y causes the CPU to trap to the operating system. Esta trap se denomina fallo de página (page fault). El sistema operativo selecciona un page frame poco utilizado y escribe su contenido de vuelta en el disco (si

no está ya allí). A continuación, recupera (también del disco) la página a la que se acaba de hacer referencia en el page frame recién liberado, modifica la asignación y reinicia la instrucción atrapada.

Con paginado es posible que cada proceso tenga por ejemplo un espacio de direcciones de la forma $[0, \text{size}]$ donde size es el tamaño de la memoria requerida por el proceso. Así, todos los programas se compilan con opciones de memoria por default y pueden coexistir en memoria referenciando las mismas direcciones lógicas ya que en ejecución se traducirán en cada caso a direcciones físicas diferentes.

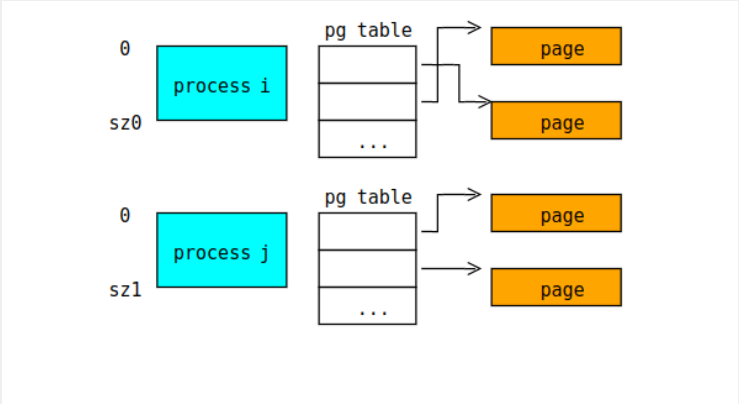


Figura 6: Procesos con espacios de direcciones lógicas no disjuntas.

Tenemos primero las direcciones lógicas que después se traducen a la dirección física. Todo esto lo hace la llamada a *exec*. El proceso de ir armando la tabla es mapear porque mapeas direcciones lógicas a direcciones físicas.

Se debe notar que cuando un proceso usa paginado un bloque de memoria con páginas con direcciones virtuales contiguas no necesariamente se corresponden a bloques de direcciones físicas contiguas. Esto permite que un heap allocator puede asignar páginas a un proceso con cualquier dirección física, la cual será mapeada a una dirección lógica del proceso, como se muestra en la siguiente figura.

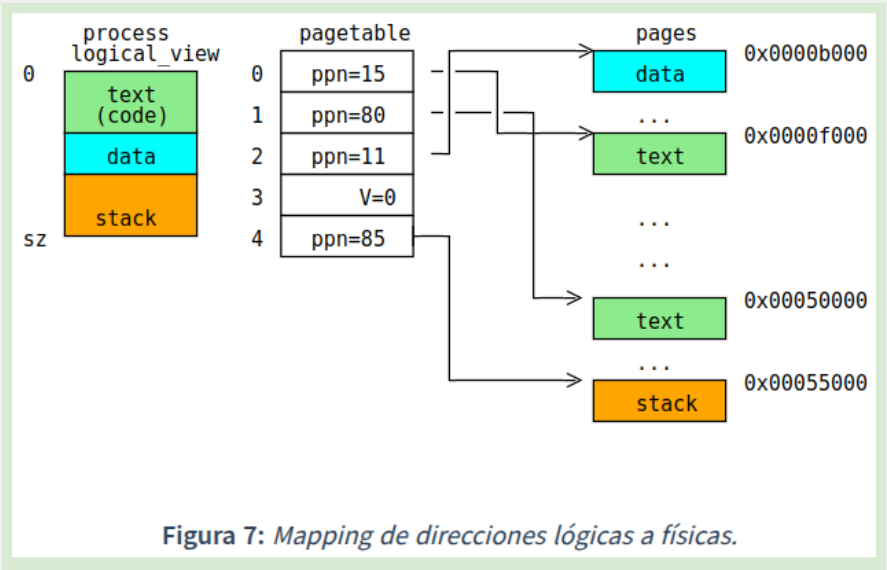


Figura 7: Mapping de direcciones lógicas a físicas.

Las páginas del lado derecho son las direcciones físicas.

El disco se usa en paginado y no en lo del heap con bloques de tamaño variable porque pasar cosas de tamaño variable al disco es un quilombo.

Cada proceso tiene su espacio de direcciones virtuales, dividido en páginas. La tabla de páginas del proceso indica en qué parte de la memoria RAM (frame) está almacenada cada una de sus páginas virtuales.

En la figura anterior se puede apreciar que el espacio contiguo de direcciones lógicas se mapea un conjunto de espacios de direcciones físicas.

En este ejemplo, el proceso tiene dos páginas (8KB) para el área de código, 1 de datos y 1 para el stack. Se deja un hueco en el espacio de direcciones virtuales para detectar stack overflow.

Se debe notar que las páginas de código y datos son lógicamente contiguas pero físicamente están dispersas en la memoria.

Thus, the purpose of the page table is to map virtual pages onto page frames. Mathematically speaking, the page table is a function, with the virtual page number as argument and the physical frame number as result. Using the result of this

function, the virtual page field in a virtual address can be replaced by a page frame field, thus forming a physical memory address.

Para los siguientes ejemplos, asumiremos que una dirección de memoria es de 32 bits y que se usan los 20 bits más significativos como index y los 12 menos significativos como offset en páginas de 4KB.

En la figura anterior, el espacio de direcciones lógicas del proceso es el intervalo $[0, sz = 0x5000]$ (20KB). Cuando una instrucción del programa haga referencia a la dirección lógica $0x2004$ (área de datos) la cpu la interpreta con $index=2$ y $offset=0x004$. La pte en la entrada 2 contiene el physical page number = 11. La dirección física final será $11 * 4096 + 004 = 0xb004$.

Viéndolo en binario y no en hexa:

Paso 2: Convertir `0x2004` a binario

Sabemos que:

- `0x2` = `0010`
- `0x0` = `0000`
- `0x0` = `0000`
- `0x4` = `0100`

Entonces:

yaml

`0x2004 = 0010 0000 0000 0100`

Eso son **16 bits** (los de más a la izquierda son ceros si queremos que sea de 32 bits).

Si lo completamos a 32 bits sería:

yaml

`0000 0000 0000 0000 0010 0000 0000 0100`

→ Entonces, los **12 bits menos significativos son el offset**

css

[20 bits para index] [12 bits de offset]

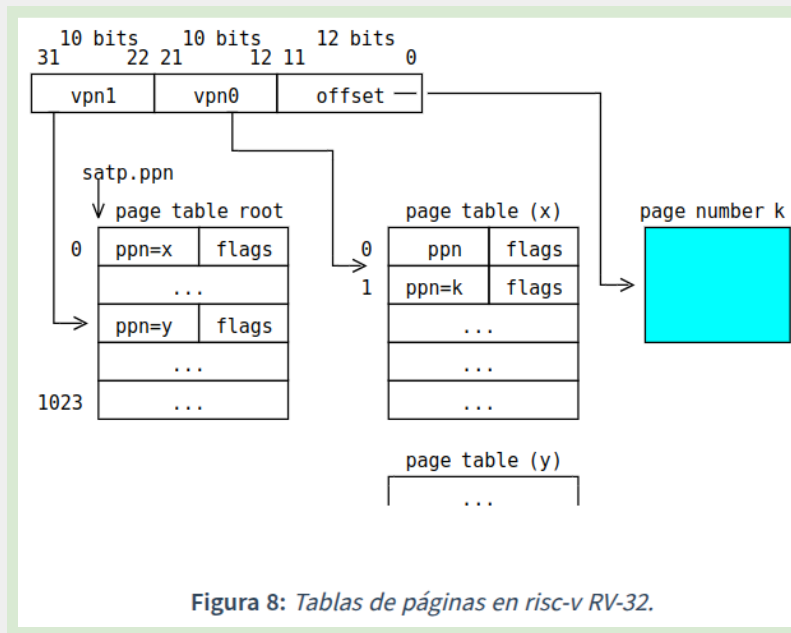
[00000000000000000010] [000000000100]

Las tablas de páginas deberían tener suficientes entradas como para barrer toda la memoria direccionable por el sistema. Comúnmente esto es necesario en un kernel, ya que debe poder acceder a toda la memoria.

Con espacios direcciones grandes, por ejemplo 4GB en una arquitectura de 32 bits, el número de entradas en la tabla de páginas puede ser considerable.

En estos casos comúnmente el hardware permite definir tablas de páginas que refieran a páginas de mayor tamaño, como 4MB o más. Esto permite reducir el número de entradas en una tabla de páginas.

Aún así comúnmente es deseable que las páginas sean pequeñas para minimizar la fragmentación interna. Además un proceso puede dejar muchos huecos en el espacio de direcciones lógicas, lo que una tabla plana podría desperdiciar memoria. Esto hace que las arquitecturas comúnmente representen la tabla como árboles de n niveles como en la siguiente figura, en vez de una tabla enorme.



Cada nodo del árbol tiene el tamaño de una página.

La figura de arriba muestra un ejemplo de una tabla de páginas en riscv (RV32), una arquitectura con un espacio de direcciones de 32 bits (4GB). Una tabla de páginas tiene la forma de un árbol de dos niveles. El registro satp contiene el número de página física de la raíz del árbol.

Una dirección lógica se divide en 3 partes: vpn1 (10 bits más significativos) es un índice o virtual page number (vpn) en el nodo raíz del árbol cuya pte contiene el ppn correspondiente a un nodo hoja, vpn0 es el índice en el nodo hoja (cuya pte contiene el ppn de la página o frame destino). Los 12 bits menos significativos son el offset dentro de la página destino.

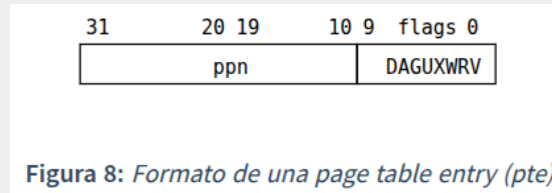
Ese vpn1 me va a hacer referencia a su pte correspondiente dentro del nodo raíz. Está pte dentro del nodo raíz va a tener una ppn y flags. Ese ppn hace referencia al nodo hoja, o sea a la tabla de nivel 1 correspondiente, porque puedes tener varias. Y los flags determinan permisos sobre esa tabla y si es válida o no. Luego tienes el vpn1, que dentro del nodo hoja, vpn1 hace referencia a una pte en particular. Este pte va a tener la dirección base de un frame en particular dentro de la RAM y los flags van a ser en base a esa frame. Una vez que ya estás en la frame correspondiente, con el offset te moves dentro de ella en base a la dirección base que te la dio el ppn.

La dirección lógica de la forma 0x0000100f que se interpreta como

vpn1	vpn0	offset
0000000000	0000000001	000000001111

Esto es que en el nodo raíz se accede a la primera entrada que llevará al nodo hoja. La segunda entrada (index 1) en el nodo hoja tiene el ppn (ppn << 12 es la dirección base del frame) de la página referenciada la cual sumada al offset 0x1111 arroja la dirección física final.

Cada entrada de una tabla de páginas (page table entry - pte) en RV32 tiene la forma:



En ésta configuración la función de traducción de una dirección virtual va a una dirección física pa sería:

$$pa = ((satp.ppn[va.vpn1].ppn * PS)[va.vpn0].ppn * PS) + va.offset$$

donde $satp.ppn$ denota el valor del campo de la physical page number del registro CSR supervisor address translation an protection (satp) y PS es el page size.

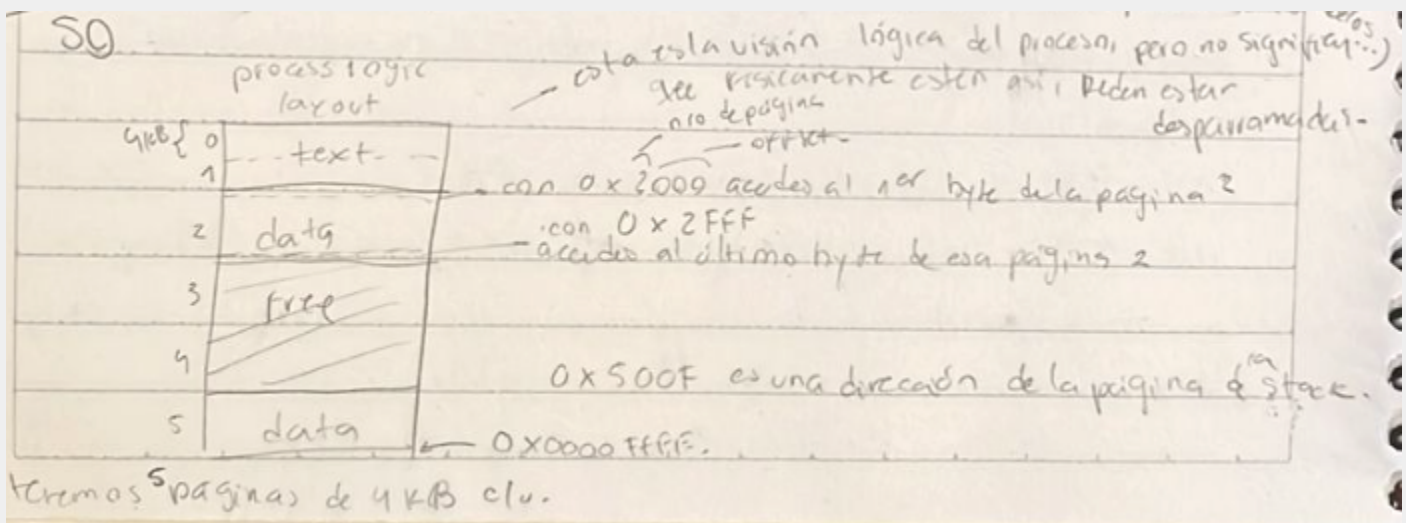
El kernel deberá cambiar el $sapt$ en cada context switch para que reference la tabla de páginas del nuevo proceso seleccionado.

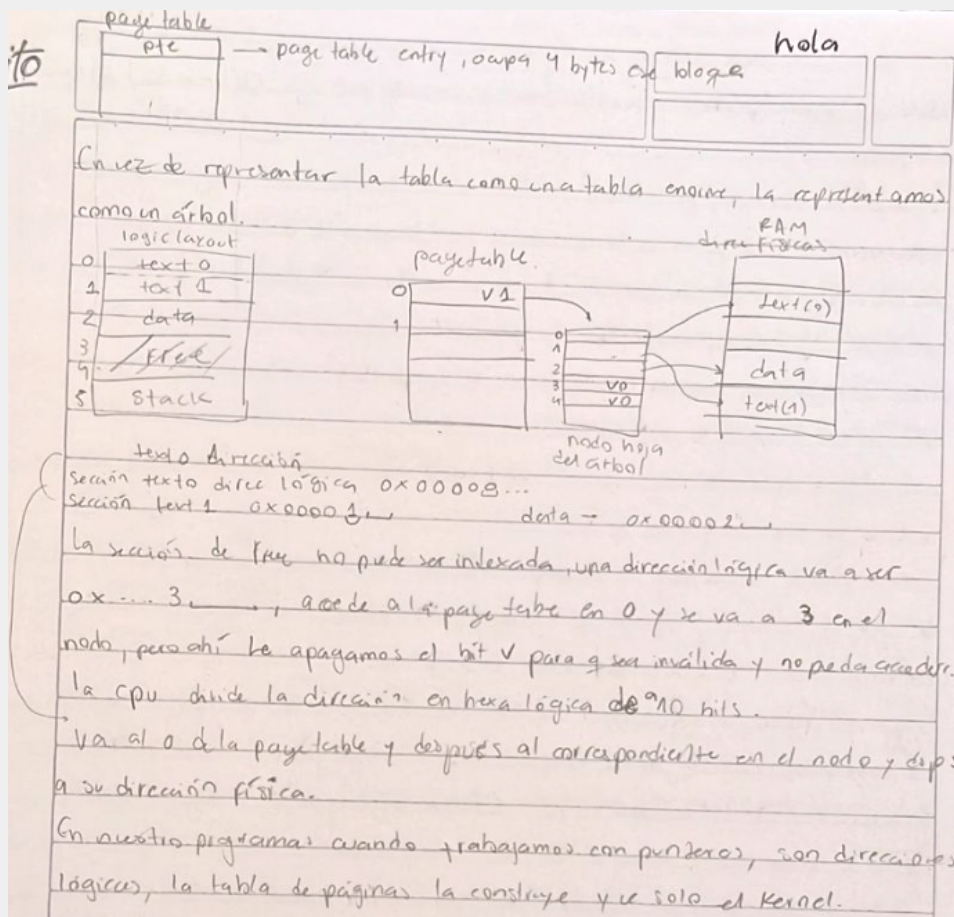
Los flags en una pte tienen los siguientes bits:

1. **V**: Valid bit. Indica si la entrada es válida.
2. **D**: Dirty bit. Encendido por el hardware cuando la página correspondiente fue modificada (escrita) por una instrucción.
3. **A**: Accessed bit. Encendido por el hardware cuando una página fue accedida para lectura o escritura.
4. **U**: User mode bit. Permiso de acceso en user mode.
5. Los bits **R,W,X** determinan permisos de lectura, escritura o ejecución de la página.
6. **G**: Global mapping. Esta entrada existe en todas las tablas.

En el proceso de traducción de una dirección lógica a una física, la cpu verifica los correspondientes flags. En caso de una violación, por ejemplo, si $V=0$ genera un page fault. El kernel podrá atrapar estas excepciones y actuar en consecuencia.

En el capítulo memoria virtual se describen técnicas de gestión de memoria virtual que hacen uso de estos mecanismos.





Paginado en IA32:

Características

- Páginas de 4KB o 4MB
- Páginas de 4KB: Árbol de dos niveles
 1. Tablas de páginas (nodos) de 1024 entradas
 2. Directorio de páginas (raíz, apuntado por CR3)
- Uso con segmentación:
paging : linear address → physical address

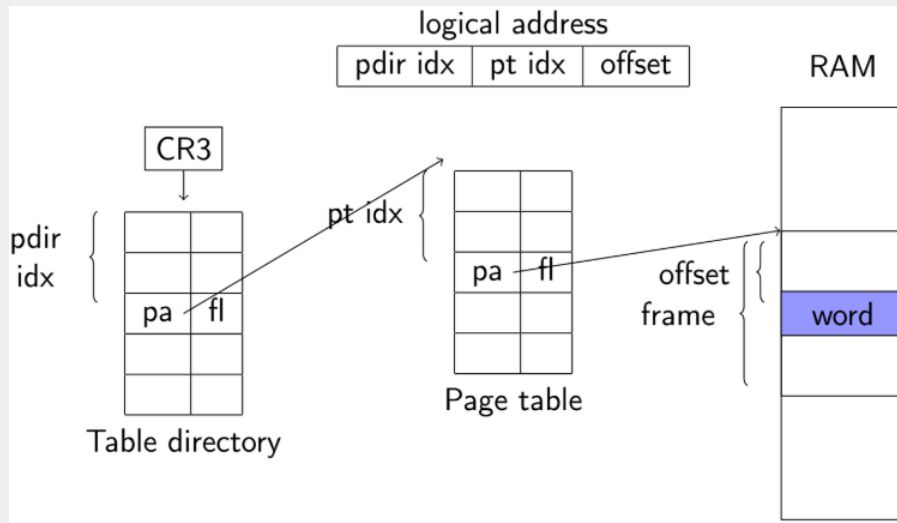
Detalles

- Dirección lógica (4kb):

pgdir idx (10 bits)	pgtbl idx (10 bits)	offset (12 bits)
---------------------	---------------------	------------------
- Entrada en tabla de página:

physical address (20 bits)	flags
----------------------------	-------

 Flags: **U**ser, **P**resent, **A**ccessed, **D**irty, ...



Paginado en RISC-V:

- El registro Supervisor Address Translation and Protection (SATP) apunta a la tabla de páginas (raíz)
- Páginas de 4KB.
- El *riscv Sv39* las *direcciones virtuales* son de 39 bits

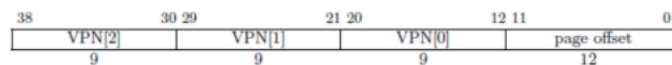


Figure 4.16: Sv39 virtual address.

VPN: Virtual Page Number: *index* en una tabla de páginas.

- Tabla de páginas: Árbol de 3 niveles
- Cada nodo del árbol es una tabla de 512 entradas (*pte*) de 64bits c/u

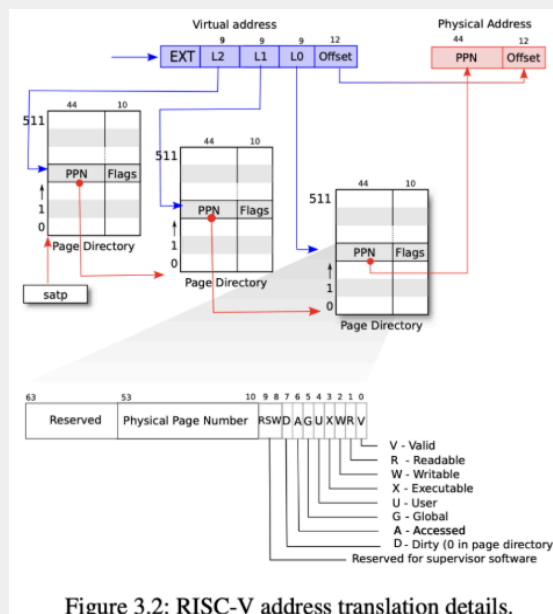


Figure 3.2: RISC-V address translation details.

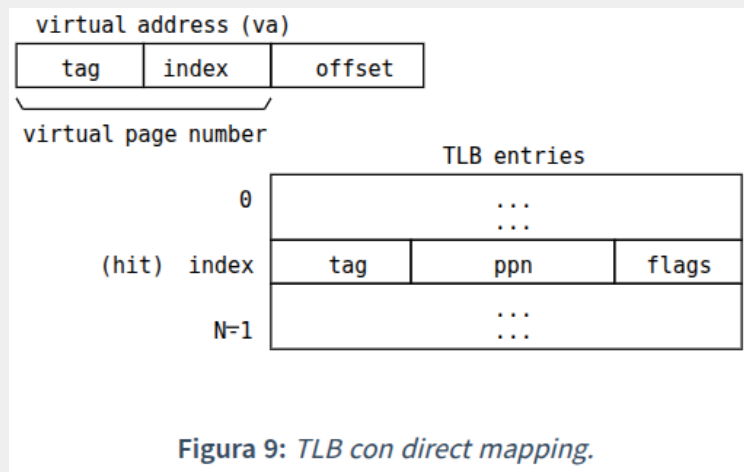
Lookaside translation buffers (TLB)

Un sistema con paginado traduce constantemente direcciones lógicas a físicas usando las tablas de páginas. Estas tablas están almacenadas en RAM. En este esquema cada acceso a memoria se debería multiplicar por 3: acceso a la pte raíz, pte de la hoja para finalmente acceder a la dirección física final.

Para evitar esta sobrecarga las cpu modernas usan una memoria caché especial para almacenar las ptes más frecuentemente referenciadas. Esta caché especial se conoce como lookaside translation buffers o TLBs. La TLB reside comúnmente entre la CPU y la caché de nivel 1 (L1).

Una TLB es comúnmente una memoria direccionable por contenido, es decir que busca una dirección lógica y si ésta existe (hit), el resultado es una dirección física, en otro caso ocurre un miss.

Generalmente implementan una función hash donde la clave de búsqueda son una parte de los bits menos significativos de la virtual page number y está asociada a un tag el cual es el conjunto de bits más significativo de la vpn. El par (tag,index) identifica una virtual page number. Esta organización se conoce como direct mapping y se muestra en el siguiente diagrama.



Cada TLB entry contiene el tag, el physical page number (ppn) y flags, entre los que se encuentra el bit valid (V) que indica si la entrada es válida o no.

En éste esquema, la TLB contendrá una de las páginas que coincidan en el index. Ante un miss con otra página coincidente en el mismo índice esa entrada será reemplazada con el tag correspondiente de la nueva dirección.

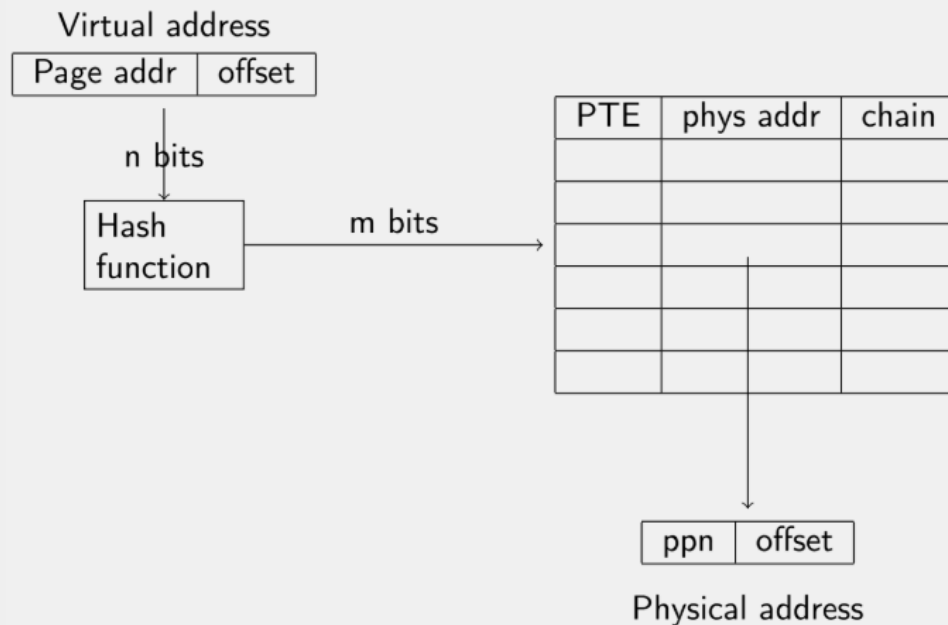
En el caso que $TLB[index].tag == va.tag$ y $TLB[index].flags \& BIT_V \neq 0$ ocurre un hit y $TLB[index].ppn$ contiene la physical page number.

En otro caso ocurre un miss y la cpu debe acceder a la RAM (o caché) a la tabla de páginas y obtener la ppn correspondiente. Luego, se setea $TLB[index]$ apropiadamente para futuras referencias.

Algunas arquitecturas, como MIPS, permiten que el kernel decida (por software) qué hacer ante un miss, generando una excepción (trap).

En los TLB cada referencia a memoria requiere acceso a tablas de páginas. Para evitar tantos accesos a memoria se requiere TLBs. Es una caché de alta velocidad de entradas de tablas de páginas (PTEs). Si una PTE no se encuentra en la TLB (cache miss) se carga desde la memoria. Generalmente se usa un algoritmo de reemplazo least recently used (LRU).

Implementación: memoria asociativa



Objetivos de la gestión de memoria:

- Uso eficiente
- Asignación de bloques de memoria requeridos por los procesos y el kernel.
- Protección: un proceso debe acceder a áreas de memoria permitidas y en el modo permitido.
- Gestión de la jerarquía de memoria (caché, RAM, buffers de E/S, . . .).

Memoria Virtual

Uso de dispositivos de almacenamiento secundario como extensión de la RAM para poder ejecutar procesos que requieran más RAM de la existente.

Objetivos y aplicaciones de la memoria virtual:

- Extender lógicamente la RAM en medios de almacenamiento (discos).
- Permite que un proceso vea más memoria que la físicamente disponible.
- Permite ejecutar programas parcialmente cargados en memoria.
- Carga de partes de procesos bajo demanda (mmap).
- Bloques no referenciados nunca se cargarán.
- Permite ejecutar más procesos de los que podrían albergarse en memoria.
- Mejores tiempos de respuesta (transferencias de E/S cortas).
- Copia de procesos (en `fork()`) bajo demanda (copy-on-write).
- Otros: bibliotecas compartidas . . .

La evolución tecnológica del hardware permite que las CPUs soporten grandes espacios de direcciones. Una cpu moderna puede soportar espacios direcciones de 64 bits (128 terabytes) o más. Esto permite el desarrollo de programas muy grandes. Comúnmente en un sistema moderno la cantidad de memoria RAM requerida por los procesos es mayor a la disponible físicamente por el hardware.

Un proceso no tiene por qué estar cargado en memoria completamente, sino sólo las partes en uso en un momento determinado. Con paginado es posible cargar (swap-in) o desalojar (swap-out) partes de un proceso con una granularidad del tamaño de páginas.

El swap-in se requiere cuando un proceso referencia a una página que reside en disco. El swap-out se requiere cuando el sistema requiere liberar una página en RAM para usarla ante la no existencia de páginas.

libres. Esta operación, comúnmente es invocada por el memory allocator. Este proceso se conoce como swapping o demand paging.

Para cada página se debe mantener su estado, es decir si está mapeada a una página física en RAM o en un bloque de disco. Esto comúnmente se hace almacenando la physical page number (ppn) o un disk block number (dbn) y el bit valid (o present en algunas arquitecturas) en 1 o cero, respectivamente.

Con paginado es posible extender la memoria RAM en dispositivos de almacenamiento con bloques del mismo tamaño como se muestra en la siguiente figura.

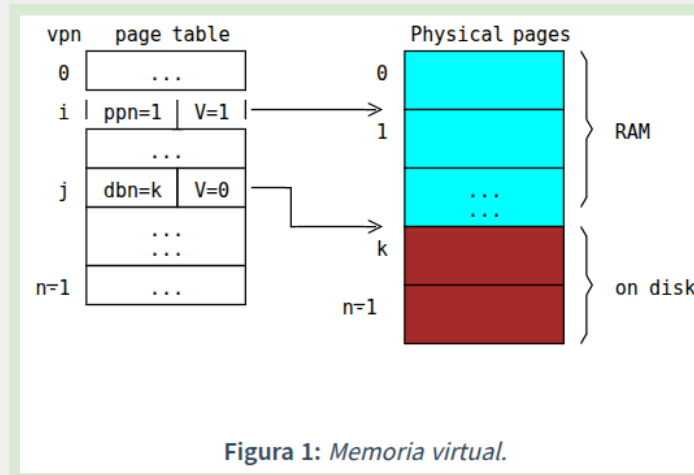


Figura 1: Memoria virtual.

En la figura anterior se muestra un esquema con un espacio de direcciones hasta n páginas con un hardware con una memoria principal física de k páginas de capacidad. Las páginas en disco se almacenan en un área conocida como swap area.

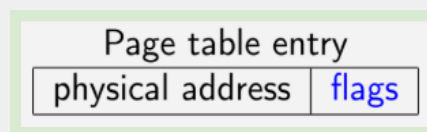
La página virtual j reside en disco, ya que la pte j tiene apagado el bit V (valid) pero contiene un disk block number.

Cuando una instrucción intente acceder a la página j se producirá una excepción. El kernel deberá hacer un swap-in:

1. Reservar una página en memoria (sea $ppn=X$).
2. Leer el contenido del bloque k en disco en la página X.
3. Mapear la pte j a X: Hacer $ppn=X$ y bit $V=1$

El swap area puede ser un archivo o una partición con el formato adecuado. Por ejemplo, GNU-Linux permite configurar el swapping area en un archivo o en una partición con formato del tipo swap. El uso de un sistema de archivos especial en lugar de un archivo mejora sustancialmente el rendimiento del sistema.

Mecanismos (permisos y flags):



- Flag V/P (valid/present) en cada entrada en tablas de páginas. Hardware causa page fault si bit V es cero en un acceso a la pte.
- Flag A (accessed): MMU setea este bit en cada acceso.
- Flag D (dirty): MMU setea este bit en cada escritura.
- Permisos de acceso (R,W,X): generan page faults en accesos indebidos.

El paginado nos da una granularidad más fina que la segmentación. Tienen el mismo objetivo, nada más que la diferencia está en la granularidad.

El offset nos permite movernos dentro de la página, porque el vpn nos lleva a la dirección base de la página, después nos tenemos que mover dentro de ella con el offset.

Cuando el SO necesita memoria y no tiene disponible para crear un nuevo proceso por ejemplo lo que hace es elegir alguna de las páginas que están siendo ocupadas por otro proceso y mandarlas a disco al área de swapping. Vemos a esta parte del disco como una extensión de la RAM. Hacemos un swap out (esto es el seleccionar una página que la vas a mandar a disco). Esa página que mandamos al disco, hay que anotarla para que cuando el proceso que usaba esa página y quiera acceder en el futuro, se produzca un page fault. Apagamos el bit V, el flag V y al estar apagado cuando el proceso quiera acceder a ella se va a producir el page fault. También tenemos que guardar en qué lugar del disco guardamos esa página para no perderla, esto lo podemos guardar en el campo ppn que antes tenía el número de página física en la RAM. Este proceso se conoce como swap out (todo lo que se hace en el `allocpage()`).

El swapping se puede deshabilitar pero si estás corriendo algo que te llena la memoria no te lo va a dejar correr.

Thrashing es cuando haces un swap out de una página pero después el proceso al que le sacamos la página se quiere ejecutar también entonces tienes que hacer otro swap out pero para eso tienes que buscar otra página que no la hay, entonces haces ese swap out. Y eso se repite y provoca que todo sea ineficiente.

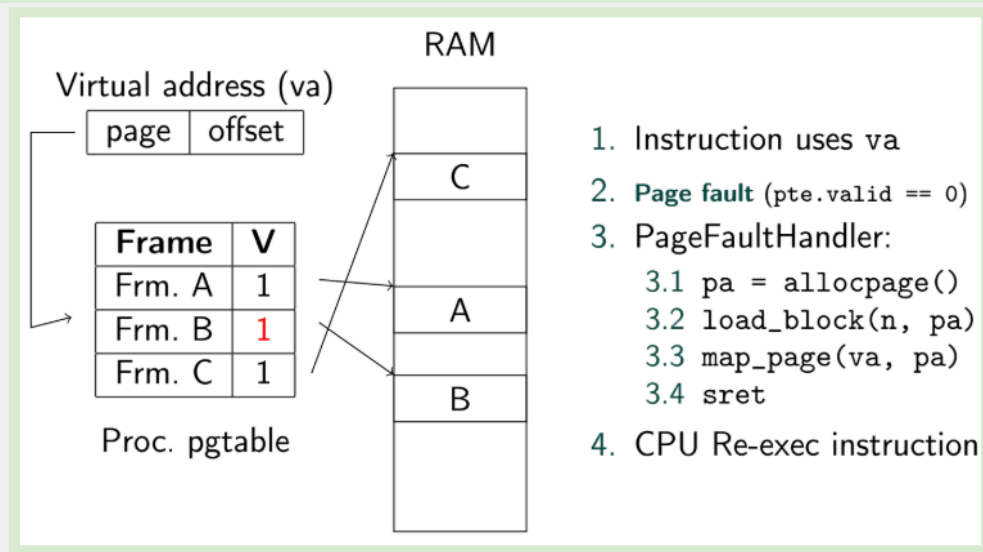
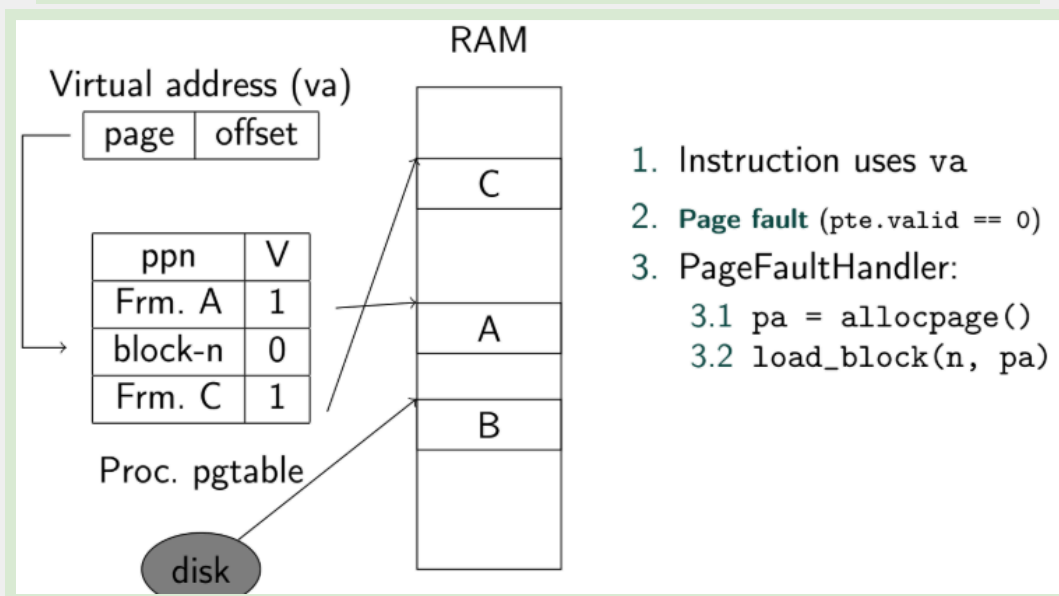
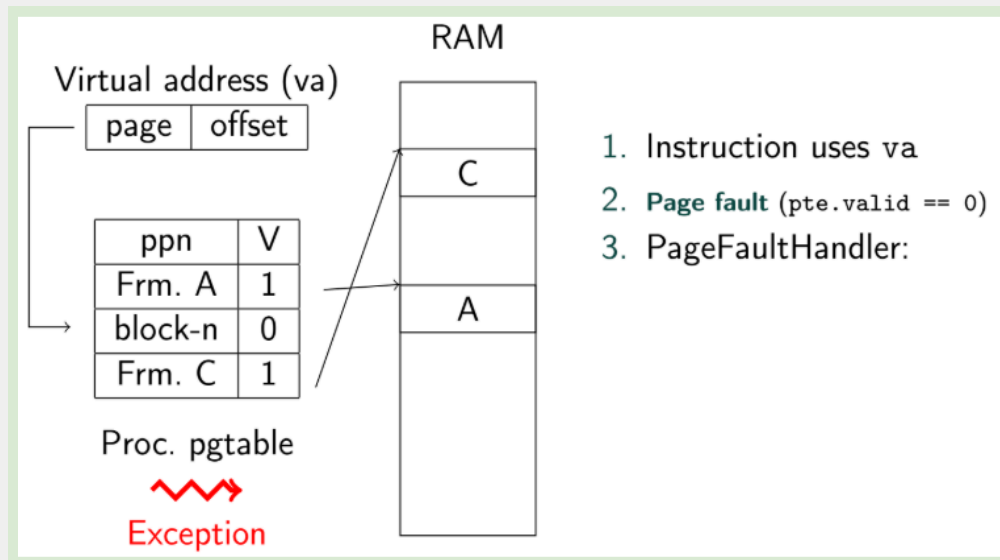
El swap in: viene a ejecutarse el proceso y ocurre un page fault porque el bit V está apagado. Se dispara un `pageFaultHandler` en donde se hace un `allocpage` (si hay una página libre joya y si no hay que mandar otra página al disco para reemplazarla por esta). Después se hace el `load_block`, se carga todos los datos del bloque de disco a la RAM y después se pone el bit V en 1. El número de página ahora tiene que apuntar a la dirección física de la RAM y no al número de bloque del disco (estas dos cosas es básicamente mapear la dirección virtual a la dirección física). Luego retornamos del trap este, porque ocurrió una interrupción. Retornamos a la instrucción que causó la page fault. La instrucción que falló la CPU la va a re ejecutar y ahora si va a tener éxito.

El swapping con segmentos es mucho más complicado porque no tengo que alocar una página, sino un segmento entero. El área del swap en el disco ya no puede ser el mismo de antes, vamos a necesitar un sistema de archivos con bloques de tamaño variable. Nos aparecen todos lo que vimos de fragmentación, etc. Con tamaño fijo es claramente mucho más fácil.

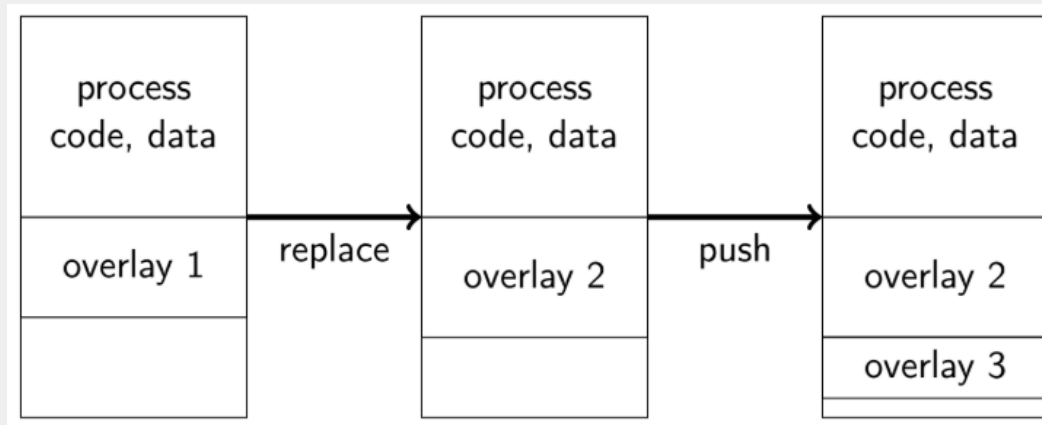
Swap out:

```
// free_list points to first page of free pages list
uint64 allocpage()
{
    if (free_list) {
        uint64 p = free_list;
        free_list = *free_list.next;
        return p;
    } else {
        // no free page. Choose one and swap out
        struct PTE * pte = select_victim();
        uint64 pa = pte.ppn * PG_SIZE;
        disk_block_no = write_to_disk(pte.ppn);
        clear_page(*pa);
        pte->v = 0;
        pte->ppn = disk_block_no;
        return pa;
    }
}
```

Swap-in:



Overlays: módulos cargables dinámicamente. Carga/descarga de módulos de un programa bajo demanda. Útil en programas con operaciones secuenciales. Ejemplo: `codegen(parse(src))`.



Mapeos de Archivos en Memoria

Una llamada al sistema del tipo `va = mmap(fd, mode)` crea un mapping de direcciones lógicas a los contenidos del archivo en disco y retorna la dirección lógica del comienzo (base) del contenido mapeado en la memoria.

El contenido del archivo se divide lógicamente en bloques del tamaño de una página. Mmap inicialmente no asigna memoria ni carga el contenido del archivo, sólo crea el espacio de direcciones (setea entradas en la tabla de páginas, indicando que están en los correspondientes bloques del disco).

La carga de los datos del archivo se hará en forma transparente por el SO bajo demanda, a medida que el proceso acceda a direcciones que pertenecen al espacio de memoria creado.

Esto permite el acceso a los datos del archivo accediendo directamente a memoria sin usar llamadas al sistema `read` o `write`.

En la primera referencia de la forma `va[i]`, donde `i` es un offset desde el comienzo del archivo, el sistema asignará una página de memoria y cargará en ella los contenidos de bloque correspondiente a esa página.

1. Reserva un espacio de direcciones virtuales en el proceso.

- No carga nada todavía.
- El sistema operativo actualiza la **tabla de páginas** con entradas especiales que dicen:
 - "Este rango de direcciones virtuales representa este archivo en disco."

2. No se carga a RAM todavía.

- Solo se reserva el espacio en la memoria virtual.
- No hay páginas físicas asignadas aún.

3. Cuando accedés a una dirección dentro del mapeo (ej: `va[i]`):

- La MMU no encuentra esa página cargada ➡ ocurre un **page fault**.
- El SO detecta que esa dirección corresponde a una página del archivo.
- **Carga el bloque correspondiente desde disco a RAM.**
- Actualiza la tabla de páginas para completar la traducción.

4. Después de eso, podés acceder al contenido normalmente como si fuera memoria RAM.

El siguiente programa muestra un ejemplo de su uso en un SO hipotético.

```

// let myfile.dat a file with contents with more than 4096 bytes as below.
// After mmap(), we can see it as a character array buf[] as
//
// buf --> +-----+ 0
//         |Hello world\0          |
//         |...                    | 4095
//         +-----+ 4096
//         |Second page\0         | ...
//         |padding (zeroes) by mmap|
//         +-----+ 8192
//
...
int fd = open("myfile.dat", O_RDONLY);
char *buf = mmap(fd);
printf("%s\n", buf);
// output: Hello world
printf("%s\n", buf + 4096);
// output: Second page
printf("%s\n", buf + 7005);
// output: empty string (zeroes in second page)
printf("%s\n", buf + 8192);
// page fault (maybe)
...

```

Listado 1: Ejemplo de uso de mmap.

Una llamada al sistema del tipo `unmap(va)` libera la memoria usada para los bloques del archivo y mapping. El mapping consiste en agregar pte's que referencien a números de bloques de disco en lugar a números de páginas de memoria. En algunos sistemas tipo UNIX el mapping se hace a partir de la dirección `brk` del proceso, la cual indica el comienzo de su bloque de espacios de direcciones libres. Comúnmente se setean flags de permisos de acceso pero con el bit `V` en cero como se muestra en la siguiente figura. Las páginas en sí en RAM para los datos del archivo no se reservan (`alloc`) inicialmente.

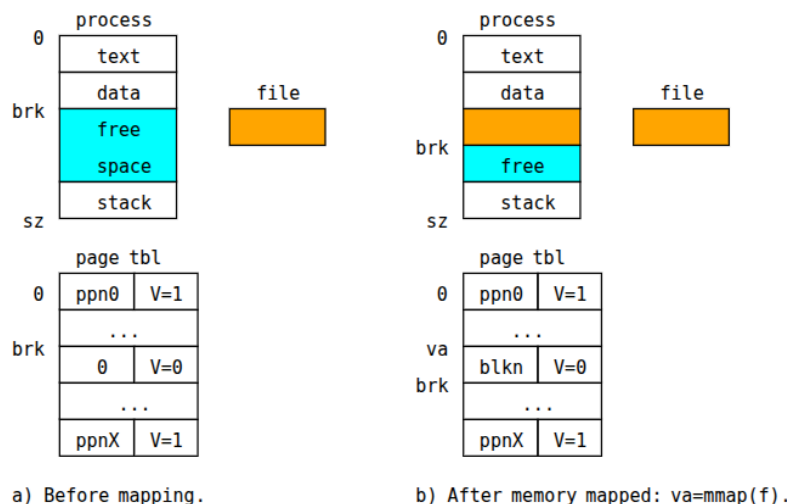


Figura 2: Ejemplo de mapeo de un archivo (de tamaño no mayor a una página) en memoria.

En la figura 2, $ppnX$ refiere a una physical page number X en memoria. El valor blk_n refiere al número de bloque (de igual tamaño que una página) del disco. La llamada al sistema `mmap` retorna `va` y avanza el puntero `brk` del proceso.

Esta configuración de la page table hace que cuando el programa acceda a la dirección que hace referencia a una página mapeada al archivo y ésta no está en la memoria, se produzca una excepción ya que la `pte` referenciada tendrá $V=0$.

El kernel podrá verificar si corresponde a una dirección lógica válida (el campo `ppn` no es cero) se deberá a que corresponde a una página que reside en disco y así podrá procesar la excepción:

1. Reservar (`alloc`) una página física. Sea si $ppn = \alpha$.
2. Cargar el bloque de disco `blk` en la página obtenida.
3. Mapearla: setear $pte.ppn = \alpha$ y los flags ($V=1$) en la `pte` correspondiente de la tabla de páginas.
4. Configurar la `cpu` para que reintente ejecutar la instrucción que causó la excepción.

Al retornar de la excepción la `cpu` podrá ejecutar exitosamente la instrucción que la causó y accederá al contenido de esa porción del archivo.

En el caso que el proceso modifique el contenido (copia) en memoria del archivo, al liberar el mapping, el SO puede escribir los datos en el archivo. Notar que el bit `dirty` indica qué página fue modificada.

Cuando un proceso cambia datos en una página que fue mapeada con `mmap()`, no modifica directamente el archivo en disco en ese instante, pero el sistema operativo marca esa página como `dirty` (sucia). Más adelante cuando el proceso haga un `unmap` o el proceso termine, escribe el contenido actualizado de esa página de vuelta al archivo en disco, solo si el mapeo lo permite (según los flags que se usaron). Y modifica la página entera, no solo una porción de esa página o bloque que fue modificado, el `mmap` no tiene forma de saber justo que se modificó para cambiar justo eso, actualiza la página entera y listo.

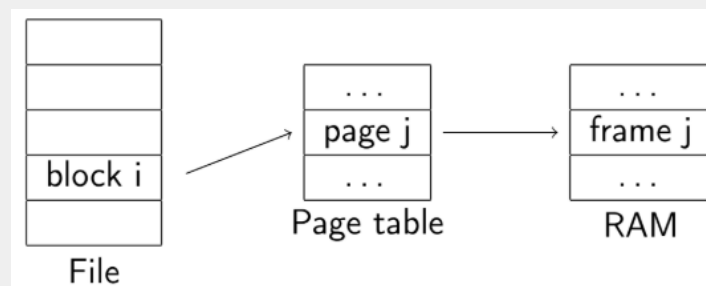
En las diapositivas:

{ I/O de archivos implícito para el programador.

Uso:

1. `int fd = open(filepath, ...);`
2. `char* ptr = mmap(..., size, flags, fd, ...);`
Mapea el archivo y retorna la dirección virtual del área de memoria mapeada (no aún `allocated`) para su contenido.
3. `char c = ptr[0];`
Esto causará un `page fault`. El OS cargará los bloques del disco a la página correspondiente (on demand allocation).
4. `c = ptr[5];` no causará un `page fault`, su contenido ya se encuentra en memoria.
5. `munmap(ptr, size);` libera el mapping, posiblemente escribiendo a disco los bloques modificados en la memoria.

Mapeo anónimo (`flag MAP_ANON`), reserva espacio de direcciones sin asociar páginas a bloques de archivos. }



1. El archivo se divide lógicamente en n páginas.
2. Se reservan n page table entries consecutivas con el bit $V=0$ con $PPN=file_block_number$.

3. La carga se hace bajo demanda (on page faults).
4. En `munmap()`, se escriben las (páginas) modificadas (bit D=1)

En vez de leer un archivo con un `read`, podemos hacer un `mmap`. Abrimos el archivo y la `mmap` mapea el archivo y nos devuelve el puntero que apunta a esa memoria. Al archivo lo divide como si fueran páginas lógicas y a este archivo lo vamos a ver como una extensión de la RAM.

Cargamos el archivo recién cuando un proceso lo quiere leer. Las páginas se van alocando bajo demanda.

Carga de programas bajo demanda

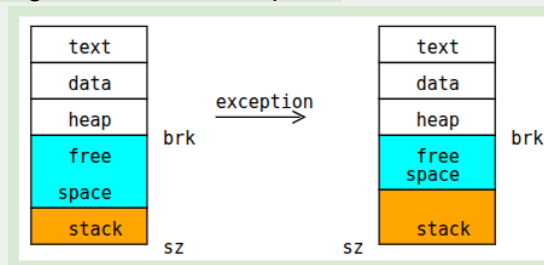
La llamada al sistema `exec(program-file, args)` debe cargar los contenidos del ejecutable en disco a memoria. Si el programa es grande, esto puede requerir tiempo y memoria.

Si `exec` mapea los segmentos cargables del ejecutable en memoria con `mmap` y eventualmente cargando sólo la página de código que contiene el entry point es posible realizar la carga del programa bajo demanda, a medida que la ejecución del proceso progrese y vaya referenciando direcciones de instrucciones y datos.

Stack con crecimiento dinámico

El problema a resolver en la gestión de la memoria es determinar cuánta memoria se debería asignar para el stack.

Con memoria virtual, podría mapearse inicialmente sólo una página en una dirección lógica alta y el SO podría asignar y mapear más páginas bajo demanda, es decir a medida que el proceso intenta escribir sobre el tope del espacio del stack asignado que generará una excepción.



De esta manera el stack crece bajo demanda como se muestra en la figura de arriba. El heap generalmente se expande hacia abajo asignándole más espacio usando la llamada al sistema `sbrk(size)`.

Memoria compartida

Para que dos o más procesos o el kernel y los procesos puedan comunicarse usando la memoria se requiere que uno o más bloques de memoria físicos pertenezcan a sus espacios de direcciones. Esto se logra mapeando en cada proceso un espacio de direcciones lógicas que resuelvan a las mismas direcciones físicas.

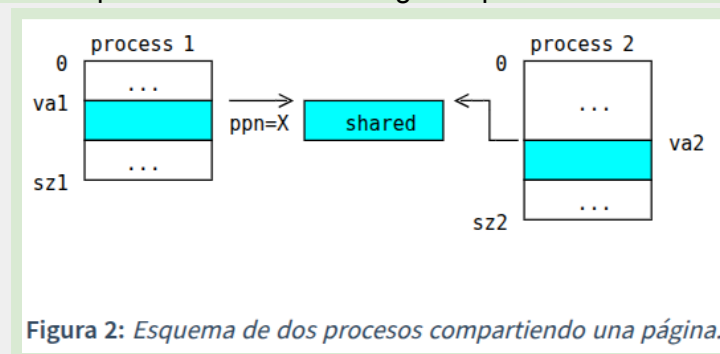


Figura 2: Esquema de dos procesos compartiendo una página.

En la figura 2 dos procesos comparten una página. Las direcciones lógicas va1 y va2 comúnmente son diferentes pero ambas están mapeadas al mismo número de página física (ppn X).

Los SO tipo UNIX, proveen llamadas al sistema para que los procesos puedan compartir memoria.

- `int shmget(key, size, flags)`: crea u obtiene un descriptor para un bloque de memoria compartida con identificador key de tamaño size.
- `void *shmat(m)`: obtiene la dirección lógica del descriptor m retornado por shmget.
- `int shmdt(address)`: libera (detach) el mapping.

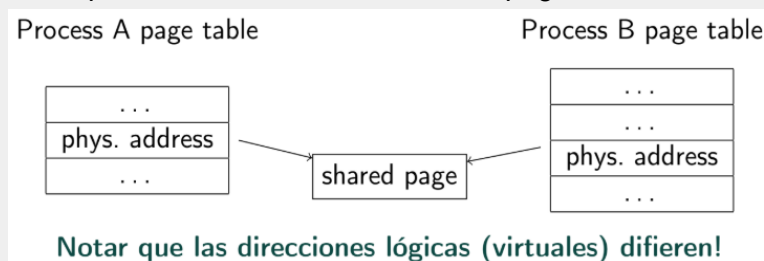
Obviamente, los procesos que se comuniquen mediante memoria compartida deberán usar algún mecanismo de sincronización (como semáforos) para evitar condiciones de carrera en el acceso a los datos o recursos compartidos.

Esta técnica junto con el mapeo de archivos en memoria se usa para mapear una biblioteca compartida en el espacio de direcciones de un proceso.

Esto complica la liberación de memoria de un proceso, por ejemplo en un `exit()` ya que hay que liberar sólo páginas no compartidas.

Una solución a esto es asociar a cada shared memory region un contador de sharing (o referencias), indicando el número de procesos que la comparten. Este contador se incrementa en un `va=shmat(m)` y se decrementa en cada `shmdt(va)`. Cuando llegue a cero, es seguro liberar la memoria compartida.

Tablas de páginas de diferentes procesos describen las mismas páginas.



Como podemos hacer para que dos procesos compartan el mismo espacio de memoria? Una página por ejemplo. No queremos que cada uno tenga su copia, que tengan memoria compartida. Podemos hacer que cada proceso en su page table, que en la entrada necesaria en cada una de estas tablas apunten a la misma dirección física. Dos procesos con diferentes memorias lógicas pueden acceder a la misma dirección física.

Copy On Write (COW):

La llamada al sistema `fork()` debe copiar la imagen del proceso padre en el hijo. Esta duplicación puede requerir altas demandas de memoria y copias de grandes cantidades de bloques.

En lugar de hacer la copia completa se copia la tabla de páginas, haciendo que ambos procesos compartan la memoria. Para lograr el efecto de confinamiento de cada proceso, en ambas tablas de páginas se quitan los permisos de escritura en cada pte, quedando la memoria compartida de sólo lectura. A cada proceso debería asociarse una shared memory block para todo su espacio de direcciones.

Cuando uno de los dos procesos desea escribir en una página (sea la virtual page number ppn) se producirá una excepción ya que no tiene permiso de acceso para escritura. El kernel captura la excepción, reconoce que se produce por la falta de permisos pero es una dirección válida (y shared). Entonces hace los siguientes pasos:

1. Asigna (alloc) una nueva página. Sea una con `ppn=n`.
2. Copia el contenido de la página original en la nueva.
3. Mapea la nueva página: `ppe.ppn=n` y `ppe.W=1` en la ppe correspondiente del proceso corriente.

4. Hace $W=1$ en la pte correspondiente del proceso padre o hijo, según corresponda.

Como en cada excepción recuperable, se debe preparar la cpu para que re-intente la ejecución de la instrucción que causó la falta en el proceso.

Este mecanismo se conoce como copy on write (COW). El paso 4 no es aplicable en múltiples forks ya que pueden haber otros procesos hijos con esa página aún compartida y el padre podría modificarla. Esto comúnmente funciona muy bien en las secuencias fork-exec.

Para generalizar COW se necesita llevar la pista de las páginas compartidas entre procesos, comúnmente usando shared memory regions.

1. Proceso padre

2. **fork()**

Page	W
A	0
B	1
C	1



pagetable proceso padre

1. Proceso padre

2. **fork()**

2.1 $W = 0$ a pte's

Page	W
A	0
B	0
C	0



pagetable proceso padre

pagetable proceso hijo

1. Proceso padre

2. **fork()**

2.1 $W = 0$ a pte's

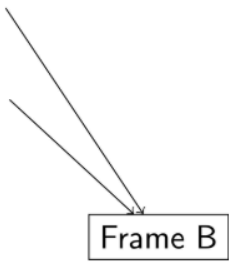
2.2 Copiar pagetable

3. Proc. hijo intenta escribir en frame B

3.1 **Page fault**

3.2 PageFaultHandler:

Page	W
A	0
B	0
C	0



pagetable proceso padre

1. Proceso padre
2. **fork()**
 - 2.1 $W = 0$ a pte's
 - 2.2 Copiar pagetable
3. Proc. hijo intenta escribir en frame B
 - 3.1 **Page fault**
 - 3.2 PageFaultHandler:
 - 3.2.1 Copiar frame

pagetable proceso hijo

Page	W
A	0
B	0
C	0

Frame B'

Frame B

pagetable proceso padre

Page	W
A	0
B	0
C	0

1. Proceso padre
2. **fork()**
 - 2.1 $W = 0$ a pte's
 - 2.2 Copiar pagetable
3. Proc. hijo intenta escribir en frame B
 - 3.1 **Page fault**
 - 3.2 PageFaultHandler:
 - 3.2.1 Copiar frame
 - 3.2.2 Mapear frame en el hijo
 - 3.2.3 $W = 1$ en ambas tablas

pagetable proceso hijo

Page	W
A	0
B'	1
C	0

Frame B'

Frame B

pagetable proceso padre

Page	W
A	0
B	1
C	0

Vamos a tratar ahora de que el SO sea perezoso, vamos a demorar las cosas todo lo posible. Si hacemos un fork y solamente copiamos la tabla de páginas, van a compartir toda la memoria entre padre e hijo, la memoria del hijo debería ser distinta a la del padre. Pero si ambos procesos únicamente leen, te sirve porque no va a haber un problema al no escribir nunca en la memoria ninguno de ellos. Esto no pasa nunca en la práctica, pero si pasa que hay áreas de la memoria en donde no se tocan nunca por ninguno de los dos, como lo es el código, las instrucciones. Lo que hacemos en copy on write, no copiamos la memoria sino que hacemos que la compartan y si uno de los procesos quiere escribir, ahí se hace la copia de la memoria para que cada uno tenga su propia copia. Se copia ante una escritura, antes de que se escriba. El kernel debe tomar el control para hacer la copia cuando alguien quiere escribir.

Si alguno quiere escribir, no te vamos a dejar, primero vamos a copiar esa página para que el que quiere escribir escriba en la copia y no en la página compartida.

Lo que podemos hacer para implementar esto es que todas las entradas de la tabla de páginas que comparen le quitamos el permiso de escritura con el bit de escritura w. Si alguno quiere escribir se produce una excepción, pero el bit V está prendido porque es válida, pero no el w. Entonces se le copia la página esa y se le prende el bit w a ambas, a la original y la nueva. Van a tener todo igual compartido menos esa página que se copió. Retornamos la instrucción y se ejecuta después bien.

Todo esto hace que los tiempos de respuesta sean mucho más rápidos, el fork va a ser más rápido. El proceso hijo va a usar la tabla de páginas del padre pero se le apagan todos los bits de escritura a las páginas.

Cuando programamos con punteros por ejemplo en un programa, esos punteros son direcciones lógicas, el kernel es el que construye la tabla de páginas y solo él la ve.

La tabla de páginas se hace para traducir y para proteger por si se quiere acceder a una dirección inválida.

Algoritmos de reemplazo de páginas

Cuando el sistema requiere asignar memoria (ej: `allocpage()`) ya sea para un proceso o para el kernel, es posible que no exista ninguna página libre. Esta situación se conoce como page fault.

Con swapping, el sistema podrá elegir una página (víctima) en uso, desalojarla a disco (swap-out) y asignarla para satisfacer el requerimiento.

Uno de los problemas a resolver es determinar cómo elegir la víctima. Lo ideal sería que ésta fuese una página que será referenciada más en el futuro (eventualmente nunca). Esto obviamente no es computable ya que requiere predecir el futuro.

Ya se ha discutido en estas notas que una forma de intentar predecir el comportamiento futuro es hacerlo en base al comportamiento pasado. Los siguientes algoritmos intentan seleccionar la víctima más adecuada en base a diferentes heurísticas.

Estos algoritmos pueden usarse en gestión de RAM (paginado) o de caché.

1. No recientemente usada (NRU)

Este algoritmo selecciona una página a reemplazar en base al soporte del hardware descripto. Se basa en determinar si las páginas son accedidas en base a períodos de tiempo. Periódicamente, en cada tick del reloj por ejemplo, el bit A (accessed) se pone en cero (cleared). Periódicamente apagar el bit *referenced* en cada pte. Por ejemplo, Bit R en x86 o el bit A en risc-v.

Ante un page fault, cada página pertenece a una de las siguientes clases: (ante una falta de página, la víctima a elegir se selecciona mediante el siguiente criterio)

1. $(A, D) = (0, 0)$: no accedida en el último período.
2. $(A, D) = (0, 1)$: no accedida en el último periodo, escrita en el período anterior ($A=0$ es producto del clearing periódico).
3. $(A, D) = (1, 0)$: leída en el último período, accedida recientemente.
4. $(A, D) = (1, 1)$: accedida en el último período, escrita en algún momento. Accedida recientemente para escritura.

El algoritmo NRU elige una página que pertenezca a la clase menor posible. El algoritmo selecciona una página (aleatoriamente) de la categoría más baja. Es una versión LRU hecha por software. El algoritmo se dispara periódicamente y la idea es categorizar páginas para elegir las. Primero se elige una víctima de la categoría 1, si no hay ninguna se elige una de la categoría 2 y así sucesivamente.

2. FIFO

El sistema mantiene una cola de las páginas en uso con la última asignada como último elemento. En un page fault el algoritmo FIFO selecciona la página a la cabeza y se encola. Esta página es la que se ha desalojado hace más tiempo.

No ofrece buen rendimiento. Anomalía de Belady: la tasa de faltas de página se incrementa al aumentar la capacidad de memoria (número de páginas). Se demostró que este algoritmo no funciona muy bien. No respeta la medida de que si se agranda la memoria, la cantidad de page faults no disminuye y esto es algo que debería pasar si el algoritmo es bueno.

Se mantiene una cola con las páginas que van siendo referenciadas. La página que este al principio va a ser una página que ha sido referenciada antes que el resto.

Tiene un problema: si tenemos una página que fue referenciada y está al principio de la cola, va a ser la primera en ser elegida pero si sigue siendo referenciada, no se va a agregar un duplicado en la cola, sigue ahí pero está mal porque está en la cabeza de la cola y fue la última en ser referenciada.

3. Second chance

El principal problema del algoritmo FIFO es que puede elegir una página usada muy frecuentemente. Este algoritmo se basa en FIFO con una simple modificación.

En lugar de asignar la página de la cabeza, se inspecciona si fue accedida (bit $A=1$). Si es así, se hace $A=0$ y se encola la página (se mueve al final). Este proceso continúa hasta encontrar una página a la cabeza con $A=0$.

Este algoritmo selecciona una página que no ha sido víctima hace más tiempo que no fue accedida desde el último período o page fault.

Una variante de este algoritmo es no usar una cola, sino una lista circular de las páginas usadas con un índice o referencia: el hand (la mano) que apunta a la página más vieja (no seleccionada como víctima).

En un page fault, se verifica el valor del bit A . Si es cero, se elige como víctima, sino se hace $A=0$ y se avanza el hand repitiendo el proceso.

Esta implementación se conoce como el algoritmo del reloj **clock**.

Según las slides:

1. Si la cabeza fue accedida (bit $A = 1$), se pone $A = 0$ y se la mueve a la cola.
2. Si no fue accedida, se selecciona como víctima.
3. Reloj: variante que usa un puntero (hand) del último elemento examinado. Esto implementa una cola circular.

Este algoritmo se corre periódicamente, cada tantos ticks se dispara. Es una solución del FIFO que soluciona el problema que tiene.

4. Menos recientemente usada (LRU)

Este algoritmo tiene como objetivo elegir como víctima una página que no ha sido usada en un lapso de tiempo determinado. Es un algoritmo basado en contadores. Requiere llevar el tiempo del último acceso a cada página. Difícil implementación sin soporte de hardware. La implementación de este algoritmo no es fácil de realizar eficientemente. Esto requiere mantener una lista de todas las páginas ordenadas por tiempo de acceso (de menor a mayor). Los tiempos de acceso pueden ser ticks. En cada tick se mueven las páginas accedidas al final de la lista.

La idea es elegir páginas que no han sido usadas por un tiempo considerable. Hay que llevar una noción de tiempo, un timestamp. Si el SO nos diera esto sería fácil de implementar pero no lo tiene, así que es bastante difícil implementarlo al algoritmo. El hardware no nos ayuda mucho para implementarlo.

Una implementación alternativa de LRU se conoce como not frequently used (NFU).

NFU asocia un contador a cada página, inicialmente en cero. Periódicamente, por ejemplo en cada interrupción de reloj, el SO analiza cada pte de cada tabla de páginas y suma el bit A al contador. Se mantiene una lista ordenada de contadores (de menor a mayor).

Ante un page fault, se selecciona la página a la cabeza de la lista. Los siguientes problemas son difíciles de resolver eficientemente:

1. Se deben recorrer todas las tablas de páginas periódicamente. Puede resolverse haciéndolo en momentos de ocio del sistema y en forma incremental.

2. Siempre el contador se incrementa, es decir nunca olvida. Puede resolverse haciendo un shift a derecha de un bit antes (dividir por dos) de sumarle el bit A. Esto implementa una idea de aging y trata de prevenir que páginas que han tenido muchos accesos en el pasado pero que actualmente no se acceden más o muy poco, vayan adelantándose en la lista.

Lo que hace es contar las veces que fue accedida la página.

Según las slides:

Algoritmo basado en contadores.

1. Asociamos un contador (counter) por cada página.
2. Periódicamente a cada $\text{counter} += R/2$.
3. En un page out, se selecciona la de menor valor de counter.

Problema: aging. Contamos números de accesos, no cuando ocurrieron. Posible solución:

$$\text{counter} = (\text{counter} \gg 1) \mid (R \ll (\text{countersize}-1))$$

5. Working set

La idea de este algoritmo se basa en que los procesos en un período de tiempo τ acceden a un conjunto de páginas. Ese conjunto de páginas recientemente accedidas conforman un working set del proceso. Es fácil de ver que un proceso en una iteración o si está accediendo a datos almacenados en forma contigua, referenciará muchas veces un conjunto relacionado de páginas.

El objetivo es registrar para cada página si ésta fue accedida en el intervalo de tiempo τ . Cada página se asocia a un reloj lógico (ticks) con el tiempo del último acceso (tick). Ante un page fault se recorre la tabla de páginas inspeccionando el bit A. Si $A=1$, se actualiza el valor del reloj asociado a la página y se agrega al working set del período corriente.

Si $A=0$ se deberá determinar si puede elegirse como víctima. Si el tiempo de acceso está dentro del intervalo actual ($\text{tick} - \text{last_access}_{\text{page}} < \tau$) se mantiene en el working set. Si no se considera fuera del working set. Se elige como víctima una página fuera del working set con menor valor de reloj.

Para evitar recorrer toda la tabla de páginas puede usarse una mejora basada en el algoritmo de reloj descripto arriba.

La idea es mantener cuál es el conjunto de páginas activas. Se puede hacer por procesos o para todo el sistema. Páginas contiguas probablemente estén en el mismo set. Tenemos un reloj lógico en base a periodos.

Working set ($ws(p)$)

Páginas referenciadas por el proceso p en el último período τ

1. Se asocia a cada página p el tiempo del último acceso ta_p
2. $ws(p)$ representado como una cola circular
3. Periódicamente (τ ticks) se recorre la cola. Sea la página p
 - 3.1 Si el bit $A = 1$: $ta_p = \tau$ y $A = 0$
 - 3.2 Si no, si $ta_p > \tau$, se elimina de $ws(p)$
Si $D = 1$ se planifica el swap-out
4. Se elige como víctima una página p con $ta_p > \tau$ y $D = 0$

El algoritmo ideal de elección de la víctima sería: aquella que se referenciará más adelante en el futuro (eventualmente nunca). Requiere predecir el futuro. Heurísticas que intentan acercarse al algoritmo ideal:

- Frecuencia o cantidad de accesos a cada página.
- Conjunto de trabajo (principios de localidad)

Otras consideraciones

Los algoritmos descriptos no consideran otros aspectos de selección de páginas a reemplazar como por ejemplo, su importancia en el sistema. Una consideración a tomar en cuenta es si el algoritmo de reemplazo de páginas debe ser global (sin tener en cuenta por quién está usada la página) o local (se consideran las páginas del proceso corriente, por ejemplo).

Por ejemplo, no es lo mismo reemplazar una página usada por el kernel o una biblioteca compartida que una usada por un proceso. Para evitar esto, comúnmente ciertas páginas se protegen o marcan de ser elegidas como víctimas. Otro ejemplo son las que pertenecen a espacios de direcciones de dispositivos mapeados en memoria (MMIO).

Uno de los peligros del page swapping es la sobre paginación (trashing), la cual es un estado en el que el sistema está ejecutando out of memory y ocurren repetidamente los siguientes eventos:

1. Se debe hacer un swap-out para liberar memoria.
2. La página debe volverse a cargar porque es referenciada.

Estos pasos se repiten a una alta frecuencia. El sistema comienza a degradar el rendimiento de manera considerable y el progreso de los procesos se hace muy lento.

Todas las páginas que no usa el kernel las podríamos elegir como víctimas, por ejemplo páginas que uso para la inicialización pero eso ya no lo va a usar más porque ya inicializó el sistema.

Manteniendo un pool de páginas libres podemos evitar que ocurra thrashing.

Caso de estudio: GNU-Linux

Linux implementa un algoritmo LRU aunque la lista de páginas no refleja exactamente un orden por tiempo de acceso. Mantiene dos listas: `active_list` e `inactive_list`. El objetivo es mantener en `active_list` un working set de todos los procesos e `inactive_list` la lista de candidatos a reemplazar. Estas listas se actualizan periódicamente. Esto hace que la política sea global.

NOTAS:

Estructura de la materia: 1 parcial más o menos a la mitad de la materia.

Bibliografía

1. G. Wolf, E. Ruiz, F. Bergero, E. Meza. [Fundamentos de Sistemas Operativos](#). 2015.
2. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. University of Wisconsin-Madison. *Operating Systems: Three Easy Pieces*. Free e-book: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
3. A. Tanenbaum, H. Bos. *Modern Operating Systems*. Fourth edition. Pearson. ISBN-10: 0-13-359162-X, ISBN-13: 978-0-13-359162-0. 2015.
4. Silberschatz, Galvin, Gagne. *Operating Systems Concepts*. 9th Ed. ISBN: 978-0-470-88920-6. 2011.
5. Prof. Edson Borin. Institute of Computing Unicamp. *An Introduction to Assembly Programming with RISC-V*. Url: <https://riscv-programming.org/book/riscv-book.html>

-
- El 1, dice que está bien como libro pero no le gusta mucho, el mejor es el 2. El 3 es el libro clásico. El 4 también es un muy buen libro. El 5 es el manual específico de la arquitectura que vamos a usar.
- Leer el libro (el del manual), es un libro de arquitectura.
- El libro que vamos a seguir es el de three easy pieces.
- El de fundamentos de sistemas operativos también está bueno para leerlo porque tiene conceptos con los que vamos a trabajar.
- El de modern operative system es EL libro de SO y recomienda leerlo también.

Arquitectura RISC-V (reduced instruction set computer)

- Risc-v virtIO emulado por QEMU: (esto buscarlo en las notas del curso, era una imagen con esa descripción)

Tenemos la CPU completa con varios cores adentro. Cada una de estas CPU tienen in clint (ahí adentro hay un reloj que genera interrupciones.). El PLIC es un chip por el que se conectan todos los dispositivos externos, uno de ellos es el UART y el disk. Los dispositivos generan interrupciones cuando terminan su operación, avisan cuando terminaron de ejecutar, por ejemplo leer en un disco y cuando termina se genera una interrupción. Esto se hace para que mientras lea en el disco, se ejecute otro proceso para que la CPU no quede ociosa, y después cuando termine lo del disco esto se avisa.

Los dispositivos nos van a aparecer en diferentes direcciones de memoria cuando programemos el SO.

- Distribución de la memoria en qemu-RV-39 (imagen, (esto buscarlo en las notas del curso, era una imagen con esa descripción): todas las posibles direcciones que podríamos llegar a tener en el memory bus es a lo que llamamos espacio de direcciones creó.

La ROM está a partir de la dirección 1000. El 80000000 es 2GB. En mapped devices se van a conectar los dispositivos como teclado, disco, etc. La RAM física está mapeada a partir del espacio de la memoria 80000000.

SLIDES DE RISC-V:

El libro que tiene todos los capítulos separados en online lo recomienda leer. El manual de risc-v también hay que leerlo. Y el mejor que recomienda es el de Modern Operating Systems de Tanenbaum.

Objdump -h
Objdump -d

Libros:

- **Arquitectura RISC-V:** “An Introduction to Assembly Programming with RISC-V”
<https://riscv-programming.org/book/riscv-book.html>
- **Operating Systems: Three Easy Pieces:** <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- **Modern Operating Systems: Tanenbaum**
https://drive.google.com/file/d/1zNGyuw5pzyTm8kxPxNn_rn1NDIb6XfrQ/view

NOTAS:

sepc: valor del program counter cuando ocurrió la interrupción.

Necesitamos bloquear el semáforo porque el mismo semáforo es un recurso compartido, sino se bloqueara tendríamos una race condition sobre el semáforo en sí mismo.

En los eosteps la memoria RAM empieza a partir de la dirección 0x80000000 (2GB), y lo primero que hay ahí es el código del kernel.

Tasks[i].wait_condition == &ticks.

- La CPU inmediatamente que ocurre una interrupción deshabilita las interrupciones.
- Si tienes tres tareas A, B y C y dormis la B por 5 ticks y recién se despierta en el tick 9 y eso es porque si se despertó esa tarea B, pasó a ser RUNNABLE pero la CPU no la eligió para ejecutarla porque la CPU eligió a la task C o A para correrla.
- Hay algoritmos para elegir una víctima. La mejor página como víctima sería una página que no se vaya a usar más.
- Para hacer todas estas cosas que vimos hoy de memoria virtual usamos la tabla de páginas. Siempre jugamos con el mismo mecanismo.

NUEVAS NOTAS

El paginado sirve para la protección de la memoria de cada proceso y provoca que sea más eficiente y hagamos un mejor uso de la memoria. Nos deja aprovechar mejor la memoria.

La protección la tenemos en la traducción de va a pa y los permisos que le damos a ciertas cosas. Tenemos otros bits también de información que nos brinda el hardware y sirven de protección también.

mmap de linux nos permite mapear parte del archivo, es más flexible.

El alloc page nunca va a fallar porque tenemos algoritmos de reemplazo de páginas, en donde agarran una usada y se la quita. Para sacar una página hacemos lo siguiente: en la entrada de la página le ponemos el bloque del disco en donde está y le apagamos el bit V.

COW: evitamos que en el fork se tenga que hacer una copia entera de la imagen de código del padre. Se copia bajo demanda.

Sobre la task 6:

Preemptive: las tareas corren normalmente y el cambio de contexto se hace automáticamente, no tienen que hacer yield explícito sino que el sistema los va a hacer mediante un sistema de interrupciones. Vamos a necesitar para esto un reloj.

Todo lo que tenemos en los archivos de arch son de la arquitectura.

Ahora vamos a guardar todos los registros de la cpu no como antes que guardabamos algunos nomas.

Packed o empaquetado significa que no hagas padding en la memoria.

Vamos a tener que setear el vector de interrupciones.

Cuando hablemos de controller vamos a estar hablando del hardware, cuando hablemos de driver vamos a estar hablando de software, de un pedazo de código.

CLINT: core local interrupt controller

Cada CPU tiene un clint. Este CLINT tiene un registro MTIME (de 64 bits) que se va aumentando en cada ciclo del reloj. Por cada core tenemos también un MTIME_CMP (un comparador).

Con 32 bits haría overflow muy rápido el MTIME, por eso es de 64.

Cuando ocurre una interrupción se interrumpe la ejecución y la CPU salta a una rutina que maneja esa interrupción que es la trap handler. El registro mtvec apunta a una rutina m_trap (esto es para modo machine) que retorna del modo machine con un mret.

Tenemos después stvec que apunta a una rutina s_trap (en el modo supervisor.) que retorna con un sret.

Si ocurre una interrupción de reloj, siempre se salta a la m_trap, todas las demás van a la del modo supervisor. Las interrupciones de reloj no se delegan, la arquitectura delega todas las interrupciones menos las interrupciones de reloj. Por eso necesitamos un trap handler en modo machine, hay que atraparlas en modo machine. Y en la m_trap en realidad lo mandamos a la s_trap para no tener que andar repitiendo código.

En la implementación de la m_trap se guardan los registros a0-a7 porque el next_timer_interrupt los usa si te fijas en el dissesamble.

Para la s_trap:

En el sepc se guarda el valor del pc. La cpu cuando ocurrió una interrupción lo primero que hace la cpu es guardar el pc ahí en el sepc.

En s_trap se guardan 30 registros (no 31 porque en uno esta el tp y no se por que el 32). El sret va a retornar al código de la tarea que fue interrumpida, lo que está en el sepc así la tarea continua desde donde

había sido interrumpida (la continuamos exactamente desde el estado en donde había dejado de ejecutarse por la interrupción).

En el registro `scause` se guarda que paso cuando se ejecuta una interrupción (para saber la causa).

En la función de la trap solo aumentamos el ticks para la cpu 0 de manera random, podría ser otra pero se hace eso porque sino se aumentaron ticks por todas las cpu que hay. Se hace un `act_timer_interrump` para apagar un bit que determina que la interrupción esa ya se trato, sino va a saltar de vuelta.

Si se quiere ejecutar una tarea pero se quedó sin memoria en su stack, se produce una excepción entonces el kernel le va a asignar más memoria para que se pueda ejecutar de vuelta asignándole una página más. Luego se va a tener que ejecutar de vuelta esa tarea porque nunca se pudo ejecutar por la excepción pero como el pc quedo apuntando a la instrucción siguiente, se tiene que volver a la tarea está restandole 4 (-4) .

PREGUNTA DE PARCIAL: el kernel vuelve a tomar el control luego de haber booteado cuando ocurra una interrupción, toma el control cada vez que ocurra una interrupción.